

Fase 0 — Planeación y Diseño del Laboratorio de Bases de Datos Distribuidas

Laboratorio académico construido sobre **una sola computadora física**, con **Oracle VirtualBox** y **Ubuntu Server LTS + MariaDB**. Sin Docker, sin Kubernetes.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Explicar qué es un sistema de bases de datos distribuidas y diferenciarlo de un sistema centralizado.
 2. Identificar los temas que se cubrirán a lo largo del laboratorio (replicación física, lógica, fragmentación horizontal/vertical/híbrida, consultas distribuidas, transparencia, tolerancia a fallos).
 3. Diseñar la arquitectura objetivo final del laboratorio antes de tocar una sola máquina virtual.
 4. Calcular los requisitos de hardware del equipo anfitrión (host) necesarios para soportar el crecimiento gradual del laboratorio.
 5. Definir una convención de nombres, un esquema de direccionamiento IP y un plan de crecimiento por fases.
 6. Justificar por qué la planeación previa reduce el retrabajo en fases posteriores.
-

B. Conceptos teóricos necesarios

1. Sistema de bases de datos distribuidas (SBDD). Conjunto de nodos interconectados, cada uno con su propio motor de base de datos, que se presentan ante el usuario como si fueran un único sistema lógico.

2. Replicación física vs. lógica.

- *Física*: se replican los cambios a nivel de bloques/eventos binarios del motor de almacenamiento (en MariaDB, vía binlog row-based o las posiciones GTID). Es la base de la alta disponibilidad clásica maestro-esclavo.
- *Lógica*: se replican declaraciones SQL o eventos lógicos interpretables (statement-based replication, o herramientas de replicación multi-maestro como Galera). Permite topologías más flexibles (multi-maestro, bidireccional).

3. Fragmentación (particionamiento de datos).

- *Horizontal*: se dividen las filas de una tabla entre varios nodos (ej. clientes del norte en un nodo, del sur en otro).
- *Vertical*: se dividen las columnas de una tabla entre varios nodos (ej. datos personales en un nodo, datos financieros en otro).
- *Híbrida*: combinación de ambas estrategias.

4. Transparencia de distribución. Capacidad del sistema de ocultar al usuario/aplicación el hecho de que los datos están distribuidos: transparencia de fragmentación, de ubicación, de replicación y de concurrencia.

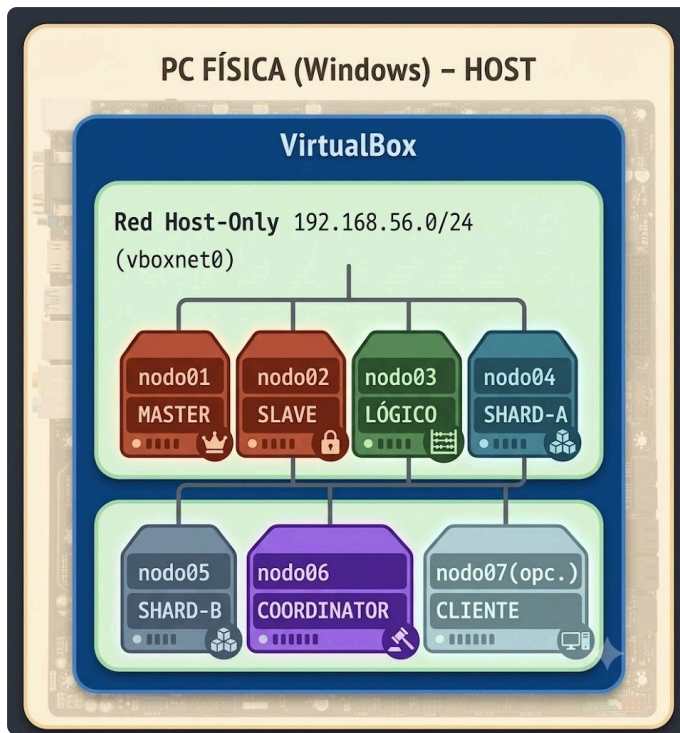
5. Teorema CAP. En presencia de una partición de red (P), un sistema distribuido debe elegir entre Consistencia © y Disponibilidad (A). Este laboratorio nos permitirá *observar* este compromiso de forma práctica, no solo teórica.

6. Tolerancia a fallos y recuperación. Capacidad del sistema de seguir operando (o de recuperarse limpiamente) ante la caída de uno o varios nodos.

C. Arquitectura utilizada

C.1 Arquitectura física (la que existe hoy)

Una sola laptop/PC con Windows ejecuta VirtualBox. Cada “nodo” del sistema distribuido es, en realidad, una máquina virtual Ubuntu Server dentro de ese mismo equipo. Una red virtual interna (Host-Only) interconecta las VMs entre sí y con el host, simulando una red de centro de datos.



C.2 Arquitectura lógica objetivo (a dónde llegaremos, gradualmente)

Nodo	Hostname propuesto	Rol funcional (fase en que se activa)
1	<code>bdd-nodo01</code>	Maestro de replicación física (Fase 10)
2	<code>bdd-nodo02</code>	Esclavo de replicación física (Fase 10)
3	<code>bdd-nodo03</code>	Nodo de replicación lógica / multi-maestro (Fase 11)
4	<code>bdd-nodo04</code>	Fragmento horizontal A (Fase 13)
5	<code>bdd-nodo05</code>	Fragmento horizontal B (Fase 13)
6	<code>bdd-nodo06</code>	Coordinador de consultas distribuidas / fragmentación vertical-híbrida (Fases 14-16)
7 (opcional)	<code>bdd-cliente</code>	Estación cliente externa para pruebas de transparencia (Fase 16-17)

Importante: no crearemos los 7 nodos ahora. Esta tabla es el mapa final. El laboratorio crecerá de 2 nodos (Fase 3) hasta 6-7 nodos (Fase 16), apagando con snapshots los nodos que no se usen en cada sesión para ahorrar recursos del host.

C.3 Plan de direccionamiento IP (reservado para cuando configuremos la red)

Nodo	IP planeada	Tipo de red
Host (Windows)	192.168.56.1	Host-Only Adapter (vboxnet0)
bdd-nodo01	192.168.56.101	Estática
bdd-nodo02	192.168.56.102	Estática
bdd-nodo03	192.168.56.103	Estática
bdd-nodo04	192.168.56.104	Estática
bdd-nodo05	192.168.56.105	Estática
bdd-nodo06	192.168.56.106	Estática
bdd-cliente	192.168.56.107	Estática

Se elige **Host-Only Adapter** (no NAT, no Bridged) porque: aísla el laboratorio de la red física/Wi-Fi real (seguridad y evita interferir con la red del salón), y permite que el host (Windows) y todas las VMs se vean entre sí, lo cual facilita administración con SSH, DBeaver, etc. Esta decisión se justificará a fondo en la Fase 2.

C.4 Dimensionamiento de recursos por nodo (referencia para Fase 3)

Nodo	vCPU	RAM	Disco (dinámico)
bdd-nodo01 / 02 / 03 / 06	1	1536 MB	20 GB
bdd-nodo04 / 05	1	1024 MB	15 GB
bdd-cliente	1	1024 MB	10 GB

Total si todos los nodos corrieran simultáneamente: ~7 vCPU, ~9.25 GB RAM, ~120 GB disco (asignación dinámica, no se ocupa todo de inmediato).

D. Procedimiento paso a paso

Paso 1 — Verificar que el equipo host cumple los requisitos mínimos.

Antes de instalar nada, confirmamos que la PC física puede soportar VirtualBox y varias VMs Linux corriendo a la vez.

Requisitos mínimos recomendados:

- CPU de 4 núcleos físicos o más, con virtualización por hardware (Intel VT-x o AMD-V) habilitada en BIOS/UEFI.
- 16 GB de RAM como mínimo (32 GB ideal si se desea correr 4+ nodos simultáneos sin apagar otros).
- 120 GB de espacio libre en disco (SSD muy recomendado; HDD funcionará pero más lento).
- Windows 10/11 de 64 bits con permisos de administrador.

Paso 2 — Confirmar que la virtualización por hardware está habilitada.

Ver sección E para los comandos exactos.

Paso 3 — Definir el alcance temático del laboratorio.

Ya está definido por el plan de 19 fases (ver tabla de C.2). En esta fase solo lo confirmamos y lo documentamos como referencia.

Paso 4 — Diseñar el diagrama de arquitectura objetivo.

Usar el diagrama de la sección C como punto de partida; el estudiante debe poder explicarlo de memoria antes de avanzar.

Paso 5 — Definir convención de nombres.

`bdd-nodoNN` para nodos de datos, `bdd-cliente` para la estación de pruebas. Esta convención se usará en hostname, en el nombre de la VM dentro de VirtualBox, y en las carpetas de configuración.

Paso 6 — Definir el esquema de direccionamiento IP.

Ya documentado en C.3. Se reserva ahora aunque se configure hasta la Fase 6.

Paso 7 — Definir la estrategia de snapshots.

Cada fase terminada y verificada debe cerrar con un snapshot de VirtualBox nombrado `faseNN-completa` en cada VM involucrada. Esto permite retroceder si algo se rompe en una fase posterior, sin perder el trabajo previo.

Paso 8 — Crear la estructura de carpetas del proyecto en el host.

Ver comandos en la sección E.

Paso 9 — Validar el plan con el checklist de la sección H.

E. Comandos completos

E.1 Verificar virtualización por hardware habilitada (Windows, PowerShell)

Abrir PowerShell (no es necesario ser administrador para esta consulta):

```
PowerShell  
systeminfo
```

Buscar manualmente en la salida la sección **"Requisitos de Hyper-V"** (en inglés *Hyper-V Requirements*). Debe aparecer:

```
Text  
  
La virtualización habilitada en firmware: Sí
```

Si dice "No", la virtualización está deshabilitada en BIOS/UEFI y debe habilitarse manualmente reiniciando el equipo y entrando a la configuración de BIOS (la tecla varía por fabricante: Del, F2, F10, Esc).

E.2 Verificar RAM total y disponible

```
PowerShell  
systeminfo | findstr /C:"Memoria física total" /C:"Memoria físic"
```

(En equipos en inglés: `findstr /C:"Total Physical Memory"`)

E.3 Verificar espacio libre en disco

```
PowerShell  
wmic logicaldisk get size,freespace,caption
```

E.4 Verificar si Hyper-V está activo (posible conflicto con VirtualBox)

```
PowerShell  
bcdedit /enum | findstr hypervisorlaunchtype
```

Si el resultado es `hypervisorlaunchtype Auto`, Hyper-V está activo. VirtualBox 7.x moderno puede coexistir con Hyper-V, pero si en fases posteriores se presentan errores de arranque de VM (`VT-x is not available`), se puede desactivar así (requiere reinicio):

 PowerShell

```
bcdedit /set hypervisorlaunchtype off
```

*Nota: en este laboratorio normalmente **no será necesario** desactivar Hyper-V. Se documenta aquí solo como referencia para la sección de solución de problemas.*

E.5 Crear la estructura de carpetas del proyecto (en el host)

 PowerShell

```
mkdir C:\LabBDD
mkdir C:\LabBDD\VMs
mkdir C:\LabBDD\ISOs
mkdir C:\LabBDD\Scripts
mkdir C:\LabBDD\Snapshots-Notas
mkdir C:\LabBDD\Documentacion
```

Propósito de cada carpeta:

- `VMs`: aquí vivirán los discos virtuales (.vdi) de cada nodo.
- `ISOs`: imagen ISO de Ubuntu Server descargada (se usará en Fase 4).
- `Scripts`: scripts de configuración reutilizables que crearemos en fases futuras.
- `Snapshots-Notas`: bitácora en texto de qué representa cada snapshot.
- `Documentacion`: este archivo y los entregables de cada fase.

F. Verificación de funcionamiento

Esta fase no instala software, por lo que la “verificación” consiste en confirmar que la planeación está completa y es correcta:

1. `systeminfo` muestra virtualización habilitada en firmware → **Sí**.
2. El host tiene al menos 16 GB de RAM totales y al menos 120 GB libres en disco.
3. La carpeta `C:\LabBDD` existe con sus 5 subcarpetas.
4. El estudiante puede dibujar de memoria el diagrama de arquitectura objetivo (C.1 y C.2) y explicar el rol de cada nodo.

5. El estudiante puede explicar, sin ver el documento, la diferencia entre fragmentación horizontal y vertical.

G. Problemas comunes y soluciones

Problema	Causa probable	Solución
<code>systeminfo</code> muestra "Virtualización habilitada en firmware: No"	VT-x/AMD-V deshabilitado en BIOS/UEFI	Reiniciar, entrar a BIOS, habilitar "Intel Virtualization Technology" o "SVM Mode" (AMD), guardar y reiniciar
RAM total reportada es menor a la física instalada	RAM reservada por gráficos integrados o módulos defectuosos	Verificar en <code>Administrador de tareas</code> → <code>Rendimiento</code> → <code>Memoria</code> ; si la diferencia es muy grande, revisar configuración de memoria compartida con GPU integrada
Poco espacio libre en disco	Disco con muchos archivos/instalaciones previas	Liberar espacio antes de avanzar; los discos virtuales dinámicos crecerán con el uso, se necesita margen
Hyper-V activo y futura VM no arranca (error <code>VT-x is not available</code>)	Conflicto entre Hyper-V y VirtualBox en versiones antiguas	Actualizar VirtualBox a la versión 7.x más reciente (soporta modo Hyper-V) o desactivar Hyper-V con <code>bcdedit</code> (ver E.4)
Antivirus bloquea la creación de carpetas o adaptadores de red más adelante	Política de seguridad corporativa/educativa muy restrictiva	Solicitar excepción para <code>C:\LabBDD</code> y para el ejecutable de VirtualBox en el antivirus institucional

H. Checklist de validación

- ☐ Verifiqué que mi CPU soporta y tiene habilitada la virtualización por hardware.
- ☐ Confirmé que tengo al menos 16 GB de RAM (idealmente 32 GB).
- ☐ Confirmé que tengo al menos 120 GB libres en disco.
- ☐ Creé la estructura de carpetas `C:\LabBDD` con sus 5 subcarpetas.
- ☐ Puedo explicar la diferencia entre replicación física y lógica.
- ☐ Puedo explicar la diferencia entre fragmentación horizontal, vertical e híbrida.
- ☐ Entiendo y puedo reproducir el diagrama de arquitectura objetivo (7 nodos, roles, IPs).
- ☐ Entiendo por qué se eligió una red Host-Only y no NAT o Bridged.

- ☐ Tengo claro el plan de crecimiento gradual (de 2 nodos en Fase 3 a 6–7 nodos en Fase 16).
 - ☐ Entiendo la estrategia de snapshots por fase.
-

I. Preparación para la siguiente fase

La **Fase 1: Instalación de VirtualBox** requerirá:

- Permisos de administrador en Windows.
- Conexión a internet para descargar el instalador desde el sitio oficial de Oracle VirtualBox.
- La estructura de carpetas `C:\LabBDD` ya creada (Paso 8 de esta fase).
- Haber resuelto cualquier conflicto de virtualización detectado en esta fase (BIOS/UEFI o Hyper-V).

No se necesita descargar todavía la ISO de Ubuntu Server; eso corresponde a la Fase 4.

Preguntas teóricas para estudiantes

1. ¿Qué diferencia existe entre la replicación física y la replicación lógica en un sistema de bases de datos distribuidas, y por qué este laboratorio planea nodos distintos para cada una?
2. ¿Qué significa “transparencia de distribución” y qué tipos de transparencia existen (fragmentación, ubicación, replicación, concurrencia)?
3. ¿Por qué es conveniente separar, desde el diseño, los nodos dedicados a fragmentación horizontal de los nodos dedicados a replicación?
4. Según el teorema CAP, ¿qué compromisos debe enfrentar quien diseña una arquitectura distribuida como la planeada en este laboratorio?
5. ¿Qué ventajas y qué limitaciones tiene usar una sola computadora física para simular un sistema distribuido de varios nodos, comparado con usar varias máquinas físicas reales?

Ejercicios prácticos

1. Verificar en su propia computadora si la virtualización por hardware (VT-x/AMD-V) está habilitada, documentando el procedimiento con los comandos de la sección E y el resultado obtenido (puede incluir capturas de pantalla).
2. Elaborar un diagrama propio de la arquitectura objetivo (puede usar papel, draw.io o cualquier herramienta) representando los 7 nodos propuestos, sus roles, IPs planeadas y la fase en la que se activa cada uno.

3. Calcular cuántos nodos podría correr su computadora simultáneamente sin saturar la RAM, usando la fórmula: $(\text{RAM total} - 4 \text{ GB reservados para el sistema operativo host}) / \text{RAM asignada por VM}$. Justificar el resultado con sus propios datos de hardware.

Reto adicional para alumnos avanzados

Proponer y justificar técnicamente una arquitectura alternativa que incluya un **octavo nodo de tipo proxy/balanceador** (por ejemplo, conceptualmente similar a ProxySQL o MariaDB MaxScale), indicando en qué fase del plan de 19 fases se debería introducir y qué problema resolvería que los 7 nodos actuales no resuelven por sí solos. No es necesario implementarlo, solo justificarlo por escrito.

Criterios de evaluación para el profesor

Criterio	Peso	Indicador de logro
Verificación de requisitos de hardware	20%	El estudiante presenta evidencia (capturas/comandos) de que su equipo cumple los mínimos, o documenta correctamente las limitaciones encontradas
Diagrama de arquitectura	25%	El diagrama incluye todos los nodos, roles, IPs y es consistente con el plan de 19 fases
Comprensión conceptual	25%	Responde correctamente las 5 preguntas teóricas con vocabulario técnico adecuado
Justificación de decisiones de diseño	15%	Explica con claridad por qué se eligió Host-Only Adapter, por qué se separan roles por nodo, y por qué se usa una estrategia de snapshots
Reto avanzado (si aplica)	15%	La propuesta del nodo proxy está técnicamente justificada y ubicada coherentemente en el plan de fases

Fase 1 — Instalación de Oracle VirtualBox

Continuación directa de la Fase 0. Se asume que ya existe la estructura de carpetas

C:\LabBDD y que se resolvió cualquier conflicto de virtualización (BIOS/UEFI o Hyper-V) detectado previamente.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Explicar qué es un hipervisor de Tipo 2 y diferenciarlo conceptualmente de uno de Tipo 1.
2. Descargar e instalar correctamente Oracle VirtualBox en un host Windows.
3. Instalar el VirtualBox Extension Pack y entender por qué es un componente separado con licencia distinta.
4. Configurar la carpeta predeterminada de máquinas virtuales para que apunte a `C:\LabBDD\VMs` en lugar de la ruta por defecto.
5. Verificar, mediante la interfaz gráfica y mediante línea de comandos (`VBoxManage`), que la instalación quedó funcional.
6. Reconocer los componentes principales de la interfaz de VirtualBox que se usarán en fases posteriores (Administrador de VMs, Administrador de medios, Administrador de redes).

B. Conceptos teóricos necesarios

1. Hipervisor. Software que permite crear y ejecutar máquinas virtuales sobre un hardware físico (host).

- *Tipo 1 (bare-metal):* se ejecuta directamente sobre el hardware, sin sistema operativo anfitrión por debajo (ej. VMware ESXi, Hyper-V en modo servidor, KVM). Suele usarse en centros de datos.
- *Tipo 2 (hosted):* se ejecuta como una aplicación dentro de un sistema operativo anfitrión ya existente (ej. VirtualBox, VMware Workstation). Es el modelo de este laboratorio: Windows es el host, y VirtualBox corre como un programa más dentro de él.

2. VirtualBox como producto. Es un hipervisor de Tipo 2 de propósito general, mantenido por Oracle, que permite crear máquinas virtuales x86_64 con soporte para Windows, Linux, macOS (Intel) y Solaris como sistemas invitados.

3. Paquete base vs. Extension Pack. VirtualBox se distribuye en dos partes con licencias distintas:

- El **paquete base** es software libre bajo licencia GNU GPL v3.
- El **Extension Pack** es un componente adicional, de licencia propietaria (gratuita para uso personal/educativo, pero no de código abierto), que agrega soporte para dispositivos USB 2.0/3.0, Remote Desktop Protocol (RDP), cifrado de disco y arranque PXE para tarjetas de red Intel. En este laboratorio se instalará porque facilitará el uso de USB y, opcionalmente, acceso remoto a las VMs en fases posteriores.

4. ¿Por qué versión 7.2.x? La rama 7.2 es, a la fecha de esta guía, la rama en mantenimiento activo de VirtualBox; las ramas 7.1, 7.0 y 6.1 ya no reciben soporte. Se recomienda siempre instalar la última versión estable de la rama activa, descargada únicamente desde el sitio oficial.

5. VBoxManage. Herramienta de línea de comandos incluida con VirtualBox que permite hacer, mediante scripts, todo lo que la interfaz gráfica permite hacer manualmente (crear VMs, discos, snapshots, redes, etc.). Se usará intensivamente a partir de la Fase 3 para automatizar la creación de nodos.

C. Procedimiento paso a paso

Paso 1 — Descargar el instalador desde el sitio oficial.

Ir a <https://www.virtualbox.org/wiki/Downloads> (nunca desde portales de terceros tipo Softonic/Uptodown, para evitar instaladores modificados). Descargar el paquete para “**Windows hosts**” de la rama estable más reciente (7.2.x al momento de escribir esta guía).

Paso 2 — Descargar el Extension Pack correspondiente.

En la misma página de descargas, descargar “**VirtualBox Extension Pack**”. Es un único archivo `.vbox-extpack` válido para todas las plataformas; debe coincidir en número de versión mayor.menor con el instalador base (ej. ambos 7.2.x).

Paso 3 — Ejecutar el instalador base con permisos de administrador.

Doble clic en el `.exe` descargado → “Sí” en el control de cuentas de usuario (UAC). Durante el asistente:

- Dejar las características marcadas por defecto (VirtualBox USB Support, VirtualBox Networking, VirtualBox Python 2.x/3.x Support si aparece).
- Cuando pregunte por la ruta de instalación del *programa* (no de las VMs), se puede dejar la ruta por defecto (`C:\Program Files\Oracle\VirtualBox\`); esto es distinto de dónde vivirán los discos de las VMs, que se configura en el Paso 6.

- Aceptar la advertencia de “se perderá temporalmente la conectividad de red” (VirtualBox instala adaptadores de red virtuales; puede desconectar brevemente el Wi-Fi/Ethernet del host durante la instalación).
- Finalizar e iniciar VirtualBox al terminar.

Paso 4 — Instalar el Extension Pack desde la interfaz gráfica.

Con VirtualBox abierto: `Archivo → Herramientas → Administrador de medios` no es la ruta correcta para esto; en su lugar ir a `Archivo → Preferencias → Extensiones → ícono "+"` (Agregar paquete), seleccionar el archivo `.vbox-extpack` descargado en el Paso 2, dar clic en “Instalar” y aceptar la licencia (PUEL — Personal Use and Evaluation License).

Paso 5 — Verificar la versión instalada.

En la interfaz: `Ayuda → Acerca de VirtualBox`. Debe coincidir la versión del programa con la del Extension Pack (ver comandos en sección D para verificación por línea de comandos).

Paso 6 — Redirigir la carpeta predeterminada de máquinas virtuales.

Por defecto, VirtualBox guarda discos y configuraciones en `C:\Users\<usuario>\VirtualBox VMs\`. Para mantener todo el laboratorio organizado dentro de `C:\LabBDD`, cambiar esta ruta: `Archivo → Preferencias → General → Carpeta predeterminada de máquinas` → cambiar a `C:\LabBDD\VMs`.

*Importante: este cambio solo afecta a las VMs que se creen **después** de modificarlo; no mueve nada existente (en este punto no debería haber ninguna VM creada todavía).*

Paso 7 — Revisar el Administrador de redes (sin crear nada todavía).

`Herramientas → Red` (o `Archivo → Herramientas → Administrador de redes de host`). Simplemente confirmar que la pestaña “Adaptadores solo-anfitrión” existe y está vacía. La creación del adaptador `vboxnet0` con la IP `192.168.56.1` planeada en la Fase 0 se hará formalmente en la **Fase 2**, no aquí.

Paso 8 — Cerrar snapshot conceptual de la fase.

Como todavía no existen VMs, no hay nada que snapshotear en VirtualBox en este punto. En su lugar, anotar en `C:\LabBDD\Snapshots-Notas\fase01-notas.txt` la versión exacta instalada (base + Extension Pack) y la fecha, para trazabilidad.

D. Comandos

D.1 Verificar la versión de VirtualBox desde PowerShell

```
PowerShell  
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" --version
```

Salida esperada (ejemplo de formato, el número exacto dependerá de la versión descargada):

```
Text  
  
7.2.8r173730
```

D.2 Agregar VBoxManage al PATH (opcional, recomendado)

Para no escribir la ruta completa cada vez, en PowerShell como administrador:

```
PowerShell  
  
[Environment]::SetEnvironmentVariable(  
    "Path",  
    $env:Path + ";C:\Program Files\Oracle\VirtualBox",  
    [EnvironmentVariableTarget]::Machine  
)
```

Cerrar y volver a abrir PowerShell para que tome efecto. Después de esto, el comando se simplifica a:

```
PowerShell  
  
VBoxManage --version
```

D.3 Listar los Extension Packs instalados

```
PowerShell  
  
VBoxManage list extpacks
```

Debe mostrar el paquete `Oracle VM VirtualBox Extension Pack` con número de versión y revisión (`r`) idéntico al del paso D.1.

D.4 Confirmar que aún no existen máquinas virtuales (esperado en esta fase)

```
PowerShell
VBoxManage list vms
```

Salida esperada: **vacío** (todavía no se ha creado ningún nodo; eso corresponde a la Fase 3).

D.5 Confirmar la carpeta predeterminada configurada en el Paso 6

```
PowerShell
VBoxManage list systemproperties | findstr /C:"Default machine folder"
```

Debe mostrar `C:\LabBDD\VMs` .

E. Verificación de funcionamiento

1. `VBoxManage --version` se ejecuta sin errores y reporta una versión de la rama 7.2.x (o la rama estable vigente al momento de instalar).
 2. `VBoxManage list extpacks` muestra el Extension Pack instalado con número de versión coincidente con el del paquete base.
 3. La interfaz gráfica de VirtualBox abre sin mensajes de error ni advertencias de drivers faltantes.
 4. `Archivo → Preferencias → General → Carpeta predeterminada de máquinas` muestra `C:\LabBDD\VMs` .
 5. `VBoxManage list vms` regresa vacío (correcto para esta fase; todavía no se crean nodos).
 6. Existe el archivo `C:\LabBDD\Snapshots-Notas\fase01-notas.txt` con la versión instalada y la fecha.
-

F. Problemas comunes y soluciones

Problema	Causa probable	Solución
El instalador falla con error de "Python Core" o "Network Interfaces" a mitad de instalación	Antivirus o políticas de grupo bloquean la instalación de drivers de red virtuales	Agregar excepción temporal para el instalador en el antivirus institucional; reintentar instalación como administrador
Mensaje <code>VT-x is not available</code> al intentar usar VirtualBox después de instalarlo	Hyper-V activo entrando en conflicto, o virtualización deshabilitada en BIOS (no resuelto desde la Fase 0)	Revisar de nuevo <code>bcdedit /enum findstr hypervisorlaunchtype</code> (ver Fase 0, sección E.4); actualizar a la última versión 7.2.x, que tiene mejor coexistencia con Hyper-V
El Extension Pack no aparece o aparece como "no compatible"	Se descargó una versión del Extension Pack que no coincide con la versión mayor.menor del paquete base	Verificar ambos números de versión en virtualbox.org/wiki/Downloads ; deben coincidir en <code>X.Y</code> (ej. ambos 7.2.x)
La red se desconecta brevemente durante la instalación y no vuelve sola	El adaptador de red físico tarda en re-negociar tras la instalación de los drivers de VirtualBox	Esperar 1–2 minutos o deshabilitar/habilitar el adaptador de red físico desde el Administrador de dispositivos
<code>VBoxManage</code> no se reconoce como comando	No se agregó al PATH (paso D.2) o no se reinició la terminal después de agregarlo	Cerrar todas las ventanas de PowerShell/CMD abiertas y abrir una nueva; o usar la ruta completa como en D.1
El instalador pide reiniciar Windows	Es normal cuando se instalan por primera vez los drivers de red/USB de VirtualBox	Reiniciar y volver a abrir VirtualBox para continuar

G. Checklist de validación

- ☐ Descargué VirtualBox únicamente desde [virtualbox.org](https://www.virtualbox.org), rama estable vigente (no de portales de terceros).

- ☐ Descargué el Extension Pack con número de versión `X.Y` coincidente con el paquete base.
 - ☐ Instalé VirtualBox con permisos de administrador y reinicié si fue solicitado.
 - ☐ Instalé el Extension Pack desde `Preferencias → Extensiones` y acepté la licencia PUEL.
 - ☐ `VBoxManage --version` y `VBoxManage list extpacks` se ejecutan correctamente desde PowerShell.
 - ☐ Cambié la carpeta predeterminada de máquinas a `C:\LabBDD\VMs`.
 - ☐ Confirmé que `VBoxManage list vms` está vacío (todavía no toca crear nodos).
 - ☐ Documenté la versión instalada en `C:\LabBDD\Snapshots-Notas\fase01-notas.txt`.
 - ☐ Puedo explicar la diferencia entre un hipervisor Tipo 1 y Tipo 2, y por qué VirtualBox es Tipo 2.
 - ☐ Puedo explicar por qué el Extension Pack es un componente separado, con licencia distinta al paquete base.
-

H. Preparación para la siguiente fase

La **Fase 2: Configuración de la red Host-Only** requerirá:

- VirtualBox y su Extension Pack ya instalados y verificados (esta fase).
- Comprender el plan de direccionamiento IP definido en la Fase 0, sección C.3 (`192.168.56.0/24`).
- Tener claro por qué se eligió Host-Only sobre NAT o Bridged (mencionado en Fase 0, se justificará a fondo en Fase 2).

Todavía no se crea ninguna máquina virtual; eso corresponde a la Fase 3, una vez configurada la red.

Fase 2 — Configuración de la Red Virtual Host-Only en VirtualBox

*Continuación directa de la Fase 0 (planeación). Esta fase **no crea ninguna máquina virtual todavía**; solo prepara la red sobre la cual vivirán todos los nodos a partir de la Fase 3. Se asume que la Fase 1 (instalación de VirtualBox) ya está completa.*

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Explicar las diferencias entre los cuatro modos de red de VirtualBox: NAT, Bridged, Internal Network y Host-Only.
 2. Justificar a fondo —como se prometió en la Fase 0— por qué este laboratorio usa **Host-Only Adapter** y no NAT ni Bridged.
 3. Crear y/o verificar el adaptador Host-Only (`vboxnet0` o, en Windows, “VirtualBox Host-Only Ethernet Adapter”) desde el VirtualBox Network Manager.
 4. Configurar manualmente la dirección IPv4 y la máscara de subred del adaptador Host-Only conforme al plan de direccionamiento de la Fase 0 (192.168.56.0/24).
 5. Decidir y aplicar la desactivación del servidor DHCP integrado de VirtualBox, dejando el camino preparado para el direccionamiento estático que se aplicará dentro de cada VM en la Fase 6.
 6. Verificar, desde el host, que la red está activa y operando correctamente antes de construir cualquier nodo.
-

B. Conceptos teóricos necesarios

1. Los cuatro modos de red en VirtualBox.

- **NAT.** Cada VM recibe una red privada propia y sale a internet a través de una traducción de direcciones que hace VirtualBox, igual que un router doméstico. Las VMs normalmente *no* se ven entre sí ni el host las ve directamente. Es el modo por defecto para “una VM aislada que solo necesita internet”.
- **Bridged.** La VM se conecta directamente a la tarjeta de red física del host y obtiene una IP de la misma red que el equipo físico (por ejemplo, la red Wi-Fi del salón o la red doméstica). Es el modo más “real”, pero también el más riesgoso para un laboratorio: expone los nodos a cualquier otro dispositivo de esa red y depende de la configuración del router/DHCP externo, algo que no controlamos.

- **Internal Network.** Crea una red completamente aislada en la que solo las VMs se ven entre sí; **ni siquiera el host** tiene acceso a ella. Útil para simular un segmento de red totalmente privado, pero inconveniente aquí porque necesitaremos administrar los nodos desde el host (SSH, DBever, scripts).
- **Host-Only.** Crea una red virtual interna que conecta **el host con todas las VMs**, pero las aísla de la red física externa. Es el punto medio ideal para este laboratorio.

2. Por qué Host-Only es la elección correcta aquí.

- *Aislamiento:* el laboratorio no depende de, ni interfiere con, la red Wi-Fi/Ethernet real del salón o de casa.
- *Visibilidad bidireccional host-VM:* el host (Windows) puede conectarse a cada nodo por SSH, DBever, etc., porque ambos comparten el mismo segmento virtual.
- *Control total de direccionamiento:* como el laboratorio depende de IPs fijas para que la replicación y los nombres de host no se rompan entre reinicios, necesitamos un segmento de red que no dependa de un router externo ni de un DHCP ajeno.
- *Repetibilidad:* cualquier estudiante que siga esta guía obtiene exactamente la misma topología de red (192.168.56.0/24), sin importar a qué red física esté conectado su equipo.

3. El servidor DHCP integrado de VirtualBox.

Por defecto, cuando se crea un adaptador Host-Only, VirtualBox también activa un pequeño servidor DHCP asociado (normalmente sirviendo el rango `.100 – .254`). Si lo dejáramos activo, cada VM podría recibir una IP diferente en cada arranque, lo cual es inaceptable para un sistema distribuido: la replicación maestro-esclavo, los nombres de host y los scripts de conexión dependen de que cada nodo tenga **siempre la misma IP**. Por eso esta fase termina **desactivando** ese DHCP; las IPs estáticas reales se asignarán dentro de cada VM hasta la Fase 6, pero el camino se prepara desde ahora.

4. `vboxnet0` vs. nombres de adaptador en Windows.

En Linux/macOS, VirtualBox nombra a los adaptadores Host-Only como `vboxnet0`, `vboxnet1`, etc. En Windows, el mismo concepto aparece en el sistema operativo como una tarjeta de red llamada “VirtualBox Host-Only Ethernet Adapter” (visible en el Panel de Control o en `Get-NetAdapter`), aunque VirtualBox internamente sigue refiriéndose a ella con un nombre técnico equivalente. Ambos términos se usan de forma intercambiable en esta guía.

C. Procedimiento paso a paso

Paso 1 — Abrir el Network Manager de VirtualBox.

En la ventana principal de VirtualBox: `Herramientas` (ícono superior) → `Network Manager` → pestaña `Host-only Networks` (en versiones más antiguas: `Archivo` → `Preferencias de Host` → `Red`).

Paso 2 — Verificar si ya existe un adaptador Host-Only por defecto.

VirtualBox suele crear uno automáticamente al instalarse. Si ya aparece uno en la lista, se reutiliza; si la lista está vacía, se crea uno nuevo con el botón `Crear`.

Paso 3 — Configurar la IPv4 del adaptador.

Seleccionar el adaptador → pestaña `Adapter` → configurar:

- **Dirección IPv4:** `192.168.56.1`
- **Máscara de red IPv4:** `255.255.255.0`

Esto coincide exactamente con la IP del host definida en la tabla de direccionamiento de la Fase 0 (C.3).

Paso 4 — Desactivar el servidor DHCP del adaptador.

En la misma ventana, pestaña `DHCP Server` → desmarcar la casilla `Habilitar servidor` (Enable Server). Confirmar/Aplicar.

Si esta casilla no se puede desmarcar desde la interfaz gráfica en tu versión de VirtualBox, usa el comando de la sección E.3 para eliminarlo por línea de comandos.

Paso 5 — Aplicar y cerrar.

Guardar los cambios y cerrar el Network Manager.

Paso 6 — Verificar desde Windows que el adaptador quedó activo.

Ejecutar los comandos de la sección E.1 para confirmar que el sistema operativo ve la tarjeta “VirtualBox Host-Only Ethernet Adapter” con la IP `192.168.56.1`.

Paso 7 — Verificar desde VirtualBox (línea de comandos) que la configuración quedó guardada.

Ejecutar el comando de la sección E.2 (`vBoxManage list hostonlyifs`) y confirmar IP, máscara y estado `Up`.

Paso 8 — Documentar la configuración aplicada.

Crear un archivo de respaldo en la carpeta de documentación ya existente desde la Fase 0, con el detalle de lo configurado (ver Paso E.4).

D. Comandos completos

D.1 Verificar el adaptador Host-Only desde Windows (PowerShell)

 PowerShell

```
Get-NetAdapter | Where-Object { $_.InterfaceDescription -like "*VirtualBox Host-Only*" }
```

Debe mostrar **Status: Up** . Para ver la IP asignada:

 PowerShell

```
ipconfig | findstr /C:"VirtualBox" /C:"IPv4"
```

Buscar manualmente el bloque correspondiente a “Ethernet adapter VirtualBox Host-Only Network” y confirmar que la dirección IPv4 sea **192.168.56.1** .

D.2 Verificar la configuración desde VirtualBox (CLI)

Desde PowerShell, ubicado en la carpeta de instalación de VirtualBox (o si **VBoxManage** ya está en el PATH):

 PowerShell

```
VBoxManage list hostonlyifs
```

Salida esperada (resumida):

 Text

```
Name:          VirtualBox Host-Only Ethernet Adapter
GUID:          ...
DHCP:          Disabled
IPAddress:      192.168.56.1
NetworkMask:    255.255.255.0
Status:        Up
```

D.3 Eliminar el servidor DHCP por línea de comandos (si el GUI no lo permite)

Primero listar los servidores DHCP existentes:

PowerShell

```
VBoxManage list dhcpservers
```

Si aparece uno asociado al adaptador Host-Only, eliminarlo indicando el nombre de interfaz exacto reportado en el paso anterior:

PowerShell

```
VBoxManage dhcpserver remove --ifname "VirtualBox Host-Only Ethernet Adapter"
```

El nombre exacto entre comillas debe coincidir con el que reportó `VBoxManage list hostonlyifs`; puede variar ligeramente entre versiones (por ejemplo, agregando un número si hay más de un adaptador Host-Only).

D.4 Crear el respaldo de documentación de esta fase

PowerShell

```
notepad C:\LabBDD\Documentacion\fase2-red.md
```

Contenido sugerido a pegar y guardar en ese archivo:

Text

```
Fase 2 - Red Host-Only
IP del host en la red virtual: 192.168.56.1
Mascara: 255.255.255.0
DHCP: deshabilitado (se usaran IPs estaticas desde Fase 6)
Adaptador: VirtualBox Host-Only Ethernet Adapter
Fecha de configuracion: <completar>
```

E. Verificación de funcionamiento

1. `Get-NetAdapter` muestra el adaptador "VirtualBox Host-Only Ethernet Adapter" con estado `Up`.
2. `ipconfig` confirma que ese adaptador tiene la IPv4 `192.168.56.1` y máscara `255.255.255.0`.
3. `VBoxManage list hostonlyifs` confirma la misma IP/máscara y reporta `DHCP: Disabled`.

4. `VBoxManage list dhcpservers` no muestra ningún servidor DHCP activo asociado a este adaptador.
 5. El estudiante puede explicar, sin ver el documento, por qué se eligió Host-Only sobre NAT, Bridged e Internal Network.
 6. Existe el archivo `C:\LabBDD\Documentacion\fase2-red.md` con la configuración documentada.
-

F. Problemas comunes y soluciones (Troubleshooting)

Problema	Causa probable	Solución
No aparece ningún adaptador Host-Only en el Network Manager	La instalación de VirtualBox no registró el driver de red virtual correctamente	Reiniciar el host; si persiste, reparar la instalación de VirtualBox desde el instalador (<code>Repair</code>)
La casilla "Enable Server" del DHCP no se puede desmarcar desde el GUI	Bug conocido en algunas versiones de VirtualBox	Usar el comando <code>VBoxManage dhcpserver remove</code> de la sección D.3
La IP del adaptador aparece distinta a <code>192.168.56.1</code> tras reiniciar Windows	Conflicto con otro software de virtualización que usa el mismo rango (VMware Workstation, WSL2/Hyper-V vEthernet)	Revisar <code>Get-NetAdapter</code> completo, identificar el adaptador en conflicto, y si es necesario cambiar el rango del Host-Only de VirtualBox a uno libre, p. ej. <code>192.168.57.0/24</code> , actualizando también la tabla de IPs de la Fase 0
<code>VBoxManage</code> no se reconoce como comando	La carpeta de instalación de VirtualBox no está en el <code>PATH</code> del sistema	Ejecutar el comando desde la carpeta de instalación (normalmente <code>C:\Program Files\Oracle\VirtualBox</code>) o agregarla al <code>PATH</code>
El host no puede hacer ping a las VMs una vez creadas (a futuro, Fase 3+)	El adaptador de red de la VM no está vinculado al Host-Only correcto, o el firewall de Windows bloquea el segmento <code>192.168.56.0/24</code>	Confirmar en la configuración de red de la VM que el adaptador está en modo "Host-only Adapter" apuntando al adaptador correcto; revisar reglas de firewall
Al asignar como host-only no arranca la máquina virtual marcando un error de tarjeta	La instalación de VirtualBox no registró el driver de red virtual correctamente	Agregue otra tarjeta virtual, con otra dirección ya que a veces el <code>192.168.56.0/24</code> esta bloqueado o trabado.

G. Checklist de validación

- ☐ Puedo explicar la diferencia entre los modos NAT, Bridged, Internal Network y Host-Only.
- ☐ Puedo justificar, con mis propias palabras, por qué este laboratorio usa Host-Only.

- ☐ El adaptador Host-Only existe y tiene la IP `192.168.56.1` / máscara `255.255.255.0` .
 - ☐ Confirmé el estado del adaptador con `Get-NetAdapter` desde Windows.
 - ☐ Confirmé la configuración con `VBoxManage list hostonlyifs` desde la línea de comandos de VirtualBox.
 - ☐ Desactivé (o eliminé) el servidor DHCP asociado a este adaptador.
 - ☐ Verifiqué con `VBoxManage list dhcpservers` que no queda ningún servidor DHCP activo en este segmento.
 - ☐ Documenté esta configuración en `C:\LabBDD\Documentacion\fase2-red.md` .
-

Preparación para la siguiente fase

La **Fase 3** creará los dos primeros nodos (`bdd-nodo01` y `bdd-nodo02`) y requerirá:

- La red Host-Only configurada y verificada en esta fase.
 - VirtualBox instalado (Fase 1).
 - La ISO de Ubuntu Server descargada (Fase 4 — nota: aunque la numeración de fases del plan original ubica la descarga de la ISO después, en la práctica deberá tenerse lista antes de poder instalar el sistema operativo en los nodos; esto se aclarará explícitamente al inicio de la Fase 3).
-

Fase 3 — Creación de las Primeras Máquinas Virtuales (bdd-nodo01 y bdd-nodo02)

*Continuación del laboratorio de bases de datos distribuidas. En esta fase se crean las **dos primeras VMs vacías** (sin sistema operativo todavía) siguiendo el dimensionamiento y la convención de nombres definidos en la Fase 0. La instalación del sistema operativo dentro de ellas corresponde a una fase posterior, una vez descargada la ISO (Fase 4).*

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Crear máquinas virtuales vacías en VirtualBox respetando el dimensionamiento de recursos (vCPU, RAM, disco) definido en la Fase 0.
2. Aplicar correctamente la convención de nombres `bdd-nodoNN` al nombrar las VMs dentro de VirtualBox.
3. Crear y habilitar la red Host-Only (`vboxnet0`) que servirá de columna vertebral de comunicación entre nodos.
4. Adjuntar el adaptador de red Host-Only a cada VM, dejando la configuración de IP estática pendiente para la fase de red.
5. Explicar la diferencia entre “crear una VM” (definir hardware virtual: CPU, RAM, disco, red) e “instalar un sistema operativo” dentro de ella.
6. Verificar, mediante línea de comandos, que las VMs creadas coinciden exactamente con lo planeado.
7. Cerrar la fase con la primera bitácora de snapshot, aunque todavía no haya sistema operativo instalado.

B. Teoría necesaria

1. ¿Qué es una VM a nivel de archivos en VirtualBox?

Cada máquina virtual está compuesta, principalmente, por dos tipos de archivo en el host: un archivo de configuración `.vbox` (XML legible, describe CPU, RAM, dispositivos, adaptadores de red) y uno o más archivos de disco virtual `.vdi` (el contenido del “disco duro” virtual).

Crear una VM, en esta fase, significa generar estos archivos; **todavía no contienen un sistema operativo**, son un contenedor de hardware virtual vacío, equivalente a comprar una PC nueva sin sistema operativo instalado.

2. Disco dinámico vs. disco de tamaño fijo.

- *Dinámico*: el archivo `.vdi` empieza pequeño (unos MB) y crece según se usa, hasta el límite definido. Ahorra espacio en el host mientras el laboratorio crece gradualmente.
- *Fijo*: reserva todo el espacio desde el inicio; es ligeramente más rápido en I/O pero desperdicia espacio si el nodo no se usa todavía.
Para este laboratorio se eligió **dinámico** (ver tabla C.4 de la Fase 0), porque el host no tiene por qué reservar 120 GB de inmediato si los nodos 4–7 no se activarán hasta fases avanzadas.

3. Chipset y firmware de la VM.

VirtualBox permite elegir chipset (PIIX3 vs. ICH9) y firmware (BIOS vs. EFI). Para Ubuntu Server LTS moderno, el valor por defecto que ofrece VirtualBox al seleccionar el tipo de sistema operativo “Linux / Ubuntu (64-bit)” es adecuado y no requiere ajustes manuales en este laboratorio.

4. Adaptador de red Host-Only: qué se hace ahora y qué se deja pendiente.

En la Fase 0 se decidió usar Host-Only Adapter en lugar de NAT o Bridged. Esa red virtual (`vboxnet0`) es un objeto independiente de las VMs: existe a nivel de VirtualBox/host, no dentro de ninguna máquina. En esta fase:

- Se **crea** la red Host-Only `vboxnet0` con IP de host `192.168.56.1` (coincide con el plan de direccionamiento de la Fase 0, sección C.3).
- Se **adjunta** el Adaptador 1 de cada VM a esa red.
- **No** se configura todavía la IP estática dentro de cada VM, porque eso requiere un sistema operativo instalado y arrancado (correspondiente a la Fase 6). Por ahora la VM ni siquiera puede arrancar: no tiene OS.

5. ¿Por qué crear las VMs antes de tener la ISO de Ubuntu?

Es una separación de responsabilidades: primero se define el “hardware virtual” (esta fase), después se obtiene el medio de instalación (Fase 4) y finalmente se instala el sistema operativo sobre el hardware ya definido (fase posterior). Esto refleja cómo se aprovisiona infraestructura real en un centro de datos: el aprovisionamiento de hardware/VM y la instalación del sistema operativo suelen ser pasos administrativos distintos.

C. Procedimiento paso a paso

Paso 1 — Verificar que VirtualBox está instalado y operativo.

Confirmar que la Fase 1 (instalación de VirtualBox) quedó completa antes de continuar.

Paso 2 — Crear la red Host-Only `vboxnet0` en el host.

Esta red es compartida por todas las VMs del laboratorio; se crea una sola vez.

Paso 3 — Crear la VM `bdd-nodo01` (vacía).

Usando el dimensionamiento de la Fase 0: 1 vCPU, 1536 MB RAM, 20 GB disco dinámico.

Paso 4 — Adjuntar el disco virtual y el controlador de almacenamiento a `bdd-nodo01`.

Paso 5 — Configurar el Adaptador 1 de `bdd-nodo01` como Host-Only, enlazado a `vboxnet0`.

Paso 6 — Repetir los Pasos 3 a 5 para `bdd-nodo02` (mismo dimensionamiento: 1 vCPU, 1536 MB, 20 GB).

Paso 7 — Verificar ambas VMs con `VBoxManage list vms` y `showvminfo`.

Paso 8 — Documentar el snapshot inicial de la fase.

Aunque no hay sistema operativo, se recomienda anotar en `Snapshots-Notas` que ambas VMs quedaron creadas y verificadas, como punto de partida antes de instalar el OS.

Paso 9 — Validar con el checklist de la sección G.

D. Comandos completos

Todos los comandos se ejecutan en PowerShell, en el host Windows. Se usa `VBoxManage`, la herramienta de línea de comandos de VirtualBox (si no se reconoce el comando, agregar `C:\Program Files\Oracle\VirtualBox` al PATH del sistema, o ejecutar desde esa carpeta).

D.1 Crear la red Host-Only `vboxnet0`

```
> PowerShell
VBoxManage list hostonlyifs
```

Si no aparece ninguna interfaz todavía, crearla:

```
> PowerShell
VBoxManage hostonlyif create
```

Asignarle la IP de host planeada en la Fase 0 (192.168.56.1):

```
> PowerShell
VBoxManage hostonlyif ipconfig "VirtualBox Host-Only Ethernet Adapter" --ip
192.168.56.1 --netmask 255.255.255.0
```

Nota: el nombre exacto de la interfaz ("VirtualBox Host-Only Ethernet Adapter", a veces con un número al final como #2) puede variar según cuántas interfaces Host-Only existan ya en el sistema. Usar el nombre exacto que devolvió list hostonlyifs en el paso anterior.

D.2 Crear la VM bdd-nodo01

PowerShell

```
VBoxManage createvm --name "bdd-nodo01" --ostype "Ubuntu_64" --basefolder "C:\LabBDD\VMs" --register
```

Configurar CPU y RAM según el dimensionamiento de la Fase 0:

PowerShell

```
VBoxManage modifyvm "bdd-nodo01" --cpus 1 --memory 1536 --vram 16
```

Crear el controlador de almacenamiento (SATA) y el disco virtual dinámico de 20 GB:

PowerShell

```
VBoxManage storagectl "bdd-nodo01" --name "SATA Controller" --add sata --controller IntelAHCI
VBoxManage createhd --filename "C:\LabBDD\VMs\bdd-nodo01\bdd-nodo01.vdi" --size 20480 --variant Standard
VBoxManage storageattach "bdd-nodo01" --storagectl "SATA Controller" --port 0 --device 0 --type hdd --medium "C:\LabBDD\VMs\bdd-nodo01\bdd-nodo01.vdi"
```

Agregar también un controlador IDE/SATA vacío para la futura unidad óptica (se usará en Fase 4 para montar la ISO):

PowerShell

```
VBoxManage storageattach "bdd-nodo01" --storagectl "SATA Controller" --port 1 --device 0 --type dvddrive --medium emptydrive
```

Configurar el Adaptador 1 como Host-Only:

PowerShell

```
VBoxManage modifyvm "bdd-nodo01" --nic1 hostonly --hostonlyadapter1 "VirtualBox Host-Only Ethernet Adapter"
```

D.3 Crear la VM `bdd-nodo02`

Repetir el mismo bloque, cambiando únicamente el nombre:

 PowerShell

```
VBoxManage createvm --name "bdd-nodo02" --ostype "Ubuntu_64" --basefolder
"C:\LabBDD\VMs" --register
VBoxManage modifyvm "bdd-nodo02" --cpus 1 --memory 1536 --vram 16
VBoxManage storagectl "bdd-nodo02" --name "SATA Controller" --add sata --
controller IntelAHCI
VBoxManage createhd --filename "C:\LabBDD\VMs\bdd-nodo02\bdd-nodo02.vdi" --
size 20480 --variant Standard
VBoxManage storageattach "bdd-nodo02" --storagectl "SATA Controller" --port 0
--device 0 --type hdd --medium "C:\LabBDD\VMs\bdd-nodo02\bdd-nodo02.vdi"
VBoxManage storageattach "bdd-nodo02" --storagectl "SATA Controller" --port 1
--device 0 --type dvddrive --medium emptydrive
VBoxManage modifyvm "bdd-nodo02" --nic1 hostonly --hostonlyadapter1
"VirtualBox Host-Only Ethernet Adapter"
```

D.4 Listar y verificar las VMs creadas

 PowerShell

```
VBoxManage list vms
```

Salida esperada:

 Text

```
"bdd-nodo01" {xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}
"bdd-nodo02" {xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}
```

Ver el detalle completo de configuración de una VM:

 PowerShell

```
VBoxManage showvminfo "bdd-nodo01"
```

Listar los discos virtuales registrados:

 PowerShell

```
VBoxManage list hdds
```

E. Verificación de funcionamiento

1. `VBoxManage list vms` muestra exactamente `bdd-nodo01` y `bdd-nodo02` (y ninguna otra VM no planeada).
 2. `VBoxManage showvminfo "bdd-nodo01"` reporta: 1 CPU, 1536 MB de memoria, adaptador de red 1 en modo `hostonly` enlazado a la interfaz correcta.
 3. Lo mismo se cumple para `bdd-nodo02`.
 4. `VBoxManage list hdds` muestra dos discos `.vdi` de 20 GB (tamaño lógico), ubicados dentro de `C:\LabBDD\VMs\`.
 5. `VBoxManage list hostonlyifs` muestra la interfaz Host-Only con IP `192.168.56.1`.
 6. Al abrir VirtualBox Manager (interfaz gráfica), ambas VMs aparecen apagadas (estado "Apagada"), sin sistema operativo instalado, listas para la Fase 4.
 7. El estudiante puede explicar por qué ninguna de las dos VMs puede arrancar todavía un sistema operativo funcional.
-

F. Problemas comunes y soluciones

Problema	Causa probable	Solución
<code>VBoxManage</code> no se reconoce como comando	La carpeta de instalación no está en el PATH de Windows	Ejecutar los comandos desde <code>C:\Program Files\Oracle\VirtualBox</code> , o agregar esa ruta al PATH del sistema y reabrir PowerShell
Error al crear la interfaz Host-Only (<code>hostonlyif create</code> falla)	Controlador de red virtual de VirtualBox no instalado correctamente, o conflicto con otro software de virtualización	Reinstalar/reparar VirtualBox desde el instalador oficial; verificar en "Conexiones de red" de Windows que aparezca el adaptador Host-Only
<code>createvm</code> falla indicando que la VM ya existe	Se ejecutó el comando dos veces, o quedó un registro previo de prueba	Eliminar la VM duplicada con <code>VBoxManage unregistervm "bdd-nodo01" --delete</code> antes de volver a crearla
<code>storageattach</code> falla con error de archivo no encontrado	La ruta del <code>.vdi</code> no coincide exactamente con la usada en <code>createhd</code> , o la carpeta no existe	Verificar que <code>C:\LabBDD\VMs\bdd-nodo01\</code> exista (VirtualBox normalmente la crea automáticamente con <code>createvm --register</code>); revisar rutas exactas
La VM aparece con 0 MB de RAM o 0 CPUs en <code>showvminfo</code>	El comando <code>modifyvm</code> se ejecutó antes de que <code>createvm</code> terminara de registrar la VM, o hubo un error silencioso	Volver a ejecutar <code>modifyvm</code> explícitamente y confirmar con <code>showvminfo</code>
El adaptador de red no aparece como <code>hostonly</code> sino como <code>nat</code> (valor por defecto)	El comando <code>modifyvm --nic1 hostonly</code> no se ejecutó o se ejecutó con un nombre de adaptador incorrecto	Volver a ejecutar <code>modifyvm</code> con el nombre exacto de la interfaz devuelto por <code>list hostonlyifs</code> (cuidado con mayúsculas y espacios)
Espacio en disco del host crece más de lo esperado a pesar de usar disco dinámico	Confusión entre tamaño lógico (20 GB) y tamaño real ocupado; en esta fase, sin OS instalado, el <code>.vdi</code> debería pesar solo unos KB-MB	Verificar con el explorador de archivos el tamaño real del <code>.vdi</code> ; si ya pesa varios GB sin haber instalado nada, revisar que se usó <code>--variant Standard</code> (dinámico) y no <code>Fixed</code>

G. Checklist de validación

- ☐ La red Host-Only `vboxnet0` existe y tiene la IP `192.168.56.1` asignada.
 - ☐ La VM `bdd-nodo01` existe, con nombre exacto según convención.
 - ☐ La VM `bdd-nodo01` tiene 1 vCPU, 1536 MB de RAM y un disco dinámico de 20 GB.
 - ☐ La VM `bdd-nodo01` tiene su Adaptador 1 en modo Host-Only, enlazado a la interfaz correcta.
 - ☐ La VM `bdd-nodo01` tiene una unidad óptica vacía lista para montar la ISO en la Fase 4.
 - ☐ Se repitieron y verificaron los 4 puntos anteriores para `bdd-nodo02`.
 - ☐ `VBoxManage list vms` muestra únicamente las dos VMs esperadas, sin duplicados ni VMs de prueba sobrantes.
 - ☐ Ambos discos `.vdi` están ubicados dentro de `C:\LabBDD\VMs\`, conforme a la estructura de carpetas de la Fase 0.
 - ☐ El estudiante puede explicar la diferencia entre “crear una VM” e “instalar un sistema operativo”.
 - ☐ Se documentó el estado de esta fase en `Snapshots-Notas` como punto de partida antes de la instalación del sistema operativo.
-

Fase 4 — Descarga y Verificación de la Imagen ISO de Ubuntu Server LTS

Continuación del Laboratorio de Bases de Datos Distribuidas. En esta fase **no se crea ninguna máquina virtual todavía**: solo se obtiene, verifica y almacena la imagen ISO que se usará en la Fase 5 para instalar el sistema operativo en los nodos.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Explicar qué es una imagen ISO y por qué se usa como medio de instalación para sistemas operativos en máquinas virtuales.
2. Distinguir entre las versiones LTS (*Long Term Support*) y las versiones interinas de Ubuntu, y justificar por qué este laboratorio usa exclusivamente una versión LTS.
3. Descargar correctamente la imagen ISO de Ubuntu Server desde una fuente oficial.
4. Verificar la integridad de un archivo descargado mediante un hash criptográfico (SHA256), y entender por qué este paso no es opcional en un entorno de laboratorio serio.
5. Ubicar el archivo descargado en la estructura de carpetas definida en la Fase 0 (`C:\LabBDD\ISOs`).

B. Conceptos teóricos necesarios

1. Imagen ISO. Un archivo `.iso` es una copia exacta (sector por sector) del contenido de un disco óptico. VirtualBox puede “montar” este archivo como si fuera un DVD físico insertado en la unidad óptica virtual de una VM, permitiendo arrancar el instalador del sistema operativo sin necesidad de quemar un disco real.

2. Ubuntu Server vs. Ubuntu Desktop. Para este laboratorio se usa **Ubuntu Server**, no Ubuntu Desktop. Ubuntu Server no incluye entorno gráfico (GNOME, etc.), lo cual reduce el consumo de RAM y disco — un factor crítico porque el plan final contempla correr varios nodos simultáneamente sobre el mismo host. Toda la administración de los nodos se hará por línea de comandos (SSH, terminal de VirtualBox).

3. Versiones LTS vs. interinas. Canonical (la empresa detrás de Ubuntu) publica una versión nueva cada seis meses, pero solo las versiones **LTS** (publicadas cada dos años, en abril de años pares) reciben soporte extendido — típicamente 5 años de actualizaciones de seguridad,

ampliables con Ubuntu Pro. Las versiones interinas (por ejemplo, 25.10) reciben soporte de solo 9 meses. Para un laboratorio académico que puede extenderse varios meses, **siempre se debe usar una versión LTS**, nunca una interina.

A la fecha de esta fase existen dos LTS disponibles:

- **Ubuntu Server 24.04 LTS “Noble Numbat”** (en su punto de actualización 24.04.4): es la opción **recomendada para este laboratorio**. Lleva más de dos años en circulación, tiene soporte hasta 2029, y prácticamente toda la documentación de MariaDB, tutoriales y foros de la comunidad está probada contra esta versión. Para un laboratorio enfocado en bases de datos distribuidas (y no en probar las características más nuevas de Linux), la estabilidad y la documentación disponible pesan más que tener la versión más reciente.
- **Ubuntu Server 26.04 LTS “Resolute Raccoon”**: es la LTS más nueva, publicada apenas en abril de este año. Es una opción válida si el estudiante prefiere trabajar con la versión más reciente, pero al ser tan reciente puede tener más fricciones (paquetes de MariaDB recién adaptados, menos tutoriales de terceros, menos tiempo de maduración del primer punto de actualización).

Decisión para este laboratorio: se usará **Ubuntu Server 24.04.x LTS**. Si el estudiante desea usar 26.04 LTS en su lugar, el procedimiento de esta fase es idéntico; solo cambia el nombre del archivo descargado.

4. Verificación de integridad (checksum). Un archivo de varios cientos de megabytes puede corromperse durante la descarga (conexión interrumpida, proxy institucional que modifica el tráfico, disco con sectores defectuosos) sin que el sistema operativo lo reporte como error. Para detectar esto, Canonical publica junto con cada ISO un archivo `SHA256SUMS` que contiene el hash criptográfico SHA-256 esperado de cada imagen. Si el hash calculado sobre el archivo descargado no coincide exactamente con el publicado, **el archivo está corrupto o fue alterado y no debe usarse** para instalar ningún nodo.

5. Por qué importa especialmente en este laboratorio. Como la imagen ISO descargada en esta fase se reutilizará para instalar los 6–7 nodos del plan completo (Fases 5 en adelante), un archivo corrupto introduciría errores intermitentes y difíciles de diagnosticar más adelante (fallas de instalación de paquetes, comportamiento errático del kernel) que podrían confundirse con problemas de replicación o de red. Verificar una sola vez aquí evita ese riesgo en las siete instalaciones futuras.

C. Procedimiento paso a paso

Paso 1 — Confirmar que la carpeta de destino existe.

La carpeta `C:\LabBDD\ISOs` debió crearse en la Fase 0 (Paso 8). Si no existe, créala antes de continuar (ver sección D).

Paso 2 — Verificar espacio libre en disco.

La ISO de Ubuntu Server pesa aproximadamente 2–3 GB. Confirma que tienes al menos 10 GB libres antes de iniciar la descarga, considerando que más adelante se necesitará espacio adicional para los discos virtuales.

Paso 3 — Ir al sitio oficial de descarga.

Navega a <https://ubuntu.com/download/server> (o directamente a <https://releases.ubuntu.com/24.04/> para acceder al listado de archivos del release). **No descargues la ISO desde sitios de terceros, blogs o “mirrors” no oficiales** — el riesgo de obtener una imagen modificada es real y el paso de verificación de checksum pierde sentido si la fuente ya no es confiable.

Paso 4 — Descargar la imagen ISO “live server”.

Selecciona la descarga directa (no torrent, salvo que tu conexión sea muy lenta o inestable, en cuyo caso el torrent es una alternativa válida y oficial). El archivo tendrá un nombre similar a:

Text

```
ubuntu-24.04.4-live-server-amd64.iso
```

Paso 5 — Descargar el archivo de checksums oficial.

En la misma página de releases, descarga también el archivo [SHA256SUMS](#) (texto plano con los hashes de todos los archivos de ese release). Guárdalo en la misma carpeta temporal donde quedó la ISO.

Paso 6 — Calcular el hash SHA-256 del archivo descargado.

Usa PowerShell para calcular el hash real del archivo que descargaste (ver comandos en sección D).

Paso 7 — Comparar el hash calculado contra el publicado.

Abre [SHA256SUMS](#) con un editor de texto y localiza la línea correspondiente al nombre exacto de tu archivo ISO. Compara el valor hexadecimal carácter por carácter (o usa el comando de comparación automática de la sección D). Deben coincidir exactamente.

Paso 8 — Mover la ISO verificada a la carpeta del proyecto.

Una vez confirmada la integridad, mueve (no copies, para no dejar duplicados) el archivo `.iso` a `C:\LabBDD\ISOs`.

Paso 9 — Documentar la descarga.

Anota en `C:\LabBDD\Documentacion` (o en tu bitácora de la Fase 0) la versión exacta descargada (ej. `24.04.4`), la fecha de descarga y el resultado de la verificación de hash. Esto

será útil si en fases posteriores necesitas reinstalar un nodo y quieres asegurarte de usar la misma versión que los demás.

Paso 10 — Validar con el checklist de la sección G.

D. Comandos completos

D.1 Crear la carpeta de destino (si no existe todavía)

```
PowerShell
mkdir C:\LabBDD\ISOs -Force
```

D.2 Verificar espacio libre en disco antes de descargar

```
PowerShell
wmic logicaldisk get size,freespace,caption
```

D.3 Calcular el hash SHA-256 del archivo ISO descargado

Ajusta la ruta al lugar donde el navegador guardó el archivo (normalmente **Descargas**):

```
PowerShell
Get-FileHash "$env:USERPROFILE\Downloads\ubuntu-24.04.4-live-server-amd64.iso"
-Algorithm SHA256
```

El resultado se ve así (el valor real será distinto):

```
Text

Algorithm      Hash
Path
-----
-----
SHA256         A1B2C3D4E5F6...
C:\Users\...\ubuntu-24.04.4-live-server-amd64.iso
```

D.4 Comparar automáticamente el hash contra el archivo SHA256SUMS

Si descargaste también `SHA256SUMS` en la misma carpeta, este script evita la comparación manual:

 PowerShell

```
$archivoIso = "ubuntu-24.04.4-live-server-amd64.iso"
$rutaIso = "$env:USERPROFILE\Downloads\$archivoIso"
$rutaSums = "$env:USERPROFILE\Downloads\SHA256SUMS"
$hashCalculado = (Get-FileHash $rutaIso -Algorithm SHA256).Hash.ToLower()
$lineaEsperada = Select-String -Path $rutaSums -Pattern $archivoIso
$hashEsperado = ($lineaEsperada.Line -split "\s+")[0].ToLower()
if ($hashCalculado -eq $hashEsperado) {
    Write-Host "OK: el hash coincide. La ISO esta integra." -
ForegroundColor Green
} else {
    Write-Host "ERROR: el hash NO coincide. Vuelve a descargar el archivo." -
ForegroundColor Red
}
```

D.5 Mover la ISO verificada a la carpeta del proyecto

 PowerShell

```
Move-Item "$env:USERPROFILE\Downloads\ubuntu-24.04.4-live-server-
amd64.iso" "C:\LabBDD\ISOs\"
```

D.6 Confirmar que el archivo quedó en su lugar y con el tamaño esperado

 PowerShell

```
Get-Item "C:\LabBDD\ISOs\ubuntu-24.04.4-live-server-amd64.iso" | Select-Object
Name, Length, LastWriteTime
```

E. Verificación de funcionamiento

Esta fase se considera completa cuando se cumple lo siguiente:

1. El archivo `.iso` existe físicamente en `C:\LabBDD\ISOs` (verificado con `Get-Item`, sección D.6).
2. El tamaño del archivo es coherente con una imagen “live server” de Ubuntu (aproximadamente 2–3 GB; un archivo de pocos KB o MB indica una descarga fallida o incompleta).

3. El hash SHA-256 calculado sobre el archivo coincide exactamente con el publicado en `SHA256SUMS` para esa misma versión.
4. El estudiante puede explicar, sin ver este documento, qué problema resuelve la verificación de checksum y qué pasaría si se omitiera este paso.
5. La versión descargada (24.04.x LTS o 26.04.x LTS) quedó documentada por escrito junto con la fecha de descarga.

F. Problemas comunes y soluciones

Problema	Causa probable	Solución
La descarga se detiene o falla repetidamente	Conexión inestable, proxy institucional que corta descargas largas	Usar la opción de descarga por BitTorrent disponible en la misma página oficial, que reanuda automáticamente partes incompletas
El hash calculado no coincide con el publicado	Descarga corrupta, o se descargó el archivo de una fuente no oficial	Eliminar el archivo y volver a descargarlo desde <code>releases.ubuntu.com</code> ; nunca usar una ISO cuyo hash no coincida
El antivirus marca el archivo <code>.iso</code> como sospechoso o lo bloquea	Falsos positivos frecuentes con archivos de instalación de Linux en software corporativo/educativo	Solicitar excepción para <code>C:\LabBDD\ISOs</code> en el antivirus institucional (mismo procedimiento que se documentó en la Fase 0 para la carpeta del proyecto)
No queda claro qué línea de <code>SHA256SUMS</code> corresponde al archivo descargado	El archivo <code>SHA256SUMS</code> lista los hashes de <i>todas</i> las variantes del release (desktop, server, distintas arquitecturas)	Buscar específicamente la línea que contiene el nombre exacto del archivo descargado (ej. <code>live-server-amd64</code>), usando <code>Select-String</code> como en el comando de la sección D.4
Poco espacio en disco al intentar mover el archivo	La carpeta <code>C:\LabBDD</code> quedó en una unidad con poco espacio libre	Liberar espacio o, si es necesario, mover toda la carpeta <code>C:\LabBDD</code> a una unidad con más espacio antes de continuar a fases posteriores
Se descargó por error una versión interina (no LTS)	Confusión en el sitio de descarga entre releases LTS e interinos	Verificar que el nombre del archivo y la página de origen indiquen explícitamente "LTS"; eliminar el archivo interino y descargar la versión correcta

G. Checklist de validación

- ☐ Confirmé que existe la carpeta `C:\LabBDD\ISOs`.
 - ☐ Verifiqué que tengo al menos 10 GB libres en disco antes de descargar.
 - ☐ Descargué la imagen ISO de Ubuntu Server **LTS** (24.04.x o 26.04.x) desde una fuente oficial (`ubuntu.com` o `releases.ubuntu.com`).
 - ☐ Descargué también el archivo `SHA256SUMS` correspondiente al mismo release.
 - ☐ Calculé el hash SHA-256 de mi archivo descargado con `Get-FileHash`.
 - ☐ Comparé el hash calculado contra el publicado y **coinciden exactamente**.
 - ☐ Moví la ISO verificada a `C:\LabBDD\ISOs`.
 - ☐ Confirmé el tamaño y la ubicación final del archivo.
 - ☐ Documenté la versión exacta y la fecha de descarga.
 - ☐ Puedo explicar qué es una imagen ISO y por qué se verifica su checksum antes de usarla.
-

Fase 5 — Instalación de Ubuntu Server

24.04 LTS en bdd-nodo01 y bdd-nodo02

*Continuación directa de las Fases 3 (creación de las VMs vacías) y 4 (descarga y verificación de la ISO). En esta fase se instala por primera vez un sistema operativo real dentro de las dos máquinas virtuales ya existentes. **No se configura todavía la IP estática** (eso es la Fase 6); aquí solo se deja Ubuntu Server funcionando y accesible desde la consola gráfica de VirtualBox.*

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Montar una imagen ISO en la unidad óptica virtual de una VM y ajustar el orden de arranque para instalar un sistema operativo desde ella.
2. Completar la instalación de Ubuntu Server 24.04 LTS usando el instalador Subiquity, tomando decisiones informadas en cada pantalla (almacenamiento, perfil, SSH, snaps).
3. Explicar por qué se instala el paquete **OpenSSH Server** durante la instalación, en lugar de agregarlo después.
4. Entender por qué la pantalla de red del instalador mostrará la interfaz sin dirección IP, y por qué eso es el comportamiento esperado dado lo configurado en la Fase 2.
5. Repetir el proceso de forma idéntica en dos nodos distintos, garantizando consistencia (mismo usuario administrador, mismas decisiones de particionado) entre ambos.
6. Retirar correctamente el medio de instalación tras el primer reinicio, evitando que la VM vuelva a arrancar el instalador en lugar del sistema ya instalado.
7. Cerrar la fase con un snapshot por VM, dejando un punto de restauración limpio antes de tocar la red (Fase 6).

B. Conceptos teóricos necesarios

1. El instalador Subiquity. Desde Ubuntu 20.04, el instalador de Ubuntu Server usa **Subiquity**, una interfaz de texto (TUI) navegable con teclado, mucho más ligera que el antiguo instalador gráfico de Ubuntu Desktop. Es el mismo instalador tanto si se usa el modo gráfico de VirtualBox como si se accede por consola serie; en este laboratorio se usará dentro de la ventana de la VM en modo gráfico, simplemente para poder ver el proceso con claridad.

2. Orden de arranque y medio de instalación. Una VM recién creada (Fase 3) tiene un disco vacío y una unidad óptica vacía: no hay nada que arrancar. Al montar la ISO de Ubuntu en la unidad óptica y arrancar la VM, el firmware intenta los dispositivos en el orden configurado (`boot1`, `boot2`, ...); si la unidad óptica tiene prioridad y contiene un medio arrancable, el instalador toma el control antes que el disco duro vacío.

3. Por qué instalar OpenSSH Server desde el instalador. Todo este laboratorio se administrará de forma remota, por SSH, desde el host Windows (Fase 6 en adelante: scripts, DBeaver, replicación). Subiquity ofrece una casilla explícita **“Install OpenSSH server”** durante la instalación; marcarla evita tener que volver a entrar por la consola gráfica de VirtualBox únicamente para instalar el paquete a mano. Es, además, la práctica estándar al aprovisionar servidores reales sin entorno gráfico.

4. Por qué la pantalla de red mostrará “sin dirección”. En la Fase 2 se desactivó deliberadamente el servidor DHCP del adaptador Host-Only, porque el laboratorio depende de IPs fijas. Como consecuencia, cuando Subiquity llegue a la pantalla de configuración de red, la interfaz `enp0s3` (o el nombre que asigne el kernel) aparecerá **sin dirección IPv4 asignada**. Esto es exactamente lo esperado en esta fase: no hay que configurar nada manualmente aquí, simplemente continuar. La IP estática se asignará dentro del sistema operativo ya instalado, en la Fase 6.

5. Consecuencia práctica: no hay acceso a internet durante la instalación. Como las VMs solo tienen un adaptador Host-Only (sin NAT) y sin DHCP, no existe ruta de salida a internet en este punto. Subiquity detectará esto y, según la versión, omitirá automáticamente la selección de mirror/proxy y la instalación de actualizaciones y snaps destacados, o mostrará una advertencia de “no se pudo verificar el mirror” que se puede ignorar y continuar. **Esto es normal y no es un error.** Si una fase posterior requiere acceso a internet dentro de las VMs (por ejemplo, para instalar el paquete de MariaDB), se resolverá en esa fase; no es necesario adelantarlo aquí.

6. Particionado guiado y LVM. Subiquity ofrece “Use an entire disk” (particionado guiado) como opción por defecto, con la casilla “Set up this disk as an LVM group” generalmente marcada. Usar LVM agrega una capa de volúmenes lógicos que facilita redimensionar particiones más adelante sin reparticionar desde cero; para discos de 20 GB dedicados a un solo nodo no es estrictamente indispensable, pero se deja el valor por defecto (LVM activado) por ser la práctica recomendada y no añade complejidad real en esta fase.

7. Snaps destacados (“Featured Server Snaps”). Subiquity ofrece instalar paquetes adicionales empaquetados como snap (Docker, Nextcloud, etc.) al final del proceso. Ninguno es necesario para este laboratorio (no se usa Docker, según el alcance definido en la Fase 0) y, sin red disponible, tampoco podrían descargarse. Se dejan todos sin marcar.

8. Consistencia entre nodos. Para que los scripts y procedimientos de fases posteriores (replicación, fragmentación) funcionen igual en todos los nodos, ambas VMs deben instalarse con el mismo usuario administrador, el mismo esquema de particionado y la misma versión de Ubuntu. Solo debe cambiar el nombre de host (`bdd-nodo01` vs. `bdd-nodo02`).

C. Procedimiento paso a paso

Paso 1 — Confirmar que las Fases 3 y 4 están completas.

Deben existir `bdd-nodo01` y `bdd-nodo02` (apagadas, sin sistema operativo) y el archivo `ubuntu-24.04.4-live-server-amd64.iso` verificado en `C:\LabBDD\ISOs`.

Paso 2 — Montar la ISO en la unidad óptica de `bdd-nodo01`.

La VM debe estar apagada. Ver comando en sección D.1.

Paso 3 — Ajustar el orden de arranque para que la unidad óptica tenga prioridad.

Ver comando en sección D.2. (En la práctica, como el disco duro está vacío, casi cualquier orden funcionaría, pero fijarlo explícitamente evita ambigüedad).

Paso 4 — Iniciar `bdd-nodo01` en modo gráfico.

Ver comando en sección D.3. Se abrirá una ventana de VirtualBox mostrando el arranque del instalador.

Paso 5 — Navegar el instalador Subiquity.

En orden, las pantallas relevantes y la decisión recomendada para este laboratorio:



Untitled

Paso 6 — Retirar el medio de instalación.

Al reiniciar, Subiquity normalmente expulsa la ISO automáticamente y muestra el mensaje "Please remove the installation medium, then press ENTER". Si la VM vuelve a arrancar el instalador en lugar del sistema instalado, apagarla y ejecutar manualmente el comando de la sección D.4 para desmontar la ISO, luego volver a iniciarla.

Paso 7 — Primer inicio de sesión y verificación básica.

Una vez que aparezca el prompt de login (`bdd-nodo01 login:`), iniciar sesión con el usuario y contraseña definidos en el Paso 5. Ejecutar los comandos de verificación de la sección D.5 dentro de la VM.

Paso 8 — Repetir los Pasos 2 a 7 para `bdd-nodo02`.

Mismo procedimiento exacto, cambiando únicamente el nombre del servidor a `bdd-nodo02` en la pantalla de perfil. Usar el mismo nombre de usuario (`bddadmin`) por consistencia.

Paso 9 — Apagar ambas VMs de forma ordenada.

Dentro de cada VM, no desde el botón de cerrar la ventana de VirtualBox (ver comando D.6).

Paso 10 — Tomar el snapshot de cierre de fase.

Un snapshot por VM, nombrado `fase05-completa` (ver sección D.7), antes de tocar la configuración de red en la Fase 6.

Paso 11 — Validar con el checklist de la sección G.

D. Comandos completos

Comandos en PowerShell sobre el host Windows, salvo donde se indique explícitamente “dentro de la VM”.

D.1 Montar la ISO en `bdd-nodo01` (VM apagada)

 PowerShell

```
VBoxManage storageattach "bdd-nodo01" --storagectl "SATA Controller" --port 1  
--device 0 --type dvddrive --medium "C:\LabBDD\ISOs\ubuntu-24.04.4-live-  
server-amd64.iso"
```

D.2 Fijar el orden de arranque (DVD primero, disco después)

 PowerShell

```
VBoxManage modifyvm "bdd-nodo01" --boot1 dvd --boot2 disk --boot3 none --  
boot4 none
```

D.3 Iniciar la VM en modo gráfico

 PowerShell

```
VBoxManage startvm "bdd-nodo01" --type gui
```

A partir de aquí la interacción es dentro de la ventana de la VM (instalador Subiquity), según el Paso 5 de la sección C. No hay más comandos de PowerShell hasta que termine la instalación.

D.4 Desmontar la ISO manualmente (si el instalador no la expulsa solo)

 PowerShell

```
VBoxManage storageattach "bdd-nodo01" --storagectl "SATA Controller" --port 1  
--device 0 --type dvddrive --medium emptydrive
```

D.5 Verificación básica dentro de la VM (tras el primer login)

 Bash

```
hostnamectl
```

Debe mostrar `Static hostname: bdd-nodo01`.

 Bash

```
lsb_release -a
```

Debe mostrar `Ubuntu 24.04.x LTS (Noble Numbat)`.

 Bash

```
ip a
```

La interfaz `enp0s3` debe aparecer **sin** dirección IPv4 (esperado; se asigna en Fase 6).

 Bash

```
systemctl status ssh
```

Debe mostrar `active (running)`, confirmando que OpenSSH Server quedó instalado y activo.

 Bash

```
free -h  
df -h /
```

Para confirmar que la RAM (≈1.5 GB) y el disco (≈20 GB, con poco uso real) coinciden con lo planeado en la Fase 0.

D.6 Apagar la VM de forma ordenada (dentro de la VM)

 Bash

```
sudo poweroff
```

Alternativa desde el host, si la VM quedó sin responder (no usar como método habitual):

 PowerShell

```
VBoxManage controlvm "bdd-nodo01" acpipowerbutton
```

D.7 Repetir D.1 a D.6 para `bdd-nodo02`

Mismos comandos, sustituyendo `bdd-nodo01` por `bdd-nodo02` en cada uno.

D.8 Tomar el snapshot de cierre de fase (ambas VMs apagadas)

 PowerShell

```
VBoxManage snapshot "bdd-nodo01" take "fase05-completa" --description "Ubuntu  
Server 24.04 LTS instalado, OpenSSH activo, sin IP estatica todavia"  
VBoxManage snapshot "bdd-nodo02" take "fase05-completa" --description "Ubuntu  
Server 24.04 LTS instalado, OpenSSH activo, sin IP estatica todavia"
```

D.9 Confirmar los snapshots creados

 PowerShell

```
VBoxManage snapshot "bdd-nodo01" list  
VBoxManage snapshot "bdd-nodo02" list
```

E. Verificación de funcionamiento

1. `bdd-nodo01` y `bdd-nodo02` arrancan y muestran el prompt de login de Ubuntu Server (no el instalador) tras reiniciar.

2. `hostnamectl` dentro de cada VM reporta el nombre de host correcto (`bdd-nodo01` / `bdd-nodo02` respectivamente).
 3. `lsb_release -a` confirma Ubuntu 24.04.x LTS en ambos nodos.
 4. `systemctl status ssh` reporta el servicio activo en ambos nodos (aunque todavía no se pueda conectar por SSH desde el host, porque falta la IP estática de la Fase 6).
 5. `ip a` muestra la interfaz `enp0s3` presente pero sin IPv4 asignada — comportamiento esperado, no un fallo.
 6. Ambas VMs se apagaron limpiamente (`VBoxManage list runningvms` no las muestra activas).
 7. `VBoxManage snapshot "bdd-nodo01" list` y el equivalente para `bdd-nodo02` muestran el snapshot `fase05-completa` .
 8. El estudiante puede explicar por qué no hay IP ni acceso a internet en este punto, y por qué eso no impide considerar la fase exitosa.
-

F. Problemas comunes y soluciones

Problema	Causa probable	Solución
La VM vuelve a arrancar el instalador después de “Reboot Now”	La ISO no se expulsó automáticamente y sigue teniendo prioridad de arranque	Apagar la VM y ejecutar el comando D.4 para desmontar la ISO; volver a iniciar
El instalador se queda “congelado” en la pantalla de mirror/red durante varios minutos	Está intentando verificar la conectividad a internet, que no existe en la red Host-Only	Esperar a que expire el timeout (1–2 minutos) y elegir continuar sin verificar; o, si el instalador lo permite, seleccionar explícitamente “Continue without network”
No se puede seleccionar “Use an entire disk” porque no aparece ningún disco	El controlador SATA o el <code>.vdi</code> no quedaron correctamente adjuntados en la Fase 3	Apagar la VM, revisar con <code>VBoxManage showvminfo "bdd-nodo01"</code> que el disco esté en <code>port 0, device 0</code> ; volver a adjuntarlo si es necesario
La pantalla del instalador se ve cortada o con artefactos visuales	Memoria de video (VRAM) insuficiente para la resolución que intenta usar VirtualBox	Apagar la VM y aumentar VRAM con <code>VBoxManage modifyvm "bdd-nodo01" --vram 32</code> , luego reiniciar la instalación
Después de instalar, <code>systemctl status ssh</code> no existe o el servicio no está	No se marcó la casilla “Install OpenSSH server” durante la instalación	Dentro de la VM, instalar manualmente con <code>sudo apt update && sudo apt install openssh-server</code> (requiere red temporal; si no hay red disponible, reinstalar el sistema repitiendo el Paso 5 y marcando la casilla correctamente)
El teclado dentro de la ventana de la VM escribe caracteres distintos a los presionados	Distribución de teclado seleccionada en el instalador no coincide con el teclado físico	Verificar la distribución elegida en el Paso 5; puede corregirse después con <code>sudo dpkg-reconfigure keyboard-configuration</code> dentro de la VM
La VM no responde al <code>sudo poweroff</code> ni a ningún comando	Cuelgue poco frecuente del invitado durante el primer arranque	Usar <code>VBoxManage controlvm "bdd-nodo01" acpipowerbutton</code> ; si tampoco responde, <code>VBoxManage controlvm "bdd-nodo01" poweroff</code> como último recurso (apagado forzado, no recomendado como práctica habitual)

G. Checklist de validación

- ☐ Monté la ISO verificada de Ubuntu Server 24.04 LTS en la unidad óptica de `bdd-nodo01`.
 - ☐ Completé la instalación de Subiquity en `bdd-nodo01`, configurando el nombre de host como `bdd-nodo01`.
 - ☐ Marqué la casilla "Install OpenSSH server" durante la instalación de `bdd-nodo01`.
 - ☐ Usé particionado guiado ("Use an entire disk") con LVM en `bdd-nodo01`.
 - ☐ No marqué ningún snap destacado en `bdd-nodo01`.
 - ☐ Retiré correctamente el medio de instalación y `bdd-nodo01` arranca directo al login de Ubuntu Server.
 - ☐ Verifiqué `hostnamectl`, `lsb_release -a` y `systemctl status ssh` dentro de `bdd-nodo01`.
 - ☐ Repetí los 7 puntos anteriores exitosamente para `bdd-nodo02`.
 - ☐ Usé el mismo nombre de usuario administrador en ambos nodos.
 - ☐ Apagué ambas VMs de forma ordenada (`sudo poweroff` dentro de cada una).
 - ☐ Tomé el snapshot `fase05-completa` en `bdd-nodo01` y en `bdd-nodo02`.
 - ☐ Puedo explicar por qué ninguna VM tiene IP ni acceso a internet en este punto, y por qué eso es lo esperado.
-

Preparación para la siguiente fase

La **Fase 6: Configuración de IP estática dentro de los nodos** requerirá:

- Ambas VMs instaladas, con OpenSSH activo y snapshot `fase05-completa` tomado (esta fase).
 - El plan de direccionamiento IP definido en la Fase 0 (sección C.3): `bdd-nodo01 → 192.168.56.101`, `bdd-nodo02 → 192.168.56.102`.
 - Acceso a la consola gráfica de cada VM en VirtualBox, ya que la configuración de red estática se hará editando archivos de Netplan directamente dentro de cada nodo, antes de que el acceso por SSH esté disponible.
-

Fase 6 — Configuración de IP Estática dentro de los Nodos (Netplan)

Continuación directa de la Fase 5. Ambas VMs (`bdd-nodo01` , `bdd-nodo02`) tienen Ubuntu Server 24.04 LTS instalado, OpenSSH activo, y un snapshot `fase05-completa` . En esta fase se asigna, **dentro de cada sistema operativo invitado**, la dirección IP estática reservada desde la Fase 0 (sección C.3), dejando los nodos accesibles por SSH desde el host por primera vez.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Explicar qué es Netplan y cómo se relaciona con `systemd-networkd` en Ubuntu Server (a diferencia de NetworkManager, usado en Ubuntu Desktop).
2. Identificar el archivo de configuración de red generado automáticamente por el instalador (`/etc/netplan/50-cloud-init.yaml`) y modificarlo de forma segura.
3. Asignar una dirección IPv4 estática a la interfaz `enp0s3` de cada nodo, conforme al plan de direccionamiento de la Fase 0.
4. Aplicar los cambios de red sin reiniciar la VM, usando `netplan apply` , y entender el mecanismo de seguridad de `netplan try` .
5. Justificar por qué **no** se configura puerta de enlace (gateway) ni servidores DNS en esta fase, dado el diseño de red Host-Only definido en la Fase 2.
6. Configurar resolución de nombres básica vía `/etc/hosts` en cada nodo (y, opcionalmente, en el host Windows) para poder referirse a los nodos por nombre en lugar de por IP en fases futuras.
7. Verificar conectividad bidireccional host↔VM y VM↔VM, y establecer la primera conexión SSH real desde el host Windows.
8. Cerrar la fase con un snapshot `fase06-completa` en ambas VMs.

B. Conceptos teóricos necesarios

1. Netplan. Es la herramienta de configuración de red por defecto en Ubuntu desde la versión 18.04 (tanto Desktop como Server). No es un servicio en sí mismo: es una capa de abstracción que traduce archivos de configuración en formato YAML (ubicados en `/etc/netplan/`) hacia el backend real que gestiona la red — en Ubuntu Server ese backend es `systemd-networkd` ;

en Ubuntu Desktop suele ser NetworkManager. Esta diferencia explica por qué los tutoriales orientados a escritorio (que hablan de editar conexiones desde una interfaz gráfica) no aplican aquí.

2. El archivo generado por Subiquity. Durante la instalación (Fase 5), como la pantalla de red no tenía nada que configurar (sin DHCP disponible), Subiquity de todos modos generó un archivo de Netplan por defecto, normalmente llamado `/etc/netplan/50-cloud-init.yaml`, con la interfaz configurada en modo `dhcp4: true`. Este archivo fue escrito por `cloud-init` durante el primer arranque (`curtin` / `cloud-init` son parte del proceso de instalación de Subiquity). A partir de Ubuntu 20.04, el instalador deja además un archivo en `/etc/cloud/cloud.cfg.d/` que **deshabilita la gestión de red por cloud-init en arranques posteriores**, por lo que es seguro editar manualmente `50-cloud-init.yaml` sin que `cloud-init` lo sobrescriba en el siguiente reinicio. Aun así, en esta guía se aplican los cambios con `netplan apply` (sin reiniciar), lo cual permite verificar el comportamiento sin depender de ese detalle.

3. Por qué no se configura gateway ni DNS en esta fase. El adaptador Host-Only (Fase 2) conecta exclusivamente al host y a las demás VMs del laboratorio; no existe ningún router en ese segmento que pueda actuar como puerta de enlace hacia internet. Definir una `gateway4` o servidores `nameservers` en este punto no tendría ningún destino válido y solo generaría confusión. Esto es consistente con lo ya advertido en la Fase 5 (sección B.5): los nodos seguirán sin acceso a internet después de esta fase. Si una fase posterior requiere que los nodos instalen paquetes (por ejemplo, MariaDB), se resolverá entonces — típicamente agregando un **segundo adaptador de red en modo NAT** únicamente para salida a internet, dejando el adaptador Host-Only intacto para la comunicación interna del clúster. Por ahora, **un solo adaptador, sin gateway, es correcto.**

4. Formato YAML de Netplan (sintaxis mínima necesaria). Una declaración de IP estática para una interfaz Ethernet sigue esta estructura:

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 192.168.56.101/24
```

`/24` es la notación CIDR equivalente a la máscara `255.255.255.0` definida en la Fase 0 y aplicada al adaptador Host-Only en la Fase 2. La indentación en YAML es significativa (espacios, nunca tabuladores); un error de indentación es la causa más común de fallas en

esta fase.

5. `netplan apply` vs. `netplan try`. `netplan apply` aplica la configuración inmediatamente y de forma permanente. `netplan try` la aplica de forma temporal (revierte automáticamente a los 120 segundos si no se confirma con Enter), lo cual es más seguro cuando se edita la red de una máquina a la que solo se tiene acceso por esa misma red — no es el caso aquí, porque se trabaja desde la consola gráfica de VirtualBox, pero es una buena práctica que conviene conocer para sistemas administrados exclusivamente por SSH.

6. Permisos del archivo de Netplan. Netplan exige que sus archivos YAML tengan permisos restrictivos (`600` , solo lectura/escritura para `root`) porque pueden contener credenciales de red (por ejemplo, contraseñas de Wi-Fi en otros contextos). Si el archivo se editó con permisos más abiertos, Netplan emite una advertencia y puede negarse a aplicarlo en versiones recientes.

7. `/etc/hosts` como resolución de nombres mínima. Este laboratorio no contempla levantar un servidor DNS propio. Para poder referirse a los nodos por nombre (`ssh bddadmin@bdd-nodo02` en vez de `ssh bddadmin@192.168.56.102`) — algo que facilitará mucho la configuración de replicación en fases futuras — se agrega una entrada estática por nodo en el archivo `/etc/hosts` de cada VM. Cada vez que se incorpore un nodo nuevo al laboratorio (Fases 11, 13, 14...), se deberá añadir su entrada correspondiente en `/etc/hosts` de **todos** los nodos existentes; por ahora solo se agregan las entradas de `bdd-nodo01` y `bdd-nodo02` .

C. Procedimiento paso a paso

Paso 1 — Encender `bdd-nodo01` desde VirtualBox.

Debe arrancar directo al prompt de login (sin instalador), tal como quedó al cierre de la Fase 5.

Paso 2 — Iniciar sesión con el usuario `bddadmin` .

Usar la contraseña definida durante la instalación.

Paso 3 — Identificar el archivo de Netplan existente.

Ver comando en sección D.1. Confirmar el nombre exacto del archivo y de la interfaz de red detectada por el kernel.

Paso 4 — Hacer una copia de respaldo del archivo original antes de editarlo.

Ver comando en sección D.2. Buena práctica antes de modificar cualquier archivo de configuración crítico.

Paso 5 — Editar el archivo de Netplan con la IP estática correspondiente a `bdd-nodo01` .

`192.168.56.101/24` , sin `gateway4` ni `nameservers` (ver justificación en B.3). Ver contenido completo en sección D.3.

Paso 6 — Corregir permisos del archivo, si es necesario.

Ver comando en sección D.4.

Paso 7 — Aplicar la configuración.

Ver comando en sección D.5. No requiere reiniciar la VM.

Paso 8 — Verificar la IP asignada dentro de la VM.

Ver comando en sección D.6.

Paso 9 — Agregar las entradas de `/etc/hosts` dentro de `bdd-nodo01`.

Ver contenido en sección D.7.

Paso 10 — Repetir los Pasos 1 a 9 para `bdd-nodo02`, usando la IP `192.168.56.102/24` y el mismo contenido de `/etc/hosts` (las entradas son idénticas en ambos nodos).

Paso 11 — Verificar conectividad desde el host Windows hacia ambos nodos.

Ver comandos en sección D.8.

Paso 12 — Verificar conectividad entre los propios nodos (VM↔VM).

Desde `bdd-nodo01`, hacer ping a `bdd-nodo02` y viceversa (sección D.9).

Paso 13 — Establecer la primera conexión SSH real desde el host.

Ver comando en sección D.10.

Paso 14 (opcional, recomendado) — Agregar entradas equivalentes en el archivo `hosts` de Windows.

Permite usar `ssh bddadmin@bdd-nodo01` desde PowerShell sin recordar la IP. Ver sección D.11.

Paso 15 — Apagar ambas VMs de forma ordenada y tomar el snapshot de cierre de fase.

Ver secciones D.12 y D.13.

Paso 16 — Validar con el checklist de la sección G.

D. Comandos completos

Los comandos marcados como “dentro de la VM” se ejecutan en la consola de Ubuntu Server abierta por VirtualBox (login con `bddadmin`). Los demás se ejecutan en PowerShell, en el host Windows.

D.1 Identificar el archivo de Netplan y el nombre de la interfaz (dentro de la VM)

```
Bash

ls /etc/netplan/
cat /etc/netplan/50-cloud-init.yaml
ip a
```

La interfaz relevante (la única, además de `lo`) debería llamarse `enp0s3`. Si el nombre reportado por `ip a` es distinto (por ejemplo `ens3` en algunas combinaciones de chipset/versión de VirtualBox), usar ese nombre exacto en todos los pasos siguientes.

D.2 Respalidar el archivo original (dentro de la VM)

```
Bash

sudo cp /etc/netplan/50-cloud-init.yaml /etc/netplan/50-
cloud-init.yaml.bak
```

D.3 Editar el archivo de Netplan (dentro de la VM)

```
Bash

sudo nano /etc/netplan/50-cloud-init.yaml
```

Contenido completo para `bdd-nodo01` (reemplazar todo el contenido del archivo):

```
YAML

network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 192.168.56.101/24
```

Para `bdd-nodo02`, el único cambio es la dirección:

 YAML

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 192.168.56.102/24
```

Guardar con `Ctrl+O`, `Enter`, y salir con `Ctrl+X`.

D.4 Corregir permisos del archivo (dentro de la VM)

 Bash

```
sudo chmod 600 /etc/netplan/50-cloud-init.yaml
```

D.5 Aplicar la configuración (dentro de la VM)

 Bash

```
sudo netplan apply
```

Si se prefiere verificar antes de confirmar de forma permanente (no estrictamente necesario aquí, ya que se trabaja desde la consola local de VirtualBox y no por SSH):

 Bash

```
sudo netplan try
```

D.6 Verificar la IP asignada (dentro de la VM)

 Bash

```
ip a show enp0s3
```

Debe mostrar `inet 192.168.56.101/24` (o `.102` en el segundo nodo) en estado `UP`.

D.7 Agregar entradas en `/etc/hosts` (dentro de cada VM, contenido idéntico en ambas)

 Bash

```
sudo nano /etc/hosts
```

Agregar al final del archivo (sin eliminar las líneas existentes de `127.0.0.1` y `::1`):

 Text

```
192.168.56.101    bdd-nodo01
192.168.56.102    bdd-nodo02
```

*Nota: a medida que se incorporen nuevos nodos en fases futuras (03, 04, 05, 06, cliente), estas líneas deberán ampliarse con las IPs correspondientes en **todos** los nodos existentes, conforme a la tabla de direccionamiento de la Fase 0 (sección C.3).*

D.8 Verificar conectividad desde el host Windows (PowerShell)

 PowerShell

```
ping 192.168.56.101
ping 192.168.56.102
```

Ambos deben responder sin pérdida de paquetes.

D.9 Verificar conectividad entre nodos (dentro de `bdd-nodo01`)

 Bash

```
ping -c 4 bdd-nodo02
```

Debe resolver el nombre vía `/etc/hosts` y recibir respuesta de `192.168.56.102`.

D.10 Primera conexión SSH real desde el host (PowerShell)

 PowerShell

```
ssh bddadmin@192.168.56.101
```

Aceptar la huella digital (fingerprint) del host la primera vez (`yes`), e introducir la contraseña definida en la Fase 5. Repetir para `192.168.56.102` . Para salir de la sesión SSH:

 Bash

```
exit
```

D.11 (Opcional) Agregar entradas en el archivo `hosts` de Windows

Editar como administrador (PowerShell elevado):

 PowerShell

```
notepad C:\Windows\System32\drivers\etc\hosts
```

Agregar al final:

 Text

```
192.168.56.101    bdd-nodo01
192.168.56.102    bdd-nodo02
```

Guardar. A partir de aquí, `ssh bddadmin@bdd-nodo01` funciona igual que usar la IP directamente.

D.12 Apagar ambas VMs de forma ordenada

Dentro de cada VM (vía consola o la sesión SSH recién verificada):

 Bash

```
sudo poweroff
```

D.13 Tomar el snapshot de cierre de fase (PowerShell, ambas VMs apagadas)

PowerShell

```
VBoxManage snapshot "bdd-nodo01" take "fase06-completa" --description "IP  
estatica 192.168.56.101 asignada via Netplan, SSH verificado desde el host"  
VBoxManage snapshot "bdd-nodo02" take "fase06-completa" --description "IP  
estatica 192.168.56.102 asignada via Netplan, SSH verificado desde el host"
```

Confirmar:

PowerShell

```
VBoxManage snapshot "bdd-nodo01" list  
VBoxManage snapshot "bdd-nodo02" list
```

E. Verificación de funcionamiento

1. `ip a show enp0s3` dentro de cada VM muestra la IP estática correcta (`.101` / `.102`), sin depender de DHCP.
 2. La configuración sobrevive a `netplan apply` sin errores ni advertencias de permisos.
 3. `ping 192.168.56.101` y `ping 192.168.56.102` responden correctamente desde el host Windows.
 4. Desde `bdd-nodo01`, `ping bdd-nodo02` resuelve el nombre vía `/etc/hosts` y recibe respuesta.
 5. `ssh bddadmin@192.168.56.101` (y `.102`) conecta exitosamente desde el host, solicitando únicamente la contraseña (sin errores de red o de host inalcanzable).
 6. Ambos snapshots `fase06-completa` existen y están asociados al estado correcto (apagadas, con IP ya configurada).
 7. El estudiante puede explicar por qué esta fase **no** configuró gateway ni DNS, y qué fase futura resolverá el acceso a internet de los nodos si llega a ser necesario.
-

F. Problemas comunes y soluciones

Problema	Causa probable	Solución
<code>netplan apply</code> devuelve un error de sintaxis YAML	Indentación incorrecta (mezcla de tabuladores y espacios, o niveles desalineados)	Revisar que cada nivel use exactamente 2 espacios adicionales respecto al nivel superior; nunca usar tabuladores; comparar contra el ejemplo exacto de la sección D.3
Advertencia "Permissions for ... are too open"	El archivo quedó con permisos distintos a <code>600</code> tras editarlo	Ejecutar <code>sudo chmod 600 /etc/netplan/50-cloud-init.yaml</code> (sección D.4) y volver a aplicar
<code>ip a</code> sigue mostrando la interfaz sin IP después de <code>netplan apply</code>	Nombre de interfaz mal escrito en el YAML (por ejemplo <code>enp0s3</code> cuando el kernel detectó <code>ens3</code>)	Confirmar el nombre real con <code>ip a</code> (sección D.1) antes de editar, y corregirlo en el archivo YAML
El host no puede hacer <code>ping</code> a la VM aunque <code>ip a</code> muestre la IP correcta	El Adaptador 1 de la VM no está realmente enlazado al adaptador Host-Only correcto, o el firewall de Windows bloquea ICMP en ese segmento	Verificar con <code>VBoxManage showvminfo "bdd-nodo01" findstr NIC</code> que el adaptador esté en modo <code>hostonly</code> ; revisar reglas de Firewall de Windows para permitir ICMP en redes privadas

<code>ssh</code> desde el host falla con "Connection refused"	El servicio SSH no está activo en la VM (poco probable si se siguió la Fase 5), o un firewall interno (<code>ufw</code>) está bloqueando el puerto 22	Dentro de la VM: <pre>sudo systemctl status ssh y sudo ufw status (debe estar inactive por defecto en Ubuntu Server; si está activo y bloqueando, sudo ufw allow 22/tcp)</pre>
<code>ssh</code> falla con "Connection timed out" o "No route to host"	La IP está mal escrita, el adaptador no está realmente arriba (<code>ip link show enp0s3</code> indica <code>DOWN</code>), o ambos nodos quedaron con la misma IP por error de copy-paste	Verificar IP exacta en ambos nodos con <code>ip a</code> ; confirmar que no se repitió la misma dirección en <code>bdd-nodo02</code> por error al copiar el archivo de <code>bdd-nodo01</code>
<code>ping bdd-nodo02</code> desde <code>bdd-nodo01</code> falla por nombre pero funciona por IP	Falta o está mal escrita la entrada en <code>/etc/hosts</code> de <code>bdd-nodo01</code>	Revisar <code>cat /etc/hosts</code> dentro de la VM y corregir la línea correspondiente (sección D.7)
Al reiniciar la VM, la IP estática se pierde y vuelve a DHCP	Cloud-init regeneró el archivo de Netplan porque la gestión de red por cloud-init no quedó deshabilitada en esa instalación particular	Verificar la existencia de un archivo en <code>/etc/cloud/cloud.cfg.d/</code> que contenga <code>network:</code> <code>{config:</code> <code>disabled}</code> ; si no existe, crearlo manualmente con ese contenido y

volver a aplicar
la configuración
de Netplan

G. Checklist de validación

- ☐ Identifiqué correctamente el archivo de Netplan (`/etc/netplan/50-cloud-init.yaml`) y el nombre real de la interfaz en cada VM.
- ☐ Respalde el archivo original antes de editarlo, en ambos nodos.
- ☐ Configuré `bdd-nodo01` con la IP estática `192.168.56.101/24` , sin gateway ni DNS.
- ☐ Configuré `bdd-nodo02` con la IP estática `192.168.56.102/24` , sin gateway ni DNS.
- ☐ Corregí los permisos del archivo YAML a `600` en ambos nodos, si fue necesario.
- ☐ Apliqué la configuración con `netplan apply` sin errores en ambos nodos.
- ☐ Agregué las entradas de `/etc/hosts` (`bdd-nodo01` y `bdd-nodo02`) en **ambas** VMs.
- ☐ Verifiqué `ping` exitoso desde el host Windows hacia ambas IPs.
- ☐ Verifiqué `ping` exitoso entre `bdd-nodo01` y `bdd-nodo02` usando nombres de host.
- ☐ Establecí conexión SSH exitosa desde el host hacia ambos nodos.
- ☐ (Opcional) Agregué las entradas equivalentes en el archivo `hosts` de Windows.
- ☐ Apagué ambas VMs de forma ordenada y tomé el snapshot `fase06-completa` en cada una.
- ☐ Puedo explicar por qué no se configuró gateway ni DNS, y en qué fase futura se resolvería el acceso a internet de los nodos si fuera necesario.

Preparación para la siguiente fase

La **Fase 7** (continuación del plan de crecimiento del laboratorio) requerirá:

- Ambos nodos con IP estática verificada y accesible por SSH desde el host (esta fase).
- Snapshot `fase06-completa` tomado en `bdd-nodo01` y `bdd-nodo02` .
- Acceso SSH funcional, ya que a partir de este punto la administración de los nodos podrá hacerse de forma remota en lugar de usar la consola gráfica de VirtualBox.

Fase 7 — Instalación y Configuración Inicial de MariaDB Server en bdd-nodo01 y bdd-nodo02

Continuación directa de la Fase 6. Ambos nodos tienen Ubuntu Server 24.04 LTS con IP estática configurada, OpenSSH activo y snapshot `fase06-completa`. En esta fase se instala el motor de base de datos **MariaDB Server** en ambas VMs y se realiza su configuración inicial (hardening básico y juego de caracteres). La instalación requiere acceso a internet, que se habilita de forma controlada añadiendo un segundo adaptador de red en modo NAT a cada VM; el adaptador Host-Only de la Fase 2 permanece intacto. La configuración específica de replicación, binary log y usuarios de replicación **no se realiza en esta fase**; corresponde a la Fase 10. Aquí solo se instala y verifica que el motor funciona correctamente en cada nodo de forma independiente.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Justificar la elección de MariaDB frente a otras alternativas (MySQL, PostgreSQL) para este laboratorio de bases de datos distribuidas.
2. Añadir un segundo adaptador de red en modo NAT a una VM ya existente y en producción, sin reinstalar ni reconfigurar el adaptador Host-Only.
3. Actualizar la configuración de Netplan para que ambos adaptadores coexistan correctamente, comprendiendo cómo el kernel de Linux construye la tabla de rutas con dos interfaces activas simultáneas.
4. Instalar `mariadb-server` desde los repositorios oficiales de Ubuntu 24.04 LTS.
5. Ejecutar `mysql_secure_installation` entendiendo el propósito de cada pregunta interactiva y las implicaciones de cada respuesta para el laboratorio.
6. Comprender la estructura de archivos de configuración de MariaDB en Ubuntu (`mariadb.conf.d/`) y el principio de sobrescritura por número de archivo, para añadir configuración propia sin modificar los archivos del paquete.
7. Configurar el juego de caracteres del servidor como `utf8mb4` / `utf8mb4_unicode_ci` y verificar que el motor los aplica.
8. Explicar qué es `bind-address`, cuál es su valor por defecto y por qué **no** se modifica en esta fase.
9. Cerrar la fase con el snapshot `fase07-completa` en ambas VMs.

B. Conceptos teóricos necesarios

1. MariaDB y su relación con MySQL.

MariaDB es una bifurcación (*fork*) directa de MySQL 5.5, creada en 2009 por los autores originales de MySQL tras la adquisición de Sun Microsystems por Oracle. A nivel de protocolo de red, formato de disco y lenguaje SQL, MariaDB mantiene compatibilidad con MySQL en la gran mayoría de los casos: los clientes (`mariadb` / `mysql` , DBeaver, Python/mysql-connector, etc.) se conectan a ambos motores de la misma forma. Sin embargo, MariaDB ha divergido intencionalmente en algunas áreas relevantes para este laboratorio:

- Incluye nativamente el motor de almacenamiento **Spider**, que implementa fragmentación distribuida directamente en el motor (se usará en las Fases 14–16).
- Su implementación de **GTID** (Global Transaction Identifiers) y replicación paralela es más madura y configurable que en MySQL para las topologías que se construirán en las Fases 10–11.
- La licencia **GPL v2** es más permisiva que la de MySQL Enterprise para uso educativo.
- El paquete `mariadb-server` está incluido y mantenido oficialmente en los repositorios de Ubuntu 24.04 LTS, sin necesidad de fuentes externas.

2. Versiones de MariaDB: LTS y versiones de desarrollo.

MariaDB tiene una política de versiones parecida a Ubuntu: existen ramas LTS con soporte extendido y ramas de desarrollo con ciclos de vida más cortos. Ubuntu 24.04 LTS incluye **MariaDB 10.11 LTS**, con soporte oficial hasta febrero de 2028. Existe también la rama **11.4 LTS** (disponible desde el repositorio oficial de la Fundación MariaDB), que es la más reciente al momento de redactar esta guía. Este laboratorio usa **la versión incluida en los repositorios de Ubuntu 24.04 (10.11.x)** por tres razones: no requiere agregar repositorios externos, la documentación disponible en foros y tutoriales de replicación/fragmentación está ampliamente probada en esta rama, y garantiza coherencia entre todos los nodos que se crearán en fases futuras.

3. Por qué se necesita internet ahora, y cómo se incorpora de forma controlada.

Las fases anteriores intencionalmente dejaron los nodos sin acceso a internet: no era necesario para instalar el sistema operativo (ya venía en la ISO) y el aislamiento de la red Host-Only es una práctica correcta de laboratorio. Ahora, para instalar paquetes adicionales mediante `apt install mariadb-server` , sí se necesita conectividad con los repositorios de Ubuntu. La solución prevista desde la Fase 6 (sección B.3 de ese documento) es añadir un **segundo adaptador de red en modo NAT** (Adaptador 2) a cada VM. VirtualBox implementa NAT de forma interna para cada adaptador de ese tipo: la VM recibe una IP del DHCP embebido de VirtualBox (rango `10.0.x.x`) y sale a internet a través del host, exactamente igual que un dispositivo detrás de un router doméstico. El Adaptador 1 (Host-Only, `enp0s3`) permanece exactamente igual: misma dirección, mismo cable virtual. El Adaptador 2 nuevo solo añade una ruta de salida hacia internet y no interfiere con la red interna del laboratorio.

4. Coexistencia de dos adaptadores y tabla de rutas.

Cuando `enp0s3` tiene una IP estática sin gateway (`192.168.56.101/24`) y `enp0s8` obtiene IP y gateway por DHCP del motor NAT de VirtualBox, el kernel de Linux mantiene dos tipos de entradas en la tabla de rutas:

- Una ruta *directamente conectada*: `192.168.56.0/24 dev enp0s3` (todo el tráfico hacia la red interna del laboratorio sale por `enp0s3`).
 - Una ruta por *defecto*: `0.0.0.0/0 via 10.0.x.1 dev enp0s8` (todo lo demás — repositorios, actualizaciones, cualquier destino externo — sale por `enp0s8`).
- No hay conflicto: la ruta más específica siempre tiene preferencia en Linux. El tráfico entre nodos y hacia el host sigue por `enp0s3` ; las descargas de paquetes van por `enp0s8` .

5. Estructura de configuración de MariaDB en Ubuntu.

En Ubuntu, MariaDB organiza su configuración bajo `/etc/mysql/` :

- `mariadb.cnf` : archivo raíz, que incluye mediante directivas `!includedir` los subdirectorios `conf.d/` y `mariadb.conf.d/` .
- `mariadb.conf.d/50-server.cnf` : archivo principal del servidor; contiene `bind-address` , rutas de datos (`datadir`), configuración de logs, tamaños de buffer, etc. Es el archivo que instala el paquete `mariadb-server` y pertenece al sistema de paquetes.
- `mariadb.conf.d/99-*.cnf` (convención de este laboratorio): archivos adicionales que se cargan después de `50-server.cnf` . Los archivos se procesan en orden alfanumérico, de modo que un archivo `99-*` sobrescribe cualquier directiva que el `50-*` haya declarado. Este mecanismo permite añadir configuración propia sin tocar los archivos del paquete, lo que facilita las actualizaciones y el diagnóstico.

6. `bind-address` y por qué no se modifica todavía.

Por defecto, `50-server.cnf` incluye `bind-address = 127.0.0.1` , lo que significa que el demonio de MariaDB solo acepta conexiones TCP desde el propio nodo (loopback). Esto es correcto y seguro para esta fase: aún no hay ninguna razón para aceptar conexiones remotas. En la **Fase 10**, cuando se configure la replicación física maestro-esclavo, se cambiará `bind-address` a `0.0.0.0` (o a la IP Host-Only específica de cada nodo) para que el maestro pueda recibir conexiones del esclavo. Modificar `bind-address` antes de ese momento abriría el motor en la red interna sin que haya todavía usuarios ni privilegios remotos configurados.

7. Autenticación de `root` en MariaDB 10.11 en Ubuntu: `unix_socket` .

A diferencia de MySQL, donde el usuario `root` usa autenticación por contraseña desde la instalación, MariaDB en Ubuntu configura `root` con el plugin de autenticación `unix_socket` por defecto. Esto significa que el usuario Unix `root` (o cualquier usuario con `sudo`) puede acceder al motor con `sudo mariadb` sin necesidad de contraseña de MariaDB, porque el kernel del sistema operativo ya verificó la identidad del proceso. Este mecanismo es más seguro que una contraseña fija para acceso local, porque no existe ninguna contraseña que

pueda filtrarse. Sin embargo, para poder hacer pruebas de conexión con contraseña explícita (útil en fases de replicación y con herramientas como DBeaver), en esta fase también se establece una contraseña de respaldo para `root`.

8. `mysql_secure_installation`.

Este script interactivo, incluido con la instalación de MariaDB (en versiones más recientes puede llamarse `mariadb-secure-installation`), realiza cuatro acciones de seguridad sobre una instalación recién hecha: ofrece gestionar la contraseña del usuario `root` de MariaDB, elimina los usuarios anónimos creados por defecto, restringe el acceso remoto del usuario `root`, y elimina la base de datos `test`. Ninguna de estas acciones afecta a la replicación (que usará usuarios dedicados, no `root`). Son tareas de higiene de instalación que en un servidor real siempre se ejecutan antes de poner el motor en uso.

9. Juego de caracteres `utf8mb4` vs. `utf8`.

En MySQL/MariaDB, el juego de caracteres históricamente llamado `utf8` es una implementación incompleta de UTF-8 que soporta únicamente hasta 3 bytes por carácter, excluyendo los emojis y muchos caracteres del plano suplementario de Unicode. El juego `utf8mb4` es la implementación completa de 4 bytes conforme al estándar. Dado que en las fases de fragmentación y consultas distribuidas se trabajará con datos de texto heterogéneos, configurar `utf8mb4` desde el inicio evita tener que migrar esquemas y datos más adelante.

C. Procedimiento paso a paso

Paso 1 — Confirmar que ambas VMs están apagadas.

Añadir un adaptador de red a una VM requiere que esté detenida. Verificar el estado antes de continuar (sección D.1).

Paso 2 — Añadir el Adaptador 2 en modo NAT a `bdd-nodo01` y `bdd-nodo02` desde el host.

Un solo comando por VM; VirtualBox registra el cambio de inmediato (sección D.2).

Paso 3 — Iniciar `bdd-nodo01` en modo sin cabeza (*headless*) y conectarse por SSH.

A partir de esta fase, la forma de trabajo preferida es SSH desde el host; la consola gráfica de VirtualBox se reserva para situaciones sin red disponible (sección D.3).

Paso 4 — Identificar el nombre del nuevo adaptador dentro de la VM.

El adaptador NAT aparecerá en el sistema operativo como una interfaz adicional (típicamente `enp0s8` en VirtualBox con chipset PIIX3). Confirmar el nombre exacto antes de editar Netplan.

Paso 5 — Actualizar el archivo de Netplan para incluir `enp0s8` con DHCP.

Añadir la sección del nuevo adaptador al YAML existente, sin modificar la sección de `enp0s3` (sección D.4).

Paso 6 — Aplicar la configuración y verificar conectividad a internet.

`netplan apply`, verificar la IP de NAT y probar el acceso real a los repositorios de Ubuntu (sección D.5).

Paso 7 — Actualizar el índice de paquetes e instalar `mariadb-server`.

`apt update` seguido de `apt install`; el paquete habilita e inicia el servicio automáticamente al terminar (sección D.6).

Paso 8 — Ejecutar `mysql_secure_installation`.

Proceso interactivo; la tabla de la sección D.7 detalla cada pregunta y la respuesta recomendada con su justificación.

Paso 9 — Crear el archivo de configuración de juego de caracteres.

Nuevo archivo `99-lab-charset.cnf` en `mariadb.conf.d/`; no se edita `50-server.cnf` (sección D.8).

Paso 10 — Reiniciar el servicio para que tome los nuevos parámetros.

`systemctl restart mariadb` y verificación de estado (sección D.9).

Paso 11 — Verificar la instalación completa dentro del motor.

Conectar con `sudo mariadb`, confirmar versión, juego de caracteres y bases de datos presentes (sección D.10).

Paso 12 — Repetir los Pasos 3 a 11 para `bdd-nodo02`.

Exactamente el mismo procedimiento. El único valor que cambia en Netplan es la IP de `enp0s3` (`.102`), que ya estaba configurada desde la Fase 6 y no se toca; la sección de `enp0s8` es idéntica. Todo lo demás de MariaDB (juego de caracteres, respuestas de `mysql_secure_installation`) es igual en ambos nodos.

Paso 13 — Apagar ambas VMs de forma ordenada y tomar el snapshot `fase07-completa`.

Sección D.11.

Paso 14 — Validar con el checklist de la sección G.

D. Comandos completos

Los comandos marcados **(host)** se ejecutan en PowerShell en Windows. Los marcados **(VM)** se ejecutan en la sesión SSH de Ubuntu Server abierta desde el host, o en la consola gráfica de VirtualBox si SSH no estuviera disponible por algún motivo.

D.1 Verificar que ambas VMs están apagadas (host)

 PowerShell

```
VBoxManage list runningvms
```

La salida debe estar vacía. Si alguna VM aparece activa, apagarla antes de continuar:

 PowerShell

```
VBoxManage controlvm "bdd-nodo01" acpipowerbutton  
VBoxManage controlvm "bdd-nodo02" acpipowerbutton
```

Esperar unos segundos y volver a ejecutar `list runningvms` para confirmar que se detuvieron.

D.2 Añadir el Adaptador 2 en modo NAT a ambas VMs (host)

 PowerShell

```
VBoxManage modifyvm "bdd-nodo01" --nic2 nat  
VBoxManage modifyvm "bdd-nodo02" --nic2 nat
```

Verificar que la configuración quedó registrada:

 PowerShell

```
VBoxManage showvminfo "bdd-nodo01" | findstr "NIC 2"  
VBoxManage showvminfo "bdd-nodo02" | findstr "NIC 2"
```

La salida debe mostrar `NIC 2:` con `Attachment: NAT` para cada VM.

D.3 Iniciar `bdd-nodo01` y conectarse por SSH (host)

 PowerShell

```
VBoxManage startvm "bdd-nodo01" --type headless
```

Esperar entre 15 y 30 segundos a que Ubuntu Server complete el arranque y, a continuación:

 PowerShell

```
ssh bddadmin@192.168.56.101
```

`--type headless` inicia la VM sin abrir la ventana gráfica de VirtualBox, dado que toda la administración es desde ahora por SSH. Si se necesita ver la consola gráfica (por ejemplo, para depurar un problema de arranque), usar `--type gui` en su lugar.

D.4 Actualizar Netplan para incluir `enp0s8` con DHCP (VM — `bdd-nodo01`)

Primero, confirmar que el nuevo adaptador ya es visible en el sistema operativo y verificar su nombre exacto:

```
Bash
```

```
ip a
```

Debe aparecer una interfaz adicional junto a `lo` y `enp0s3` ; en VirtualBox con chipset PIIX3 suele llamarse `enp0s8` . Si el nombre reportado es diferente (por ejemplo, `ens8` o `eth1`), usar ese nombre exacto en el paso siguiente.

Editar el archivo de Netplan:

```
Bash
```

```
sudo nano /etc/netplan/50-cloud-init.yaml
```

Contenido completo del archivo tras la edición (para `bdd-nodo01`):

```
YAML
```

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 192.168.56.101/24
    enp0s8:
      dhcp4: yes
```

Para `bdd-nodo02` , la única diferencia es la IP de `enp0s3` ; la sección de `enp0s8` es idéntica:

YAML

```
network:
  version: 2
  ethernet:
    enp0s3:
      dhcp4: no
      addresses:
        - 192.168.56.102/24
    enp0s8:
      dhcp4: yes
```

Guardar con `Ctrl+O`, `Enter` y salir con `Ctrl+X`. Corregir los permisos del archivo:

Bash

```
sudo chmod 600 /etc/netplan/50-cloud-init.yaml
```

D.5 Aplicar la configuración de red y verificar conectividad a internet (VM)

Bash

```
sudo netplan apply
```

Confirmar que `enp0s8` obtuvo una IP del DHCP interno de VirtualBox (rango típico `10.0.2.x`):

Bash

```
ip a show enp0s8
```

Verificar que la tabla de rutas tiene tanto la ruta interna como la ruta por defecto:

Bash

```
ip route
```

Debe mostrar, al menos, estas dos líneas (los valores exactos de la ruta NAT variarán):

Text

```
default via 10.0.2.2 dev enp0s8 proto dhcp ...  
192.168.56.0/24 dev enp0s3 proto kernel scope link src 192.168.56.101
```

Confirmar acceso real a los repositorios de Ubuntu:

Bash

```
ping -c 3 archive.ubuntu.com
```

Si responde sin pérdida de paquetes, la conectividad está lista. Si falla la resolución DNS, revisar la sección F.

D.6 Actualizar el índice de paquetes e instalar MariaDB (VM)

Bash

```
sudo apt update  
sudo apt install -y mariadb-server
```

La descarga e instalación trae varios paquetes (~30–60 MB según el estado de la caché) y puede tardar algunos minutos. Al finalizar, `apt` habrá habilitado e iniciado el servicio `mariadb` automáticamente.

Verificar que el servicio quedó activo inmediatamente después:

Bash

```
sudo systemctl status mariadb
```

Debe mostrar `Active: active (running)`.

D.7 Ejecutar `mysql_secure_installation` (VM)

Bash

```
sudo mysql_secure_installation
```

En algunas versiones de MariaDB o distribuciones el comando puede llamarse `mariadb-secure-installation`. Si `mysql_secure_installation` no se encuentra, intentar con ese nombre alternativo.

El script es interactivo. La siguiente tabla detalla cada pregunta, la respuesta recomendada y el motivo para este laboratorio:

Pregunta del script	Respuesta recomendada	Justificación
<code>Enter current password for root (enter for none):</code>	Enter (vacío)	La instalación fresca usa <code>unix_socket</code> contraseña de MariaDB establecida to
<code>Switch to unix_socket authentication [Y/n]</code>	n	<code>unix_socket</code> ya está activo por defecto; responder <code>n</code> mantiene el estado actual sin cambios
<code>Change the root password? [Y/n]</code>	Y → ingresar y confirmar una contraseña fuerte	Establece una contraseña de respaldo de <code>root</code> ; documentarla en <code>C:\LabBDD\Snapshots-Notas\</code>
<code>Remove anonymous users? [Y/n]</code>	Y	Elimina los usuarios sin nombre que permiten conexión sin credenciales
<code>Disallow root login remotely? [Y/n]</code>	Y	El usuario <code>root</code> de MariaDB no debe aceptar conexiones desde otros nodos; en la Fase 7 se crearán usuarios de replicación dedicados
<code>Remove test database and access to it? [Y/n]</code>	Y	Elimina la base de datos <code>test</code> de ejemplo innecesaria en un laboratorio serio
<code>Reload privilege tables now? [Y/n]</code>	Y	Aplica todos los cambios anteriores de configuración inmediato, sin necesidad de reiniciar el servicio

La contraseña de `root` de MariaDB es independiente de la contraseña del usuario Unix `bddadmin`. Es la contraseña del superusuario administrativo del **motor de base de datos**, no del sistema operativo. Usar la misma contraseña en ambos nodos simplifica la administración del laboratorio; documentarla en la bitácora de la Fase 7 en `C:\LabBDD\Snapshots-Notas\fase07-notas.txt`.

D.8 Crear el archivo de configuración de juego de caracteres (VM)

 Bash

```
sudo nano /etc/mysql/mariadb.conf.d/99-lab-charset.cnf
```

Contenido completo (idéntico en `bdd-nodo01` y `bdd-nodo02`):

Text

```
# Configuración de juego de caracteres para el Laboratorio BDD
# Creado en la Fase 7. No modificar 50-server.cnf directamente.
# Este archivo se carga después de 50-server.cnf (orden alfanumérico)
# y sobrescribe las directivas de charset que declare.
[server]
character-set-server = utf8mb4
collation-server      = utf8mb4_unicode_ci
[client]
default-character-set = utf8mb4
```

Guardar con `Ctrl+O`, `Enter` y salir con `Ctrl+X`.

D.9 Reiniciar MariaDB para aplicar el nuevo juego de caracteres (VM)

Bash

```
sudo systemctl restart mariadb
sudo systemctl status mariadb
```

El servicio debe volver a estado `active (running)` en unos segundos.

D.10 Verificación completa dentro del motor (VM)

Conectar al motor como `root` local usando `unix_socket` (no se solicita contraseña en el prompt del sistema operativo cuando se precede con `sudo`):

Bash

```
sudo mariadb
```

Dentro del prompt `MariaDB [(none)]>`, ejecutar los siguientes comandos de verificación:

SQL

```
-- Versión instalada
SELECT VERSION();
-- Juego de caracteres del servidor
SHOW VARIABLES LIKE 'character_set_server';
-- Intercalación del servidor
SHOW VARIABLES LIKE 'collation_server';
-- Bases de datos presentes (solo las del sistema; no debe existir 'test')
SHOW DATABASES;
-- Confirmar bind-address (debe ser 127.0.0.1)
SHOW VARIABLES LIKE 'bind_address';
EXIT;
```

Resultados esperados (los valores de versión exactos variarán según la actualización disponible en los repositorios):

Text

```
+-----+
| VERSION()                                |
+-----+
| 10.11.x-MariaDB-0ubuntu0.24.04.x        |
+-----+
+-----+-----+
| Variable_name      | Value      |
+-----+-----+
| character_set_server | utf8mb4    |
+-----+-----+
+-----+-----+
| Variable_name      | Value      |
+-----+-----+
| collation_server   | utf8mb4_unicode_ci |
+-----+-----+
+-----+
| Database           |
+-----+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+-----+
+-----+-----+
| Variable_name| Value      |
+-----+-----+
| bind_address  | 127.0.0.1  |
+-----+-----+
```

Verificar también que MariaDB está habilitado para arrancar automáticamente con el sistema:

Bash

```
sudo systemctl is-enabled mariadb
```

Debe responder **enabled**.

D.11 Apagar ambas VMs y tomar el snapshot **fase07-completa** (host)

Desde la sesión SSH en cada VM (o la consola gráfica):

 Bash

```
sudo poweroff
```

Confirmar que ambas VMs están detenidas desde el host:

 PowerShell

```
VBoxManage list runningvms
```

La salida debe estar vacía. Tomar los snapshots:

 PowerShell

```
VBoxManage snapshot "bdd-nodo01" take "fase07-completa" `
--description "MariaDB 10.11 instalado; utf8mb4 configurado;
mysql_secure_installation ejecutado; Adaptador NAT activo en enp0s8"
VBoxManage snapshot "bdd-nodo02" take "fase07-completa" `
--description "MariaDB 10.11 instalado; utf8mb4 configurado;
mysql_secure_installation ejecutado; Adaptador NAT activo en enp0s8"
```

Confirmar la lista de snapshots de ambas VMs:

 PowerShell

```
VBoxManage snapshot "bdd-nodo01" list
VBoxManage snapshot "bdd-nodo02" list
```

Cada VM debe mostrar dos snapshots: `fase05-completa` , `fase06-completa` y `fase07-completa` .

E. Verificación de funcionamiento

Esta fase se considera completa cuando se cumplen todos los puntos siguientes en **ambos nodos**:

1. `VBoxManage showvminfo "bdd-nodo01" | findstr NIC` muestra el Adaptador 1 como `hostonly` y el Adaptador 2 como `nat` ; lo mismo para `bdd-nodo02` .

2. `ip a` dentro de cada VM muestra `enp0s3` con su IP estática de la Fase 6 (`192.168.56.101 / .102`) y `enp0s8` con una IP del rango NAT de VirtualBox (`10.0.2.x`), ambas en estado `UP`.
 3. `ip route` dentro de cada VM muestra una ruta `default` apuntando a `enp0s8` y la ruta directa `192.168.56.0/24` por `enp0s3`.
 4. `ping -c 3 archive.ubuntu.com` desde cada VM responde sin pérdida de paquetes.
 5. La conectividad Host-Only de la Fase 6 **sigue intacta**: `ping 192.168.56.101` y `ping 192.168.56.102` desde PowerShell en el host responden correctamente; el adaptador NAT no la interrumpió.
 6. `sudo systemctl status mariadb` muestra `Active: active (running)` en ambos nodos.
 7. `sudo systemctl is-enabled mariadb` responde `enabled` en ambos nodos.
 8. `SELECT VERSION()` dentro de `sudo mariadb` muestra una versión `10.11.x-MariaDB`.
 9. `SHOW VARIABLES LIKE 'character_set_server'` devuelve `utf8mb4`.
 10. `SHOW VARIABLES LIKE 'collation_server'` devuelve `utf8mb4_unicode_ci`.
 11. `SHOW VARIABLES LIKE 'bind_address'` devuelve `127.0.0.1` (no se ha modificado).
 12. `SHOW DATABASES` muestra las cuatro bases de datos del sistema y **no existe** la base de datos `test`.
 13. Ambos snapshots `fase07-completa` existen y están asociados al estado apagado de cada VM.
 14. La contraseña de `root` de MariaDB está documentada en `C:\LabBDD\Snapshots-Notas\fase07-notas.txt`.
-

F. Problemas comunes y soluciones

Problema	Causa probable	Solución
<code>VBoxManage</code> <code>modifyvm ... -</code> <code>-nic2 nat</code> falla con "VM is currently running"	La VM estaba encendida al intentar añadir el adaptador	Apagar la VM con <code>VBoxManage</code> <code>controlvm</code> <code>"bdd-nodo01"</code> <code>acpipowerbutton</code> (o <code>sudo poweroff</code> desde dentro), esperar a que se detenga completamente y repetir el comando
<code>enp0s8</code> no aparece en <code>ip a</code> tras arrancar la VM	VirtualBox asignó un nombre de interfaz distinto al esperado, o la VM arrancó antes de que VirtualBox registrara el cambio de adaptador	Ejecutar <code>ip a</code> completo para identificar la interfaz nueva (es la que no es <code>lo</code> ni <code>enp0s3</code>); usar ese nombre exacto en el YAML de Netplan
<code>enp0s8</code> aparece en <code>ip a</code> sin dirección IP tras <code>netplan apply</code>	El nombre de la interfaz en el archivo YAML no coincide exactamente con el del kernel	Corregir el nombre en <code>/etc/netplan/50-cloud-init.yaml</code> para que sea idéntico al que muestra <code>ip a</code> (sensible a mayúsculas y números) y volver a ejecutar <code>sudo netplan apply</code>
<code>ping archive.ubuntu.com</code> falla con "Name or service not known"	La interfaz NAT no obtuvo configuración DNS del DHCP de VirtualBox	Verificar con <code>resolvectl status</code> si existe un servidor DNS asignado a

		<code>enp0s8</code> ; si no, forzar la renovación DHCP con <code>sudo</code> <code>dhclient</code> <code>enp0s8</code> y reintentar
<code>ping</code> <code>archive.ubuntu</code> <code>.com</code> falla con “Network is unreachable”	La ruta por defecto no se estableció porque <code>enp0s8</code> no recibió IP del DHCP	Ejecutar <code>ip</code> <code>route</code> para confirmar si existe la ruta <code>default</code> ; si no existe, ejecutar <code>sudo dhclient</code> <code>enp0s8</code> y verificar de nuevo
<code>sudo apt</code> <code>update</code> falla con errores de certificado SSL o de conexión	El reloj del sistema de la VM está desincronizado (diferencia de más de algunos minutos invalida certificados TLS)	Dentro de la VM: <code>sudo</code> <code>timedatectl</code> <code>set-ntp true</code> y esperar unos segundos; luego reintentar <code>apt</code> <code>update</code>
<code>apt install</code> <code>mariadb-</code> <code>server</code> falla con “E: Unable to locate package”	El índice de paquetes no se actualizó antes de instalar, o la URL de los repositorios en <code>sources.list</code> no es correcta para Ubuntu 24.04	Ejecutar primero <code>sudo apt</code> <code>update</code> ; si persiste, verificar con <code>cat</code> <code>/etc/apt/sourc</code> <code>es.list</code> que los repositorios apunten a <code>noble</code> (nombre en clave de Ubuntu 24.04)
Tras la instalación, <code>systemctl</code> <code>status</code> <code>mariadb</code> muestra <code>failed</code> o <code>inactive</code>	Conflicto de puertos (el puerto 3306 estaba en uso por otro proceso) o problema de	Revisar el log de MariaDB con <code>sudo</code> <code>journalctl -u</code> <code>mariadb --no-</code> <code>pager -n 50</code> para identificar el error exacto;

	permisos en el directorio de datos	lo más habitual es un problema de permisos que se resuelve con <code>sudo</code> <code>mysql_install_db --</code> <code>user=mysql</code>
<code>mysql_secure_installation</code> devuelve "Can't connect to local MySQL server through socket"	El servicio MariaDB no está activo	Ejecutar <code>sudo systemctl start mariadb</code> antes de lanzar el script
<code>SHOW VARIABLES LIKE 'character_set_server'</code> devuelve <code>latin1</code> o <code>utf8</code> en lugar de <code>utf8mb4</code>	El archivo <code>99-lab-charset.cnf</code> tiene un error de sintaxis, está en la ruta incorrecta, o el servicio no se reinició tras crearlo	Verificar la ruta exacta (<code>/etc/mysql/mariadb.conf.d/99-lab-charset.cnf</code>), revisar la sintaxis (sección <code>[server]</code> presente, sin espacios extra alrededor del <code>=</code>), y ejecutar <code>sudo systemctl restart mariadb</code>
Al reiniciar la VM, la interfaz <code>enp0s8</code> pierde la IP y no hay internet	El DHCP de NAT de VirtualBox a veces tarda en responder en el arranque	Esto no afecta a las funciones del laboratorio (los snapshots y las fases posteriores solo requieren la red Host-Only); si se necesita internet en un momento concreto, ejecutar <code>sudo dhclient enp0s8</code> dentro de la VM

La conectividad Host-Only (<code>enp0s3</code>) dejó de funcionar tras añadir <code>enp0s8</code>	Conflicto de rutas: la ruta por defecto de NAT puede interferir con el tráfico Host-Only en casos raros	Verificar con <code>ip route</code> que la ruta <code>192.168.56.0/24 dev enp0s3</code> sigue presente; si no aparece, reactivar la interfaz con <code>sudo ip link set enp0s3 up</code> y <code>sudo netplan apply</code>
---	---	--

G. Checklist de validación

- ☐ Ambas VMs estaban apagadas antes de añadir el Adaptador 2.
- ☐ Añadí el Adaptador 2 en modo NAT a `bdd-nodo01` con `VBoxManage modifyvm "bdd-nodo01" --nic2 nat`.
- ☐ Añadí el Adaptador 2 en modo NAT a `bdd-nodo02` con `VBoxManage modifyvm "bdd-nodo02" --nic2 nat`.
- ☐ Actualicé `/etc/netplan/50-cloud-init.yaml` en `bdd-nodo01` añadiendo `enp0s8: dhcp4: yes`, sin modificar la sección de `enp0s3`.
- ☐ `netplan apply` se ejecutó sin errores en `bdd-nodo01` y `enp0s8` obtuvo una IP del rango NAT de VirtualBox.
- ☐ `ping archive.ubuntu.com` respondió desde `bdd-nodo01`, confirmando acceso a internet.
- ☐ `sudo apt update && sudo apt install -y mariadb-server` completó sin errores en `bdd-nodo01`.
- ☐ Ejecuté `mysql_secure_installation` en `bdd-nodo01` respondiendo correctamente cada pregunta, y documenté la contraseña de `root` de MariaDB en la bitácora del laboratorio.
- ☐ Creé `/etc/mysql/mariadb.conf.d/99-lab-charset.cnf` con las directivas `utf8mb4` en `bdd-nodo01`.
- ☐ Reinicié el servicio `mariadb` en `bdd-nodo01` y verifiqué que `character_set_server = utf8mb4` está activo dentro del motor.
- ☐ `SHOW DATABASES` en `bdd-nodo01` muestra las cuatro bases de datos del sistema y **no** existe la base de datos `test`.
- ☐ `SHOW VARIABLES LIKE 'bind_address'` en `bdd-nodo01` devuelve `127.0.0.1` (sin modificar).
- ☐ `sudo systemctl is-enabled mariadb` en `bdd-nodo01` devuelve `enabled`.
- ☐ Repetí y verifiqué todos los puntos anteriores en `bdd-nodo02`.

- ☐ La conectividad Host-Only sigue funcionando: `ping 192.168.56.101` y `ping 192.168.56.102` responden desde el host Windows.
 - ☐ Apagué ambas VMs de forma ordenada y tomé el snapshot `fase07-completa` en cada una.
 - ☐ Puedo explicar qué es `bind-address`, cuál es su valor actual y por qué no se modificó en esta fase.
 - ☐ Puedo explicar la diferencia entre la contraseña del usuario Unix `bddadmin` y la contraseña del usuario `root` de MariaDB.
-

Preguntas teóricas para estudiantes

1. ¿Por qué se eligió MariaDB y no MySQL o PostgreSQL para este laboratorio? Menciona al menos dos características específicas de MariaDB que serán relevantes en fases posteriores del laboratorio.
 2. ¿Qué ocurre en la tabla de rutas del kernel de Linux cuando una misma VM tiene dos adaptadores de red activos simultáneamente, uno con IP estática y sin gateway, y otro con DHCP que sí proporciona gateway? ¿Qué tipo de tráfico seguiría cada ruta?
 3. Explica el mecanismo de `unix_socket` authentication en MariaDB. ¿Por qué es considerado más seguro que la autenticación por contraseña para el acceso local de `root`? ¿Cuándo *no* sería suficiente esta autenticación?
 4. ¿Qué diferencia existe entre el juego de caracteres `utf8` y `utf8mb4` en MariaDB, y por qué esta diferencia importa para bases de datos que almacenarán texto de usuarios finales?
 5. ¿Por qué en esta fase se crea un archivo `99-lab-charset.cnf` en lugar de editar directamente `50-server.cnf`? ¿Qué problema podría causar editar `50-server.cnf` directamente si en el futuro se actualiza el paquete `mariadb-server`?
-

Preparación para la siguiente fase

La **Fase 8** requerirá:

- MariaDB 10.11 instalado, configurado con `utf8mb4` y accesible en ambos nodos (esta fase).
- Snapshot `fase07-completa` tomado en `bdd-nodo01` y `bdd-nodo02`.
- Acceso SSH funcional desde el host hacia ambos nodos (Fase 6).
- `bind-address = 127.0.0.1` en su valor por defecto (no se ha modificado todavía).

A partir de la Fase 8, toda la interacción con los nodos se realizará exclusivamente por SSH desde el host; no será necesario abrir la ventana gráfica de VirtualBox para las tareas normales del laboratorio.

Fase 8 — Administración Básica de MariaDB

MGTI. Baltazar Martinez Galla

Continuación directa de la Fase 7. Ambos nodos tienen MariaDB 10.11 instalado, configurado con `utf8mb4`, con `mysql_secure_installation` ejecutado y snapshot `fase07-completa` tomado. En esta fase se realiza la administración inicial del motor: creación del **esquema de prueba del laboratorio** (que se usará en todas las fases de distribución posteriores), gestión de usuarios y privilegios, habilitación del slow query log, respaldo básico con `mysqldump` y conexión remota segura desde el host Windows mediante **DBeaver con túnel SSH**. El acceso a internet (adaptador NAT de la Fase 7) sigue activo en ambos nodos para eventuales actualizaciones. Todo el trabajo se realizará exclusivamente por SSH desde el host, sin abrir la ventana gráfica de VirtualBox.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Navegar las bases de datos del sistema de MariaDB (`information_schema` , `performance_schema` , `mysql` , `sys`) y extraer información administrativa mediante consultas SQL.
2. Diseñar y crear un esquema relacional pensado desde el inicio para su futura distribución (fragmentación horizontal, vertical e híbrida) en fases posteriores.
3. Comprender los cuatro niveles del sistema de privilegios de MariaDB (global, base de datos, tabla y columna) y aplicarlos correctamente con `GRANT` y `REVOKE` .
4. Diferenciar los plugins de autenticación `unix_socket` y `mysql_native_password` , y elegir el adecuado para cada tipo de usuario y contexto.
5. Habilitar y consultar el *slow query log*, interpretando sus entradas para identificar consultas candidatas a optimización.
6. Ejecutar respaldos lógicos con `mysqldump` y comprender sus parámetros esenciales, distinguiendo entre un respaldo de esquema y un respaldo de datos completo.
7. Conectar una herramienta gráfica (DBeaver) al motor de MariaDB de forma segura, usando un **túnel SSH** en lugar de exponer el puerto 3306 directamente a la red.
8. Ejecutar los comandos de monitoreo básico del motor (`SHOW STATUS` , `SHOW PROCESSLIST` , `SHOW ENGINE INNODB STATUS`) e interpretar sus salidas más relevantes.
9. Cerrar la fase con el snapshot `fase08-completa` en ambos nodos, en el estado de mayor preparación alcanzado hasta ahora para las fases de distribución.

B. Conceptos teóricos necesarios

1. Sistema de privilegios de MariaDB: cuatro niveles.

MariaDB controla el acceso mediante una jerarquía de cuatro niveles de granularidad, evaluados de mayor a menor especificidad:

- **Global** (`GRANT ... ON *.*`): el privilegio aplica a cualquier base de datos y cualquier tabla del servidor. Se almacena en la tabla `mysql.user`.
- **Base de datos** (`GRANT ... ON basedatos.*`): aplica a todas las tablas de una base de datos específica. Se almacena en `mysql.db`.
- **Tabla** (`GRANT ... ON basedatos.tabla`): aplica solo a una tabla. Se almacena en `mysql.tables_priv`.
- **Columna** (`GRANT SELECT (col1, col2) ON ...`): aplica a columnas específicas. Se almacena en `mysql.columns_priv`.

Para cada privilegio, MariaDB también evalúa la dirección IP o nombre de host desde el que conecta el usuario (parte `@'host'` en la definición). Un usuario se identifica siempre por el par `usuario@host`, nunca solo por el nombre de usuario.

2. Plugins de autenticación: `unix_socket` y `mysql_native_password`.

MariaDB soporta múltiples *plugins* para verificar la identidad de quien conecta:

- `unix_socket`: el motor pregunta al kernel del sistema operativo si el proceso que intenta conectar pertenece al usuario Unix cuyo nombre coincide con el usuario de MariaDB. Solo funciona desde el mismo servidor (socket local). No hay contraseña que transmitir ni almacenar; la identidad la garantiza el SO. Es el método por defecto para el usuario `root` en Ubuntu.
- `mysql_native_password`: esquema clásico de contraseña con hash SHA1 doble almacenado en `mysql.user`. Funciona tanto desde localhost como desde conexiones TCP remotas. Es el método que utilizará la herramienta DBeaver, los scripts de replicación (Fase 10) y cualquier aplicación que conecte desde otro nodo.

Ambos pueden coexistir en el mismo servidor para distintos usuarios.

3. Las cuatro bases de datos del sistema.

- `information_schema`: base de datos virtual (sin archivos reales) que expone metadatos del servidor — tablas, columnas, índices, vistas, privilegios, etc. Siempre está disponible en modo de solo lectura para cualquier usuario conectado.

- `mysql` : base de datos física que almacena los datos del propio motor — tablas de privilegios (`user` , `db` , `tables_priv`), rutinas almacenadas del sistema, historial de eventos. Solo debe modificarse con instrucciones `GRANT` / `REVOKE` y `CREATE USER` , nunca con `INSERT` / `UPDATE` directos.
- `performance_schema` : base de datos de instrumentación (también virtual) que registra métricas internas del motor — contadores de espera, *memory usage*, ejecución de sentencias. Se leerá en fases de diagnóstico.
- `sys` : conjunto de vistas y procedimientos contruidos sobre `performance_schema` que ofrecen resúmenes legibles para humanos. Disponible desde MariaDB 10.5+.

4. Tipos de logs de MariaDB relevantes para este laboratorio.

- **Error log** (`/var/log/mysql/error.log`): errores de arranque, advertencias del motor y eventos críticos. Siempre activo; es el primer lugar donde buscar problemas.
- **Slow query log**: registra consultas que superan un umbral de tiempo configurable (`long_query_time`). Esencial para detectar consultas mal optimizadas antes de introducir la distribución en fases posteriores. Se habilita en esta fase.
- **General query log**: registra *todas* las consultas recibidas. Tiene un impacto significativo en el rendimiento y en el tamaño del log; solo debe activarse puntualmente para depuración, nunca de forma permanente. **No se habilita en esta fase.**
- **Binary log** (`binlog`): registro de todos los cambios de datos en un formato especial, usado por la replicación. Es la piedra angular de las Fases 10 y 11. Se configurará en detalle entonces; en esta fase solo se introduce el concepto.

5. `mysqldump` : respaldo lógico.

`mysqldump` genera un archivo de texto que contiene las sentencias SQL necesarias para recrear el esquema y/o los datos de una o varias bases de datos. Es el método de respaldo más simple y portable; su principal limitación es que la restauración es lenta en bases de datos grandes (hay que ejecutar miles de `INSERT`) y no es consistente con el estado exacto del binlog (para eso se usa `mariabackup` , que se introduce como concepto en esta fase pero cuyo uso detallado corresponde a la Fase 18 de recuperación). Para los volúmenes de este laboratorio, `mysqldump` es completamente adecuado.

6. Túnel SSH para conexión remota segura.

El parámetro `bind-address = 127.0.0.1` (configurado en la instalación de la Fase 7 y no modificado hasta la Fase 10) significa que MariaDB solo acepta conexiones TCP desde el propio servidor. Una herramienta gráfica en el host Windows no puede conectar directamente al puerto 3306 de una VM. La solución es un **túnel SSH**:

Text

```
DBeaver (Windows) → SSH → bdd-nodo01:22 → reenvío local → 127.0.0.1:3306
```

El cliente SSH abre una conexión cifrada al puerto 22 de `bdd-nodo01` y “reenvía” localmente el tráfico del puerto 3306: desde el punto de vista de MariaDB, la conexión parece provenir de `127.0.0.1` (loopback del propio servidor). Esto mantiene `bind-address` intacto y añade una capa de cifrado al canal de administración, sin abrir ningún puerto adicional en la VM.

7. Diseño del esquema `lab_bdd` : pensado para la distribución.

El esquema que se crea en esta fase no es arbitrario. Cada decisión de diseño anticipa los experimentos de las fases 13 a 16:

- La tabla `clientes` tiene una columna `region ENUM('norte','sur','este','oeste')` : el predicado natural para la **fragmentación horizontal** de la Fase 13.
- La tabla `pedidos` replica ese campo `region` (redundancia controlada) para poder fragmentarse independientemente de `clientes` sin necesidad de joins entre nodos.
- La tabla `productos` mezcla columnas de consulta frecuente (precio, stock) con columnas de detalle pesado (descripcion, ficha_tecnica, imagen_url): la separación natural para la **fragmentación vertical** de la Fase 14.
- Las relaciones entre tablas (claves foráneas) se conservan para trabajar con **consultas distribuidas** en la Fase 16.

C. Procedimiento paso a paso

Paso 1 — Iniciar ambas VMs en modo headless y conectarse por SSH.

Arrancar `bdd-nodo01` y `bdd-nodo02` . La mayor parte del trabajo de esta fase se realiza en `bdd-nodo01` ; los pasos que corresponden a `bdd-nodo02` se indican explícitamente.

Paso 2 — Explorar las bases de datos del sistema.

Antes de crear nada nuevo, familiarizarse con las bases de datos ya existentes ejecutando las consultas de exploración de la sección D.1 dentro de `sudo mariadb` .

Paso 3 — Crear el esquema del laboratorio `lab_bdd` .

Ejecutar el script de creación de tablas de la sección D.2. Este esquema es el mismo en ambos nodos por ahora; en la Fase 9 se diseñará cómo distribuirlo.

Paso 4 — Insertar los datos de prueba.

Ejecutar el script de la sección D.3 para poblar `clientes`, `productos`, `pedidos` y `detalle_pedidos` con un conjunto inicial de datos representativos.

Paso 5 — Crear y verificar los usuarios del laboratorio.

Crear dos usuarios con `mysql_native_password`: `lab_admin@'localhost'` con privilegios amplios sobre `lab_bdd`, y `app_user@'localhost'` con privilegios de solo lectura/escritura de datos (no de estructura). Ver sección D.4.

Paso 6 — Verificar el esquema de privilegios aplicado.

Usar `SHOW GRANTS` e `information_schema` para confirmar que los privilegios quedaron exactamente como se planeó (sección D.5).

Paso 7 — Habilitar el slow query log.

Crear el archivo de configuración `99-lab-logs.cnf` en `mariadb.conf.d/` y reiniciar el servicio para activarlo (sección D.6). Generar una consulta lenta artificial para verificar que se registra.

Paso 8 — Ejecutar los comandos de monitoreo del motor.

Practicar los comandos de diagnóstico instantáneo de MariaDB (`SHOW STATUS`, `SHOW PROCESSLIST`, `SHOW ENGINE INNODB STATUS`) e interpretar las métricas más importantes para el laboratorio (sección D.7).

Paso 9 — Realizar un respaldo básico con `mysqldump`.

Generar respaldos del esquema (solo DDL) y de esquema más datos de `lab_bdd`, guardados en la carpeta de trabajo del laboratorio dentro del nodo (sección D.8).

Paso 10 — Instalar DBeaver en el host Windows y configurar la conexión por túnel SSH.

DBeaver Community Edition es gratuita y no requiere instalación de JDK separado en versiones recientes. La conexión a cada nodo se configura usando las credenciales SSH de `bddadmin` y el usuario `lab_admin` de MariaDB (sección D.9).

Paso 11 — Repetir los Pasos 3 a 9 en `bdd-nodo02`.

El esquema `lab_bdd`, los datos de prueba, los usuarios y la configuración de logs deben ser idénticos en ambos nodos. Esto garantiza que las fases de replicación puedan iniciarse desde un estado coherente.

Paso 12 — Apagar ambas VMs y tomar el snapshot de cierre de fase.

Ver sección D.10.

Paso 13 — Validar con el checklist de la sección G.

D. Comandos completos

Los comandos marcados **(VM)** se ejecutan dentro de una sesión SSH en la VM indicada. Los marcados **(host)** se ejecutan en PowerShell en Windows. Los bloques `MariaDB>` se ejecutan dentro del prompt del motor, accedido con `sudo mariadb`.

D.1 Iniciar las VMs y conectarse por SSH (host)

 PowerShell

```
VBoxManage startvm "bdd-nodo01" --type headless  
VBoxManage startvm "bdd-nodo02" --type headless
```

Esperar 20–30 segundos y conectarse:

 PowerShell

```
ssh bddadmin@192.168.56.101
```

D.2 Explorar las bases de datos del sistema (VM — bdd-nodo01)

 Bash

```
sudo mariadb
```

Dentro del prompt `MariaDB [(none)]>`:

SQL

```
-- Listar todas las bases de datos del sistema
SHOW DATABASES;
-- ¿Qué tablas existen en la base de datos del sistema 'mysql'?
SHOW TABLES IN mysql;
-- Tabla de usuarios: quién tiene acceso y con qué plugin de autenticación
SELECT User, Host, plugin, authentication_string IS NOT NULL
AS tiene_password
FROM mysql.user;
-- Ver todos los privilegios globales actuales
SELECT User, Host, Select_priv, Insert_priv, Super_priv, Repl_slave_priv
FROM mysql.user;
-- Cuántas tablas existen en cada base de datos del sistema
SELECT TABLE_SCHEMA AS base_datos,
       COUNT(*)      AS num_tablas
FROM information_schema.TABLES
GROUP BY TABLE_SCHEMA
ORDER BY num_tablas DESC;
-- Variables de configuración activas más relevantes
SHOW VARIABLES LIKE 'character%';
SHOW VARIABLES LIKE 'collation%';
SHOW VARIABLES LIKE 'bind_address';
SHOW VARIABLES LIKE 'datadir';
SHOW VARIABLES LIKE 'log_error';
EXIT;
```

D.3 Crear el esquema del laboratorio `lab_bdd` (VM — bdd-nodo01)

Crear el archivo del script SQL para mantener el DDL documentado:

```
cat > /tmp/crear_schema_lab_bdd.sql << 'EOF'
-- =====
-- Esquema del Laboratorio de Bases de Datos Distribuidas
-- Creado en la Fase 8. Diseñado para fragmentación futura.
-- =====

CREATE DATABASE IF NOT EXISTS lab_bdd
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
USE lab_bdd;

-- -----
-- CLIENTES: campo 'region' -> fragmentación horizontal (F13)
-- -----

CREATE TABLE IF NOT EXISTS clientes (
    id            INT            AUTO_INCREMENT PRIMARY KEY,
    nombre        VARCHAR(100) NOT NULL,
    apellido      VARCHAR(100) NOT NULL,
    email         VARCHAR(150) UNIQUE,
    telefono      VARCHAR(20),
    region        ENUM('norte','sur','este','oeste') NOT NULL,
    ciudad        VARCHAR(100),
    fecha_alta    DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB COMMENT='Fragmentación horizontal por region (Fase 13)';

-- -----
-- PRODUCTOS: columnas básicas + detalle -> frag. vertical (F14)
-- -----

CREATE TABLE IF NOT EXISTS productos (
    id            INT            AUTO_INCREMENT PRIMARY KEY,
    sku           VARCHAR(50)    NOT NULL UNIQUE,
    nombre        VARCHAR(150)  NOT NULL,
    categoria     VARCHAR(50),
    precio        DECIMAL(10,2) NOT NULL,
    stock         INT           DEFAULT 0,
    -- Columnas de detalle (candidatas a nodo separado en Fase 14)
    descripcion   TEXT,
    ficha_tecnica TEXT,
    imagen_url    VARCHAR(255),
    peso_kg       DECIMAL(6,3),
    fecha_creacion DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB COMMENT='Fragmentación vertical: basico vs detalle (Fase 14)';

-- -----
-- PEDIDOS: campo 'region' propio -> frag. horizontal (F13)
-- -----

CREATE TABLE IF NOT EXISTS pedidos (
    id            INT            AUTO_INCREMENT PRIMARY KEY,
    cliente_id    INT            NOT NULL,
```

```

region          ENUM('norte','sur','este','oeste') NOT NULL,
fecha_pedido    DATETIME          DEFAULT CURRENT_TIMESTAMP,
estado          ENUM('pendiente','procesado','enviado',
                    'entregado','cancelado') DEFAULT 'pendiente',
total          DECIMAL(10,2),
FOREIGN KEY (cliente_id) REFERENCES clientes(id)
                ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB COMMENT='Fragmentación horizontal por region (Fase 13)';
-- -----
-- DETALLE_PEDIDOS: tabla de relación N:M
-- -----
CREATE TABLE IF NOT EXISTS detalle_pedidos (
    id            INT              AUTO_INCREMENT PRIMARY KEY,
    pedido_id     INT              NOT NULL,
    producto_id   INT              NOT NULL,
    cantidad      INT              NOT NULL CHECK (cantidad > 0),
    precio_unitario DECIMAL(10,2) NOT NULL,
    subtotal      DECIMAL(10,2) GENERATED ALWAYS AS
                (cantidad * precio_unitario) STORED,
    FOREIGN KEY (pedido_id) REFERENCES pedidos(id)
                ON UPDATE CASCADE ON DELETE CASCADE,
    FOREIGN KEY (producto_id) REFERENCES productos(id)
                ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB COMMENT='Detalle de lineas de pedido';
-- Confirmar la creación
SHOW TABLES;
EOF

```

Ejecutar el script:

 Bash

```
sudo mariadb < /tmp/crear_schema_lab_bdd.sql
```

Verificar que las cuatro tablas existen:

 Bash

```
sudo mariadb -e "SHOW TABLES IN lab_bdd;"
```

D.4 Insertar datos de prueba (VM — bdd-nodo01)

 Bash

```
cat > /tmp/insertar_datos_lab_bdd.sql << 'EOF'
USE lab_bdd;
-- ----- CLIENTES (20 registros, 5 por region) -----
-----
INSERT INTO clientes (nombre, apellido, email, telefono, region,
ciudad) VALUES
-- Norte
('Ana',      'Gutiérrez',  'ana.gutierrez@lab.test',    '8181000001',
'norte',    'Monterrey'),
('Carlos',   'Mendoza',    'carlos.mendoza@lab.test',    '6141000002',
'norte',    'Chihuahua'),
('Laura',    'Ibarra',      'laura.ibarra@lab.test',      '6641000003',
'norte',    'Tijuana'),
('Roberto',  'Soto',        'roberto.soto@lab.test',      '6621000004',
'norte',    'Hermosillo'),
('Verónica', 'Reyes',       'veronica.reyes@lab.test',    '8711000005',
'norte',    'Torreón'),
-- Sur
('Miguel',   'Castro',      'miguel.castro@lab.test',     '9991000006',
'sur',      'Mérida'),
('Sofía',    'Domínguez',  'sofia.dominguez@lab.test',   '9981000007',
'sur',      'Cancún'),
('Jorge',    'Hernández',  'jorge.hernandez@lab.test',   '9511000008',
'sur',      'Oaxaca'),
('Patricia', 'Vázquez',    'patricia.vazquez@lab.test',   '9931000009',
'sur',      'Villahermosa'),
('Ernesto',  'Luna',       'ernesto.luna@lab.test',      '9611000010',
'sur',      'Tuxtla Gtz.'),
-- Este
('Isabel',   'Morales',    'isabel.morales@lab.test',    '2291000011',
'este',     'Veracruz'),
('Héctor',   'Jiménez',    'hector.jimenez@lab.test',    '2281000012',
'este',     'Xalapa'),
('Daniela',  'Torres',     'daniela.torres@lab.test',    '8331000013',
'este',     'Tampico'),
('Alejandro', 'Ríos',       'alejandro.rios@lab.test',    '9211000014',
'este',     'Coatzacoalcos'),
('Fernanda', 'Peña',       'fernanda.pena@lab.test',     '7821000015',
'este',     'Poza Rica'),
-- Oeste
('Arturo',   'Flores',     'arturo.flores@lab.test',     '3331000016',
'oeste',    'Guadalajara'),
('Carmen',   'Rojas',      'carmen.rojas@lab.test',      '3121000017',
'oeste',    'Colima'),
```

```

('Ramón',      'Salinas',      'ramon.salinas@lab.test',      '3111000018',
'oeste',      'Tepic'),
('Lucía',      'Medina',      'lucia.medina@lab.test',      '4431000019',
'oeste',      'Morelia'),
('Eduardo',    'Vargas',      'eduardo.vargas@lab.test',    '4491000020',
'oeste',      'Aguascalientes');
-- ----- PRODUCTOS (10 registros) -----
INSERT INTO productos (sku, nombre, categoria, precio, stock,
                      descripcion, ficha_tecnica, peso_kg) VALUES
('ELEC-001', 'Monitor 24" FHD',      'Electrónica', 3500.00, 25,
'Monitor LED 24 pulgadas Full HD 1920x1080, 75Hz, panel IPS.',
'Resolución: 1920x1080 | Frecuencia: 75 Hz | Panel: IPS | Entradas:
HDMI, VGA',
3.200),
('ELEC-002', 'Teclado Mecánico TKL', 'Electrónica', 950.00, 40,
'Teclado mecánico TKL con switches azules, retroiluminación RGB.',
'Switches: Blue | Layout: TKL 87 teclas | Retroiluminación: RGB |
Conector: USB-C',
0.850),
('ELEC-003', 'Mouse Ergonómico',      'Electrónica', 480.00, 60,
'Mouse inalámbrico ergonómico, sensor óptico 1600 DPI, receptor
USB nano.',
'DPI: 800/1200/1600 | Batería: AA 12 meses | Receptor: nano USB |
Botones: 6',
0.120),
('COMP-001', 'SSD 500 GB SATA',      'Almacenamiento', 1200.00, 30,
'Unidad de estado sólido SATA III 500 GB, velocidad lectura 560 MB/s.',
'Capacidad: 500 GB | Interfaz: SATA III | Lectura: 560 MB/s | Escritura:
520 MB/s',
0.060),
('COMP-002', 'Memoria RAM 16 GB',      'Almacenamiento', 1850.00, 20,
'Módulo de memoria DDR4 16 GB 3200 MHz, latencia CL16.',
'Capacidad: 16 GB | Tipo: DDR4 | Velocidad: 3200 MHz | Latencia: CL16',
0.040),
('RED-001', 'Switch 8 puertos',      'Redes',      650.00, 15,
'Switch no administrado 8 puertos Gigabit Ethernet 10/100/1000.',
'Puertos: 8 x GbE | Capacidad: 16 Gbps | Formato: Sobremesa | PoE: No',
0.350),
('RED-002', 'Cable UTP Cat6 5m',      'Redes',      85.00, 120,
'Cable de red UTP categoría 6 de 5 metros con conectores
RJ45 moldeados.',
'Categoría: Cat6 | Longitud: 5 m | Conector: RJ45 |
Apantallamiento: UTP',
0.100),
('PER-001', 'Silla Gamer Pro',      'Mobiliario', 4200.00, 8,
'Silla de oficina/gaming con soporte lumbar, reposabrazos 4D y
reclinación 135°.',
'Material: Cuero PU | Reclinación: 90-135° | Reposabrazos: 4D | Peso máx:

```

```

120 kg',
18.500),
('PER-002', 'Escritorio L 140 cm', 'Mobiliario', 2800.00, 5,
'Escritorio en forma de L 140x120 cm con superficie de melamina 25 mm.',
'Medidas: 140x120x75 cm | Material: MDP 25 mm | Acabado: Melamina |
Color: Roble',
35.000),
('ACC-001', 'Hub USB-C 7 en 1', 'Accesorios', 420.00, 50,
'Hub multifunción USB-C con HDMI 4K, 3 USB-A 3.0, SD, MicroSD y USB-C
PD 100W.',
'Entradas: 1 USB-C | Salidas: HDMI 4K, 3xUSB-A, SD, MicroSD | PD: 100W',
0.080);
-- ----- PEDIDOS (20 registros, mix de regiones y estados)
-----
INSERT INTO pedidos (cliente_id, region, estado, total) VALUES
( 1, 'norte', 'entregado', 4450.00),
( 2, 'norte', 'enviado', 1430.00),
( 3, 'norte', 'procesado', 3500.00),
( 4, 'norte', 'pendiente', 2800.00),
( 5, 'norte', 'entregado', 735.00),
( 6, 'sur', 'entregado', 5050.00),
( 7, 'sur', 'enviado', 2050.00),
( 8, 'sur', 'procesado', 565.00),
( 9, 'sur', 'pendiente', 4620.00),
(10, 'sur', 'cancelado', 850.00),
(11, 'este', 'entregado', 1200.00),
(12, 'este', 'enviado', 3980.00),
(13, 'este', 'procesado', 6020.00),
(14, 'este', 'pendiente', 505.00),
(15, 'este', 'entregado', 1650.00),
(16, 'oeste', 'entregado', 4650.00),
(17, 'oeste', 'enviado', 3285.00),
(18, 'oeste', 'procesado', 480.00),
(19, 'oeste', 'pendiente', 7000.00),
(20, 'oeste', 'entregado', 3220.00);
-- ----- DETALLE_PEDIDOS -----
INSERT INTO detalle_pedidos (pedido_id, producto_id, cantidad,
precio_unitario) VALUES
(1, 1, 1, 3500.00), (1, 3, 2, 480.00),
(2, 2, 1, 950.00), (2, 3, 1, 480.00),
(3, 1, 1, 3500.00),
(4, 9, 1, 2800.00),
(5, 7, 5, 85.00), (5, 6, 1, 650.00),
(6, 8, 1, 4200.00), (6, 2, 1, 950.00),
(7, 5, 1, 1850.00), (7, 3, 1, 480.00),
(8, 6, 1, 650.00), (8, 7, 1, 85.00),
(9, 4, 1, 1200.00), (9, 5, 1, 1850.00), (9, 8, 1, 4200.00),
(10, 2, 1, 950.00),

```

```

(11, 4, 1, 1200.00),
(12, 1, 1, 3500.00), (12, 10, 1, 420.00),
(13, 8, 1, 4200.00), (13, 9, 1, 2800.00),
(14, 7, 3, 85.00), (14, 6, 1, 650.00),
(15, 5, 1, 1850.00),
(16, 8, 1, 4200.00), (16, 3, 1, 480.00),
(17, 1, 1, 3500.00), (17, 7, 3, 85.00),
(18, 3, 1, 480.00),
(19, 9, 1, 2800.00), (19, 1, 1, 3500.00),
(20, 1, 1, 3500.00), (20, 7, 1, 85.00);
-- Actualizar totales calculados en pedidos
UPDATE pedidos p
SET total = (
    SELECT SUM(subtotal)
    FROM detalle_pedidos dp
    WHERE dp.pedido_id = p.id
);
-- Resumen final
SELECT 'clientes' AS tabla, COUNT(*) AS registros FROM clientes
UNION ALL
SELECT 'productos', COUNT(*) FROM productos
UNION ALL
SELECT 'pedidos', COUNT(*) FROM pedidos
UNION ALL
SELECT 'detalle_pedidos', COUNT(*) FROM
detalle_pedidos;
EOF

```

 Bash

```
sudo mariadb < /tmp/insertar_datos_lab_bdd.sql
```

Verificar el conteo de registros:

 Bash

```
sudo mariadb -e "
SELECT 'clientes'          AS tabla, COUNT(*) AS registros FROM
lab_bdd.clientes          UNION ALL
SELECT 'productos',        COUNT(*)          FROM
lab_bdd.productos         UNION ALL
SELECT 'pedidos',          COUNT(*)          FROM
lab_bdd.pedidos           UNION ALL
SELECT 'detalle_pedidos',  COUNT(*)          FROM
lab_bdd.detalle_pedidos;
"
```

Salida esperada:

 Text

```
+-----+-----+
| tabla          | registros |
+-----+-----+
| clientes       |         20 |
| productos      |         10 |
| pedidos        |         20 |
| detalle_pedidos |         36 |
+-----+-----+
```

D.5 Crear los usuarios del laboratorio (VM — bdd-nodo01)

 Bash

```
sudo mariadb << 'EOF'

-- -----
-- Usuario: lab_admin
-- Propósito: administración del esquema lab_bdd desde el
--             host Windows vía DBeaver + túnel SSH, y desde
--             scripts de mantenimiento locales.
-- Autenticación: mysql_native_password (necesario para DBeaver)
-- -----
CREATE USER IF NOT EXISTS 'lab_admin'@'localhost'
    IDENTIFIED BY 'LabAdmin_2025!';
GRANT ALL PRIVILEGES ON lab_bdd.* TO 'lab_admin'@'localhost';
-- Acceso a information_schema y performance_schema en modo lectura
-- (para que DBeaver pueda explorar metadatos)
GRANT SELECT ON information_schema.* TO 'lab_admin'@'localhost';
GRANT SELECT ON performance_schema.* TO 'lab_admin'@'localhost';
-- -----
-- Usuario: app_user
-- Propósito: simular un usuario de aplicación; solo puede
--             leer y escribir datos, nunca modificar el esquema.
-- -----
CREATE USER IF NOT EXISTS 'app_user'@'localhost'
    IDENTIFIED BY 'AppUser_2025!';
GRANT SELECT, INSERT, UPDATE, DELETE ON lab_bdd.*
TO 'app_user'@'localhost';
-- -----
-- Aplicar los cambios de privilegios de inmediato
-- -----
FLUSH PRIVILEGES;
-- Verificar que los usuarios quedaron registrados
SELECT User, Host, plugin FROM mysql.user
WHERE User IN ('lab_admin', 'app_user', 'root')
ORDER BY User;
EOF
```

Importante: documentar ambas contraseñas en

`C:\LabBDD\Snapshots-Notas\fase08-notas.txt`. Son credenciales de laboratorio intencionalmente simples; nunca usar contraseñas con este patrón en producción.

D.6 Verificar los privilegios otorgados (VM — bdd-nodo01)

 Bash

```
sudo mariadb << 'EOF'
-- Ver privilegios de cada usuario
SHOW GRANTS FOR 'lab_admin'@'localhost';
SHOW GRANTS FOR 'app_user'@'localhost';
-- Confirmar a través de information_schema
SELECT GRANTEE, TABLE_SCHEMA, PRIVILEGE_TYPE, IS_GRANTABLE
FROM information_schema.SCHEMA_PRIVILEGES
WHERE TABLE_SCHEMA = 'lab_bdd'
ORDER BY GRANTEE, PRIVILEGE_TYPE;
EOF
```

Probar que `app_user` puede leer datos pero NO puede modificar el esquema:

 Bash

```
# Debe funcionar: SELECT es un privilegio de app_user
mariadb -u app_user -p'AppUser_2025!' lab_bdd \
-e "SELECT COUNT(*) AS total_clientes FROM clientes;"
# Debe fallar con Access denied: DROP no fue otorgado a app_user
mariadb -u app_user -p'AppUser_2025!' lab_bdd \
-e "DROP TABLE clientes;" 2>&1 | grep -i "access denied\|error"
```

D.7 Habilitar el slow query log (VM — bdd-nodo01 y bdd-nodo02)

Crear el archivo de configuración de logs:

 Bash

```
sudo nano /etc/mysql/mariadb.conf.d/99-lab-logs.cnf
```

Contenido completo (idéntico en ambos nodos):

Text

```
# Configuración de logs para el Laboratorio BDD
# Creado en la Fase 8. Cargado después de 50-server.cnf.
# No editar 50-server.cnf directamente.
[mariadb]
# --- Slow Query Log ---
slow_query_log                = 1
slow_query_log_file           = /var/log/mysql/mariadb-slow.log
long_query_time                = 2
# Registrar también consultas que no usan índices (útil para diagnóstico)
log_queries_not_using_indexes = 0
# --- General Query Log (deshabilitado; activar solo para depuración
# puntual) ---
general_log                   = 0
general_log_file              = /var/log/mysql/mariadb-general.log
```

Guardar con **Ctrl+O**, **Enter**, **Ctrl+X**. Reiniciar MariaDB:

Bash

```
sudo systemctl restart mariadb
sudo systemctl status mariadb
```

Verificar que el slow query log está activo dentro del motor:

Bash

```
sudo mariadb -e "SHOW VARIABLES LIKE 'slow_query%';"
sudo mariadb -e "SHOW VARIABLES LIKE 'long_query_time';"
```

Generar una consulta artificialmente lenta para confirmar que se registra (usa **SLEEP()** para simular una consulta que tarda más de 2 segundos):

Bash

```
sudo mariadb lab_bdd -e "SELECT SLEEP(3), COUNT(*) FROM clientes;"
```

Consultar el log para verificar que la entrada apareció:

 Bash

```
sudo tail -20 /var/log/mysql/mariadb-slow.log
```

Se debe ver una entrada con `Query_time: 3.xxx` apuntando al `SELECT SLEEP(3)` .

D.8 Comandos de monitoreo del motor (VM — bdd-nodo01)

 Bash

```
sudo mariadb << 'EOF'
-- ---- Estado global del servidor ----
-- Conexiones acumuladas desde el último arranque
SHOW GLOBAL STATUS LIKE 'Connections';
-- Conexiones activas en este momento
SHOW GLOBAL STATUS LIKE 'Threads_connected';
-- Consultas ejecutadas desde el arranque
SHOW GLOBAL STATUS LIKE 'Questions';
-- Bytes enviados y recibidos
SHOW GLOBAL STATUS LIKE 'Bytes_%';
-- Operaciones de InnoDB (lecturas/escrituras de páginas)
SHOW GLOBAL STATUS LIKE 'Innodb_pages_%';
-- ---- Procesos activos ----
-- Lista todos los hilos de conexión activos en este momento
SHOW FULL PROCESSLIST;
-- ---- Estado de InnoDB ----
-- Salida extensa; incluye buffer pool, transacciones activas,
-- locks, I/O. Se usa para diagnóstico avanzado.
SHOW ENGINE INNODB STATUS\G
-- ---- Resumen de tablas del laboratorio ----
SELECT TABLE_NAME          AS tabla,
       TABLE_ROWS          AS filas_aprox,
       ROUND(DATA_LENGTH/1024, 1) AS datos_KB,
       ROUND(INDEX_LENGTH/1024, 1) AS indices_KB
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'lab_bdd'
ORDER BY DATA_LENGTH DESC;
-- ---- Consulta de negocio de ejemplo para verificar los datos ----
SELECT c.region,
       COUNT(DISTINCT c.id)   AS clientes,
       COUNT(p.id)           AS pedidos,
       ROUND(SUM(p.total), 2) AS ingresos_total
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id
GROUP BY c.region
ORDER BY ingresos_total DESC;
EOF
```

D.9 Respaldo básico con `mysqldump` (VM — bdd-nodo01)

Crear la carpeta de respaldos dentro del nodo:

 Bash

```
sudo mkdir -p /opt/lab_bdd_backups
sudo chown bddadmin:bddadmin /opt/lab_bdd_backups
```

Respaldo 1 — Solo estructura (DDL):

 Bash

```
mysqldump -u lab_admin -p'LabAdmin_2025!' \
  --no-data \
  --routines \
  --triggers \
  --single-transaction \
  lab_bdd > /opt/lab_bdd_backups/lab_bdd_schema_$(date +%Y%m%d).sql
ls -lh /opt/lab_bdd_backups/
```

Respaldo 2 — Estructura y datos completos:

 Bash

```
mysqldump -u lab_admin -p'LabAdmin_2025!' \
  --routines \
  --triggers \
  --single-transaction \
  --quick \
  lab_bdd > /opt/lab_bdd_backups/lab_bdd_completo_$(date +%Y%m%d).sql
ls -lh /opt/lab_bdd_backups/
wc -l /opt/lab_bdd_backups/lab_bdd_completo_$(date +%Y%m%d).sql
```

Verificar que el archivo de respaldo es válido (primeras y últimas líneas):

 Bash

```
head -20 /opt/lab_bdd_backups/lab_bdd_completo_$(date +%Y%m%d).sql
tail -10 /opt/lab_bdd_backups/lab_bdd_completo_$(date +%Y%m%d).sql
```

La última línea debe ser `-- Dump completed on ...` seguida de la marca de tiempo.

Sobre los parámetros usados:

- `--single-transaction` : toma un snapshot consistente de InnoDB sin bloquear las tablas durante el respaldo (equivalente a un `START TRANSACTION` interno).
- `--quick` : descarga las filas de a una en lugar de cargar toda la tabla en memoria; esencial para bases de datos grandes.
- `--routines` y `--triggers` : incluye procedimientos, funciones y disparadores si los hubiera (en este laboratorio no hay aún, pero es buena práctica incluirlos).

D.10 Conectar DBeaver desde el host Windows con túnel SSH

Instalación de DBeaver (host Windows):

Descargar **DBeaver Community Edition** desde <https://dbeaver.io/download/> (archivo `.exe` o `.zip` para Windows). No requiere instalación de JDK separado en versiones 23+; el JDK embebido viene incluido en el instalador.

Configurar la conexión con túnel SSH en DBeaver:

1. Abrir DBeaver → `Archivo` → `Nueva conexión` → seleccionar **MariaDB**.
2. En la pestaña **Principal**:
 - Host: `127.0.0.1`
 - Puerto: `3306`
 - Base de datos: `lab_bdd`
 - Usuario: `lab_admin`
 - Contraseña: `LabAdmin_2025!`
3. En la pestaña **SSH**:
 - Marcar `Use SSH Tunnel`
 - Host/IP: `192.168.56.101`
 - Puerto SSH: `22`
 - Usuario: `bddadmin`
 - Método de autenticación: `Password`
 - Contraseña SSH: (contraseña definida para `bddadmin` en la Fase 5)
4. Hacer clic en `Test Connection`. DBeaver primero abre el túnel SSH al puerto 22 de la VM y luego conecta al `127.0.0.1:3306` que el túnel reenvía.
5. Si la prueba es exitosa: `Finalizar`.

Repetir los pasos para crear una segunda conexión apuntando a `192.168.56.102` (`bdd-nodo02`), con los mismos usuarios y contraseñas de MariaDB.

Si DBeaver solicita instalar el driver JDBC de MariaDB, aceptar y dejar que lo descargue automáticamente (requiere internet en el host Windows, no en la VM).

D.11 Repetir los Pasos D.3 a D.9 en `bdd-nodo02` (VM — bdd-nodo02)

Conectarse a `bdd-nodo02` por SSH y ejecutar exactamente los mismos scripts.

El contenido de `lab_bdd` debe ser idéntico en ambos nodos:

 PowerShell

```
# Desde el host, abrir una segunda terminal SSH
ssh bddadmin@192.168.56.102
```

Dentro de `bdd-nodo02`, ejecutar en orden:

 Bash

```
# Copiar los scripts desde bdd-nodo01 usando scp (ejecutar desde
bdd-nodo02)
scp bddadmin@192.168.56.101:/tmp/crear_schema_lab_bdd.sql /tmp/
scp bddadmin@192.168.56.101:/tmp/insertar_datos_lab_bdd.sql /tmp/
# Aplicar el esquema y los datos
sudo mariadb < /tmp/crear_schema_lab_bdd.sql
sudo mariadb < /tmp/insertar_datos_lab_bdd.sql
# Crear usuarios
sudo mariadb << 'EOF'
CREATE USER IF NOT EXISTS 'lab_admin'@'localhost'
    IDENTIFIED BY 'LabAdmin_2025!';
GRANT ALL PRIVILEGES ON lab_bdd.* TO 'lab_admin'@'localhost';
GRANT SELECT ON information_schema.* TO 'lab_admin'@'localhost';
GRANT SELECT ON performance_schema.* TO 'lab_admin'@'localhost';
CREATE USER IF NOT EXISTS 'app_user'@'localhost'
    IDENTIFIED BY 'AppUser_2025!';
GRANT SELECT, INSERT, UPDATE, DELETE ON lab_bdd.*
TO 'app_user'@'localhost';
FLUSH PRIVILEGES;
EOF
# Crear configuración de logs (idéntica a bdd-nodo01)
sudo tee /etc/mysql/mariadb.conf.d/99-lab-logs.cnf > /dev/null << 'EOF'
[mariadb]
slow_query_log                = 1
slow_query_log_file            = /var/log/mysql/mariadb-slow.log
long_query_time                = 2
log_queries_not_using_indexes = 0
general_log                    = 0
general_log_file               = /var/log/mysql/mariadb-general.log
EOF
sudo systemctl restart mariadb
# Crear carpeta de respaldos y ejecutar respaldo inicial
sudo mkdir -p /opt/lab_bdd_backups
sudo chown bddadmin:bddadmin /opt/lab_bdd_backups
mysqldump -u lab_admin -p'LabAdmin_2025!' \
    --routines --triggers --single-transaction --quick \
    lab_bdd > /opt/lab_bdd_backups/lab_bdd_completo_$(date +%Y%m%d).sql
# Verificación final
sudo mariadb -e "
    SELECT 'clientes', COUNT(*) FROM lab_bdd.clientes          UNION ALL
    SELECT 'productos', COUNT(*) FROM lab_bdd.productos        UNION ALL
    SELECT 'pedidos', COUNT(*) FROM lab_bdd.pedidos             UNION ALL
    SELECT 'detalle', COUNT(*) FROM lab_bdd.detalle_pedidos;
"
```

D.12 Apagar ambas VMs y tomar el snapshot `fase08-completa` (host)

Desde cada sesión SSH (ejecutar en cada nodo):

```
 Bash  
  
sudo poweroff
```

Confirmar desde el host que ambas VMs están detenidas:

```
 PowerShell  
  
VBoxManage list runningvms
```

La salida debe estar vacía. Tomar los snapshots:

```
 PowerShell  
  
VBoxManage snapshot "bdd-nodo01" take "fase08-completa" \  
--description "lab_bdd creada: 20 clientes, 10 productos, 20 pedidos,  
35 detalles. Usuarios lab_admin y app_user. Slow log activo. Respaldo  
mysqldump generado."  
VBoxManage snapshot "bdd-nodo02" take "fase08-completa" \  
--description "lab_bdd creada: idéntica a bdd-nodo01. Usuarios lab_admin y  
app_user. Slow log activo. Respaldo mysqldump generado."
```

Confirmar los snapshots:

```
 PowerShell  
  
VBoxManage snapshot "bdd-nodo01" list  
VBoxManage snapshot "bdd-nodo02" list
```

Cada VM debe mostrar cuatro snapshots en orden: `fase05-completa` , `fase06-completa` , `fase07-completa` y `fase08-completa` .

E. Verificación de funcionamiento

Esta fase se considera completa cuando se cumplen los siguientes puntos en **ambos nodos** (`bdd-nodo01` y `bdd-nodo02`):

1. `SHOW DATABASES;` dentro de `sudo mariadb` muestra `lab_bdd` además de las cuatro bases de datos del sistema. La base de datos `test` **no** existe.
 2. `SHOW TABLES IN lab_bdd;` muestra exactamente cuatro tablas: `clientes` , `productos` , `pedidos` , `detalle_pedidos` .
 3. La consulta de conteo confirma 20 clientes, 10 productos, 20 pedidos y 36 registros de detalle.
 4. `SHOW GRANTS FOR 'lab_admin'@'localhost';` muestra `GRANT ALL PRIVILEGES ON lab_bdd.*` .
 5. `SHOW GRANTS FOR 'app_user'@'localhost';` muestra únicamente los privilegios `SELECT, INSERT, UPDATE, DELETE` sobre `lab_bdd.*` y ninguno adicional.
 6. El intento de `DROP TABLE` con `app_user` falla con `Access denied` .
 7. `SHOW VARIABLES LIKE 'slow_query_log';` devuelve `ON` .
 8. `SHOW VARIABLES LIKE 'long_query_time';` devuelve `2.000000` .
 9. El archivo `/var/log/mysql/mariadb-slow.log` existe y contiene al menos una entrada del `SELECT SLEEP(3)` ejecutado en la sección D.7.
 10. El archivo de respaldo `lab_bdd_completo_YYYYMMDD.sql` existe en `/opt/lab_bdd_backups/` con un tamaño mayor a cero y la línea `-- Dump completed on` al final.
 11. Desde DBeaver en el host Windows, la conexión con túnel SSH a `bdd-nodo01` y a `bdd-nodo02` abre exitosamente y muestra las tablas de `lab_bdd` .
 12. La conectividad Host-Only sigue intacta: `ping 192.168.56.101` y `ping 192.168.56.102` responden desde el host Windows.
 13. Los snapshots `fase08-completa` existen en `bdd-nodo01` y `bdd-nodo02` .
 14. Las credenciales de `lab_admin` y `app_user` están documentadas en `C:\LabBDD\Snapshots-Notas\fase08-notas.txt` .
-

F. Problemas comunes y soluciones

Problema	Causa probable	Solución
<code>CREATE USER</code> falla con <code>ERROR 1396: Operation CREATE USER failed</code>	El usuario ya existe de un intento anterior fallido	Usar <code>DROP USER IF EXISTS 'lab_admin'@'localhost';</code> antes de volver a crear, o usar la sintaxis <code>CREATE USER IF NOT EXISTS</code>
<code>SHOW GRANTS</code> devuelve un error de <code>no such grant</code>	El usuario se creó pero <code>GRANT</code> no se ejecutó correctamente, o se especificó un <code>host</code> diferente	Verificar con <code>SELECT User, Host FROM mysql.user;</code> que el usuario y el host coincidan exactamente con lo que se usó en <code>GRANT</code>
<code>mysqldump</code> falla con <code>Access denied; you need the PROCESS privilege</code>	La versión de MariaDB/MySQL es $\geq$ 10.5 y <code>mysqldump</code> necesita <code>SHOW MASTER STATUS</code> al usar <code>--single-transaction</code> sin el flag <code>--no-tablespaces</code>	Añadir el flag <code>--no-tablespaces</code> al comando de <code>mysqldump</code> , o usar <code>root</code> con <code>sudo</code> : <code>sudo mysqldump -u root ...</code>
DBeaver muestra <code>Communications link failure</code> o <code>Connection refused</code>	El túnel SSH no se estableció correctamente antes de intentar la conexión a MariaDB	En DBeaver, verificar que la pestaña SSH tenga la IP del Host-Only (<code>.101 / .102</code>), usuario <code>bddadmin</code> y contraseña correcta; probar primero un cliente SSH externo (PowerShell) para descartar problemas de credenciales SSH
DBeaver muestra <code>Public Key Retrieval is not allowed</code>	El driver JDBC de MariaDB requiere ajuste de parámetros SSL para conexión local	En la configuración de conexión de DBeaver, pestaña <code>Driver properties</code> , añadir la propiedad <code>allowPublicKeyRetrieval = true</code>
El slow query log no registra la consulta <code>SELECT SLEEP(3)</code>	El archivo de configuración <code>99-lab-logs.cnf</code> tiene un error de sintaxis o el servicio no se reinició tras crearlo	Verificar la sintaxis con <code>sudo mariadb -e "SHOW VARIABLES LIKE 'slow_query_log';"</code> ; si devuelve <code>OFF</code> , revisar el archivo de configuración y ejecutar <code>sudo systemctl restart mariadb</code>
<code>scp</code> falla al copiar los scripts de <code>bdd-nodo01</code> a <code>bdd-nodo02</code>	La clave pública SSH no está configurada entre nodos (se usa	Agregar la flag <code>-o StrictHostKeyChecking=no</code> al <code>scp</code> la primera vez e introducir la contraseña cuando se solicite;

	autenticación por contraseña)	alternativamente, copiar el contenido del script manualmente
El conteo de <code>detalle_pedidos</code> da menos de 36	Alguna clave foránea falló durante la inserción, posiblemente porque los <code>producto_id</code> o <code>pedido_id</code> referenciados no existían en ese momento	Verificar el orden de inserción (primero <code>clientes</code> , luego <code>productos</code> , luego <code>pedidos</code> , por último <code>detalle_pedidos</code>); si hay errores de FK, truncar todas las tablas (<code>SET FOREIGN_KEY_CHECKS=0</code>) y reejecutar los scripts en orden
<code>UPDATE pedidos SET total = ...</code> actualiza 0 filas	El subquery de <code>detalle_pedidos</code> no encontró filas porque la inserción anterior falló parcialmente	Verificar con <code>SELECT COUNT(*) FROM detalle_pedidos;</code> si hay filas; si no hay, repetir la inserción de <code>detalle_pedidos</code> antes de ejecutar el <code>UPDATE</code>
<code>systemctl restart mariadb</code> tarda más de 30 segundos o falla	Error de sintaxis en <code>99-lab-logs.cnf</code> que impide que el motor arranque	Revisar el log de arranque: <code>sudo journalctl -u mariadb --no-pager -n 30</code> ; buscar la línea <code>[ERROR]</code> o <code>unknown variable</code> para identificar el parámetro mal escrito

G. Checklist de validación

- ☐ `SHOW DATABASES;` muestra `lab_bdd` en ambos nodos; no existe `test`.
- ☐ Las cuatro tablas (`clientes`, `productos`, `pedidos`, `detalle_pedidos`) existen en `lab_bdd` en ambos nodos con las columnas y tipos correctos.
- ☐ Los datos de prueba están insertados: 20 clientes, 10 productos, 20 pedidos, 36 detalles en **ambos nodos**.
- ☐ `SHOW GRANTS FOR 'lab_admin'@'localhost';` muestra `ALL PRIVILEGES ON lab_bdd.*` en ambos nodos.
- ☐ `SHOW GRANTS FOR 'app_user'@'localhost';` muestra únicamente los privilegios DML (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) sobre `lab_bdd.*` en ambos nodos.
- ☐ El intento de ejecutar `DROP TABLE` con `app_user` devuelve `Access denied`.
- ☐ `/etc/mysql/mariadb.conf.d/99-lab-logs.cnf` existe en ambos nodos con el contenido correcto.
- ☐ `SHOW VARIABLES LIKE 'slow_query_log';` devuelve `ON` en ambos nodos.
- ☐ `/var/log/mysql/mariadb-slow.log` contiene la entrada del `SELECT SLEEP(3)` ejecutado durante la verificación, en ambos nodos.

- ☐ Los archivos de respaldo `mysqldump` existen en `/opt/lab_bdd_backups/` con la línea `-- Dump completed on` al final, en ambos nodos.
 - ☐ La conexión DBeaver con túnel SSH funciona hacia `bdd-nodo01` y `bdd-nodo02` y muestra las tablas de `lab_bdd`.
 - ☐ La consulta de resumen por región ejecutada en D.8 devuelve 4 filas (norte, sur, este, oeste) con datos de clientes, pedidos e ingresos.
 - ☐ La conectividad Host-Only sigue funcionando: `ping 192.168.56.101` y `ping 192.168.56.102` responden desde el host Windows.
 - ☐ Ambas VMs se apagaron de forma ordenada.
 - ☐ Los snapshots `fase08-completa` existen en `bdd-nodo01` y `bdd-nodo02`.
 - ☐ Las credenciales de los usuarios creados están documentadas en `C:\LabBDD\Snapshots-Notas\fase08-notas.txt`.
 - ☐ Puedo explicar la diferencia entre los plugins `unix_socket` y `mysql_native_password` y cuándo se usa cada uno.
 - ☐ Puedo explicar por qué el esquema `lab_bdd` fue diseñado con el campo `region` y la separación entre columnas básicas y de detalle en `productos`.
-

Preguntas teóricas para estudiantes

1. MariaDB identifica a los usuarios mediante el par `usuario@host` en lugar de solo por el nombre de usuario. Explica qué implicación tiene esto en el modelo de seguridad: ¿puede existir un usuario `app_user@'localhost'` y al mismo tiempo un `app_user@'192.168.56.%'` con privilegios totalmente distintos? ¿En qué escenario de este laboratorio sería relevante esa diferencia?
2. Describe el flujo completo de una conexión DBeaver al motor MariaDB usando túnel SSH: ¿qué componentes participan, en qué orden establecen la comunicación, y por qué el motor de MariaDB solo ve conexiones provenientes de `127.0.0.1` a pesar de que DBeaver está en una máquina diferente?
3. ¿Qué diferencia hay entre `mysqldump --single-transaction` y un respaldo sin esa opción? ¿Por qué es importante la opción `--single-transaction` para bases de datos InnoDB, y en qué situación **no** sería suficiente para garantizar consistencia?
4. El diseño de la tabla `pedidos` incluye una columna `region` que duplica (de forma controlada) la región del cliente asociado. Discute las ventajas y desventajas de esta redundancia desde la perspectiva de la **fragmentación horizontal** planeada para la Fase 13. ¿Qué problema resuelve esta redundancia al momento de fragmentar?

5. El *slow query log* se configuró con `long_query_time = 2`. Explica qué criterios se usarían para elegir un umbral más bajo (por ejemplo, 0.5 segundos) o más alto (por ejemplo, 10 segundos) en un sistema real, y qué consecuencias tiene sobre el tamaño del log y el rendimiento del servidor cada elección.
-

Ejercicios prácticos

1. **Exploración de metadatos con `information_schema`.**

Escribir una consulta SQL que liste, para cada tabla de `lab_bdd`, el nombre de cada columna, su tipo de dato, si admite NULL, si tiene valor por defecto y si forma parte de una clave primaria o foránea. La consulta debe usar `information_schema.COLUMNS` y `information_schema.KEY_COLUMN_USAGE` con un `JOIN`, y ordenar el resultado por tabla y por posición de columna. Ejecutarla dentro de `sudo mariadb` y documentar los resultados.

2. **Verificación de integridad del respaldo.**

Crear una base de datos temporal `lab_bdd_test` en `bdd-nodo01`, restaurar el respaldo generado con `mysqldump` usando `mariadb lab_bdd_test < archivo.sql`, y ejecutar la misma consulta de conteo de registros en ambas bases de datos (`lab_bdd` y `lab_bdd_test`) para confirmar que los datos son idénticos. Al finalizar, eliminar `lab_bdd_test` con `DROP DATABASE`.

3. **Análisis del slow query log.**

Habilitar temporalmente `log_queries_not_using_indexes = 1` en `99-lab-logs.cnf`, reiniciar MariaDB y ejecutar cinco consultas variadas sobre `lab_bdd` (al menos dos sin cláusula `WHERE` con índice, dos con `JOIN` entre tablas y una con `GROUP BY`). Revisar el slow query log con `sudo tail -50 /var/log/mysql/mariadb-slow.log` e identificar cuáles consultas aparecen y por qué. Restaurar el valor original (`log_queries_not_using_indexes = 0`) al finalizar.

Reto adicional para alumnos avanzados

Implementar un **script de respaldo automatizado** en Bash que: (a) genere un `mysqldump` de `lab_bdd` con fecha y hora en el nombre del archivo, (b) comprima el resultado con `gzip`, © elimine automáticamente los respaldos con más de 7 días de antigüedad para evitar llenar el disco, y (d) registre en un archivo de log propio (`/opt/lab_bdd_backups/backup.log`) la fecha, hora, tamaño del archivo generado y si el proceso fue exitoso o no. El script debe poder

ejecutarse manualmente y también estar preparado para ser invocado por `cron`.
Incluir en la entrega el script comentado y la entrada de `crontab` sugerida para ejecutarlo diariamente a las 02:00 AM.

Criterios de evaluación para el profesor

Criterio	Peso	Indicador de logro
Creación correcta del esquema <code>lab_bdd</code>	20%	Las cuatro tablas existen en ambos nodos con los tipos de dato correctos, claves foráneas definidas y comentarios que identifican el rol de cada tabla en la distribución futura
Gestión de usuarios y privilegios	20%	<code>lab_admin</code> y <code>app_user</code> existen con exactamente los privilegios descritos; se demuestra que <code>app_user</code> no puede ejecutar DDL
Configuración y verificación de logs	15%	<code>99-lab-logs.cnf</code> existe y está correcto; <code>SHOW VARIABLES</code> confirma el slow log activo; el log muestra al menos una entrada de la consulta de prueba
Respaldo con <code>mysqldump</code>	15%	Se generaron los dos tipos de respaldo (solo DDL y completo); el alumno puede explicar la diferencia entre ambos y el propósito de <code>--single-transaction</code>
Conexión remota con DBeaver y túnel SSH	15%	DBeaver conecta exitosamente a ambos nodos sin errores; el alumno puede explicar por qué se usa un túnel en lugar de abrir directamente el puerto 3306
Comprensión conceptual (preguntas teóricas)	15%	Responde correctamente las preguntas 1 a 5 usando vocabulario técnico apropiado y haciendo referencia al contexto específico del laboratorio

Preparación para la siguiente fase

La **Fase 9: Diseño de la Arquitectura Distribuida** requerirá:

- El esquema `lab_bdd` idéntico en `bdd-nodo01` y `bdd-nodo02` (esta fase).
- Snapshot `fase08-completa` tomado en ambas VMs.

- Comprensión del diseño del esquema: por qué cada tabla tiene los campos que tiene y cómo se mapean a estrategias de fragmentación futuras.
- Acceso SSH y DBeaver funcionando (para consultar y documentar el esquema durante la fase de diseño).

La Fase 9 no instalará ni configurará nada nuevo en las VMs: será una fase teórico-analítica en la que se tomará el esquema `lab_bdd` ya existente y se diseñará, sobre papel y diagramas, el plan de distribución completo que se implementará en las Fases 10 a 16.

Fase 9 — Diseño de la Arquitectura Distribuida

MGTI. Baltazar Martinez Galla

Continuación directa de la Fase 8. Ambos nodos tienen MariaDB 10.11 instalado con el esquema `lab_bdd` poblado con datos de prueba y snapshot `fase08-completa` tomado. Esta fase **no instala ni configura nada nuevo** en las máquinas virtuales: es una fase de análisis y diseño en la que se toma el esquema centralizado existente y se elabora el plan completo de distribución que se implementará en las Fases 10 a 16. El principal entregable es el **Documento de Diseño Distribuido (DDD)**, que servirá como referencia técnica en todas las fases posteriores.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Aplicar la metodología **top-down** de diseño de bases de datos distribuidas para traducir un esquema relacional centralizado en un plan de distribución formal.
2. Distinguir y construir los tres esquemas del diseño distribuido: **esquema global**, **esquema de fragmentación** y **esquema de asignación**.
3. Definir fragmentos horizontales mediante **predicados simples** y combinaciones de **predicados minterm**, verificando su correctitud (completitud, disjunción y reconstrucción).
4. Definir fragmentos verticales usando el criterio de **afinidad de columnas** y verificar su correctitud (completitud y reconstrucción mediante JOIN sobre la PK).
5. Diseñar la estrategia de **fragmentación híbrida** que emerge de la combinación de las fragmentaciones horizontal y vertical sobre distintas tablas del mismo esquema.
6. Justificar las **decisiones de asignación de datos** a nodos específicos usando criterios de localidad de acceso, balanceo de carga y co-localización de tablas relacionadas por claves foráneas.
7. Elaborar la **matriz de asignación completa** del laboratorio, especificando para cada fragmento el nodo destino y la fase de implementación.
8. Escribir y ejecutar consultas SQL que simulen, en el nodo centralizado actual, las operaciones de reconstrucción que el sistema distribuido realizará en fases posteriores, verificando que producen resultados correctos.
9. Producir el **Documento de Diseño Distribuido** (`fase09-disenyo-distribuido.md`) que actuará como especificación técnica de referencia para las Fases 10 a 16.

B. Conceptos teóricos necesarios

1. Metodología top-down de diseño de BDD.

El diseño de una base de datos distribuida puede abordarse de dos formas:

- *Top-down*: se parte de los requisitos globales, se diseña el esquema centralizado y después se decide cómo distribuirlo. Es el enfoque de este laboratorio: el esquema `lab_bdd` ya existe como punto de partida centralizado y se redistribuirá.
- *Bottom-up*: se integran esquemas preexistentes en distintos nodos en un esquema global unificado. Se usa cuando los datos ya existen en múltiples sistemas heredados.

El proceso top-down involucra tres fases de diseño, que se desarrollan en esta fase del laboratorio:

 Paso	 Nombre	 Pregunta que responde
1	 Diseño del esquema global	 ¿Qué datos existen y cómo se relacionan?
2	 Diseño del esquema de fragmentación	 ¿Cómo se divide cada relación en partes?
3	 Diseño del esquema de asignación	 ¿Qué fragmento va a qué nodo?

2. Fragmentación horizontal.

Una fragmentación horizontal divide las **filas** de una relación `R` en subconjuntos llamados fragmentos `R1, R2, ..., Rn`, cada uno definido por un predicado de selección:

 Text

$$R_i = \sigma(\text{predicado}_i)(R)$$

- *Predicado simple*: condición sobre un único atributo; por ejemplo, `region = 'norte'`.

- *Predicado minterm*: combinación booleana completa de todos los predicados simples relevantes. Si hay n predicados simples $p_1, p_2, \dots, p_n$, existen hasta 2^n minterms posibles; solo los minterms *no vacíos* generan fragmentos reales.

Para `region` con cuatro valores distintos, los predicados simples son `p1: region = 'norte'`, `p2: region = 'sur'`, `p3: region = 'este'`, `p4: region = 'oeste'`. Este laboratorio los agrupa en dos fragmentos de dos minterms cada uno, por limitación de nodos de sharding disponibles (dos: nodo04 y nodo05).

3. Condiciones de correctitud de la fragmentación.

Toda fragmentación debe cumplir tres condiciones para ser válida:

- **Compleitud** (*completeness*): $\forall t \in R, \exists i \text{ tal que } t \in R_i$. Ninguna fila puede perderse; toda tupla de la relación global debe aparecer en al menos un fragmento.
- **Disjunción** (*disjointness*): $\forall i \neq j, R_i \cap R_j = \emptyset$. En fragmentación horizontal, ninguna fila debe pertenecer a más de un fragmento. En fragmentación vertical, las columnas no clave no deben repetirse (la PK sí se duplica, y es obligatorio hacerlo para la reconstrucción).
- **Reconstrucción** (*reconstruction*): la relación global puede obtenerse a partir de sus fragmentos — mediante `UNION ALL` para fragmentación horizontal, o mediante `JOIN` sobre la PK para fragmentación vertical.

4. Fragmentación vertical.

Una fragmentación vertical divide las **columnas** de una relación `R`. Formalmente:

Text

$$V_i = \pi(C_i \cup \{PK\})(R)$$

Cada fragmento proyecta un subconjunto de columnas `Ci`, más la **clave primaria**, que siempre debe estar presente en todos los fragmentos para permitir la reconstrucción por JOIN. Los conjuntos de columnas no clave deben ser disjuntos: $C_i \cap C_j = \emptyset$ para todo $i \neq j$.

La técnica de **afinidad de columnas** estima qué tan frecuentemente se acceden conjuntamente dos columnas. Las de alta afinidad entre sí se agrupan en el mismo fragmento. En la práctica del laboratorio se usa el criterio de *patrón de uso*: columnas que aparecen en consultas frecuentes (listados, búsquedas, cálculo de precios) se separan de columnas de descripción extensa/TEXT consultadas solo en pantallas de detalle.

5. Fragmentación híbrida.

La fragmentación híbrida combina ambas estrategias sobre el mismo esquema. En este laboratorio surge de forma natural: `clientes`, `pedidos` y `detalle_pedidos` se fragmentan **horizontalmente** por región (Fase 13), mientras que `productos` se fragmenta **verticalmente** por grupos de columnas (Fase 14). Las consultas que combinan ambas tablas en un mismo resultado (Fase 15) cruzan los dos tipos de fragmentación, constituyendo el escenario híbrido sin necesidad de introducir una tercera tabla de ejemplo.

6. Estrategias de asignación de datos.

Una vez definidos los fragmentos, se decide en qué nodo(s) reside cada uno:

- *Particionada*: cada fragmento existe en exactamente un nodo. Maximiza el uso del espacio disponible; la caída de un nodo hace inaccesibles sus fragmentos.
- *Replicada*: el fragmento (o la relación completa) se copia en varios nodos. Mejora disponibilidad y rendimiento de lectura; complica las escrituras (consistencia).
- *Híbrida*: algunos fragmentos son particionados y otros son replicados.

Este laboratorio usa el modelo híbrido: `lab_bdd` completa se *replica* entre nodo01 y nodo02 (Fase 10, replicación física), y sobre los nodos 04 y 05 se aplica *particionamiento horizontal* de algunas tablas (Fase 13).

7. Co-localización de tablas relacionadas.

Cuando dos tablas se unen frecuentemente mediante JOIN (como `pedidos` y `detalle_pedidos`), y ambas se van a fragmentar horizontalmente, la decisión de **co-localizar** sus fragmentos en el mismo nodo elimina los joins remotos entre nodos, que son la operación más costosa en un sistema distribuido (transferencia de datos por red). La regla práctica es: si existe una clave foránea entre dos tablas y la tabla hija puede fragmentarse derivadamente por el mismo predicado de la tabla padre, deben co-localizarse en el mismo nodo.

8. El nodo coordinador y el motor Spider.

En un sistema distribuido, algún nodo debe actuar como **coordinador de consultas globales**: recibe las consultas del cliente, las descompone en subconsultas dirigidas a los nodos con los fragmentos relevantes, recibe los resultados parciales y los combina en la respuesta final. En este laboratorio, `bdd-nodo06` desempeñará ese rol mediante el motor **Spider** de MariaDB — un motor de almacenamiento nativo de MariaDB que actúa como tabla puente hacia tablas remotas en otros servidores MariaDB. Desde el punto de vista del cliente, Spider hace que las tablas distribuidas en nodo04/nodo05 aparezcan como tablas locales en nodo06, implementando así la transparencia de fragmentación y ubicación.

C. Procedimiento paso a paso

Paso 1 — Iniciar `bdd-nodo01` y conectarse por SSH.

Solo se necesita un nodo para el análisis; ambos tienen el mismo esquema y datos.

Paso 2 — Ejecutar el análisis estadístico de `lab_bdd`.

Recopilar métricas de distribución de datos y de tamaño de columnas que justificarán cada decisión de diseño con evidencia cuantitativa (sección D.2).

Paso 3 — Diseñar y verificar la fragmentación horizontal.

Definir los predicados de los dos fragmentos de `clientes`, `pedidos` y `detalle_pedidos`; ejecutar las consultas de verificación de completitud, disjunción y reconstrucción sobre los datos reales (sección D.3).

Paso 4 — Diseñar y verificar la fragmentación vertical de `productos`.

Definir los dos grupos de columnas, justificar la agrupación según el patrón de uso esperado y ejecutar la verificación de correctitud (sección D.4).

Paso 5 — Simular las consultas distribuidas en el entorno centralizado.

Escribir las cuatro consultas que en las Fases 13–16 el coordinador ejecutará de forma distribuida y ejecutarlas localmente para confirmar que producen resultados correctos antes de dispersar los datos (sección D.5).

Paso 6 — Revisar la topología de replicación.

Generar el diagrama de texto de la topología completa del laboratorio dentro del nodo, que se trasladará al documento de diseño (sección D.6).

Paso 7 — Generar la matriz de asignación completa.

Visualizar como consulta SQL la tabla que relaciona cada fragmento con su nodo destino, su IP y su fase de implementación (sección D.7).

Paso 8 — Crear el Documento de Diseño Distribuido en el host.

Generar el archivo `C:\LabBDD\Documentacion\fase09-disenyo-distribuido.md` que consolida el diseño completo y servirá como referencia obligatoria en las Fases 10–16 (sección D.8).

Paso 9 — Apagar la VM y tomar el snapshot de cierre de fase.

Aunque no hubo cambios en el sistema operativo ni en el motor, el snapshot marca el estado verificado del laboratorio en el punto en que el diseño fue completado (D.9).

D. Comandos completos

Los comandos marcados **(VM)** se ejecutan en una sesión SSH en `bdd-nodo01`. Los marcados **(host)** se ejecutan en PowerShell en Windows. Los bloques iniciados con `sudo mariadb` se ejecutan dentro del prompt del motor.

D.1 Iniciar `bdd-nodo01` y conectarse (host)

➤ PowerShell

```
VBoxManage startvm "bdd-nodo01" --type headless
```

Esperar 20–30 segundos y conectarse:

➤ PowerShell

```
ssh bddadmin@192.168.56.101
```

D.2 Análisis estadístico del esquema lab_bdd (VM)

 Bash

```
sudo mariadb lab_bdd << 'EOF'

-- =====
-- BLOQUE 1: Estructura del esquema global
-- =====

-- Tablas, motor y tamaño aproximado
SELECT TABLE_NAME                AS tabla,
       ENGINE                     AS motor,
       TABLE_ROWS                AS filas_aprox,
       ROUND((DATA_LENGTH + INDEX_LENGTH) / 1024.0, 1) AS total_KB
FROM   information_schema.TABLES
WHERE  TABLE_SCHEMA = 'lab_bdd'
ORDER BY DATA_LENGTH DESC;

-- Columnas por tabla (base para decisiones de fragmentación vertical)
SELECT TABLE_NAME                AS tabla,
       COLUMN_NAME                AS columna,
       ORDINAL_POSITION          AS pos,
       DATA_TYPE                 AS tipo,
       CHARACTER_MAXIMUM_LENGTH AS max_chars,
       IS_NULLABLE               AS nutable,
       COLUMN_KEY                 AS clave
FROM   information_schema.COLUMNS
WHERE  TABLE_SCHEMA = 'lab_bdd'
ORDER BY TABLE_NAME, ORDINAL_POSITION;

-- Restricciones de clave foránea (determinan co-localización obligatoria)
SELECT CONSTRAINT_NAME            AS restriccion,
       TABLE_NAME                AS tabla_hijo,
       COLUMN_NAME                AS columna_hijo,
       REFERENCED_TABLE_NAME      AS tabla_padre,
       REFERENCED_COLUMN_NAME     AS columna_padre
FROM   information_schema.KEY_COLUMN_USAGE
WHERE  TABLE_SCHEMA = 'lab_bdd'
       AND REFERENCED_TABLE_NAME IS NOT NULL
ORDER BY TABLE_NAME;

-- =====
-- BLOQUE 2: Distribución del atributo de fragmentación
-- =====

-- Distribución de clientes por región (atributo candidato
a fragmentación)
SELECT region,
       COUNT(*)                    AS num_clientes,
       ROUND(COUNT(*) * 100.0 / 20, 1) AS pct_del_total,
       GROUP_CONCAT(ciudad ORDER BY ciudad) AS ciudades
FROM   clientes
GROUP BY region
```

```

ORDER BY region;
-- Distribución de pedidos por región y estado
SELECT region,
       COUNT(*) AS num_pedidos,
       ROUND(SUM(total), 2) AS ingresos
FROM pedidos
GROUP BY region
ORDER BY region;
-- Líneas de detalle co-localizables con cada fragmento de pedidos
SELECT p.region,
       COUNT(DISTINCT p.id) AS pedidos,
       COUNT(dp.id) AS lineas_detalle,
       ROUND(SUM(dp.subtotal), 2) AS facturacion
FROM pedidos p
JOIN detalle_pedidos dp ON dp.pedido_id = p.id
GROUP BY p.region
ORDER BY p.region;
-- =====
-- BLOQUE 3: Análisis de columnas de 'productos'
--           (candidatas a fragmentación vertical)
-- =====
-- Longitud promedio por columna de tipo texto (identifica las "pesadas")
SELECT 'sku' AS columna, ROUND(AVG(LENGTH(sku)), 1) AS
bytes_prom FROM productos UNION ALL
SELECT 'nombre', ROUND(AVG(LENGTH(nombre)), 1)
FROM productos UNION ALL
SELECT 'categoria', ROUND(AVG(LENGTH(categoria)), 1)
FROM productos UNION ALL
SELECT 'descripcion', ROUND(AVG(LENGTH(descripcion)), 1)
FROM productos UNION ALL
SELECT 'ficha_tecnica', ROUND(AVG(LENGTH(ficha_tecnica)), 1)
FROM productos UNION ALL
SELECT 'imagen_url', ROUND(AVG(LENGTH(imagen_url)), 1)
FROM productos;
EOF

```

Registrar los resultados de los tres bloques. Estos datos son la evidencia cuantitativa que respalda cada decisión de diseño en los pasos siguientes.

D.3 Fragmentación horizontal — definición y verificación de correctitud (VM)

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- =====
-- DEFINICIÓN DE FRAGMENTOS HORIZONTALES
-- =====
-- Atributo de fragmentación: region ENUM('norte','sur','este','oeste')
-- Número de fragmentos: 2 (limitado a los nodos de sharding disponibles)
--
-- frag_A → bdd-nodo04 (192.168.56.104): region IN ('norte', 'este')
-- frag_B → bdd-nodo05 (192.168.56.105): region IN ('sur', 'oeste')
--
-- Justificación del agrupamiento: 5 clientes por valor de región
-- (datos uniformes) → ambos fragmentos tendrán 10 filas de clientes,
-- produciendo un balanceo de carga perfecto.
-- =====
-- ----- Vista de cada fragmento -----
SELECT 'clientes_frag_A (norte+este)' AS fragmento,
       COUNT(*)                      AS filas,
       GROUP_CONCAT(DISTINCT region) AS regiones_incluidas
FROM clientes WHERE region IN ('norte', 'este');
SELECT 'clientes_frag_B (sur+oeste)' AS fragmento,
       COUNT(*)                      AS filas,
       GROUP_CONCAT(DISTINCT region) AS regiones_incluidas
FROM clientes WHERE region IN ('sur', 'oeste');
-- ----- VERIFICACIÓN DE CORRECTITUD - CLIENTES -----
-- 1. COMPLETITUD: suma de fragmentos debe igualar el total global
SELECT 'Compleitud clientes'          AS condicion,
       (SELECT COUNT(*) FROM clientes) AS total_global,
       (SELECT COUNT(*) FROM clientes WHERE region IN ('norte','este')) +
       (SELECT COUNT(*) FROM clientes WHERE region IN ('sur','oeste'))
AS suma_fragmentos,
CASE
  WHEN (SELECT COUNT(*) FROM clientes) =
       (SELECT COUNT(*) FROM clientes WHERE region IN
('norte','este')) +
       (SELECT COUNT(*) FROM clientes WHERE region
IN ('sur','oeste'))
  THEN 'OK' ELSE 'FALLA'
END AS resultado;
-- 2. DISJUNCIÓN: ninguna fila puede pertenecer a ambos fragmentos
-- (se verifica buscando filas cuyo valor de region satisfaga
-- simultáneamente los dos predicados; debe devolver 0)
SELECT 'Disjunción clientes'          AS condicion,
       COUNT(*)                      AS
```

```

filas_en_interseccion,
    CASE WHEN COUNT(*) = 0 THEN 'OK' ELSE 'FALLA' END AS resultado
FROM clientes
WHERE region IN ('norte','este') AND region IN ('sur','oeste');
-- 3. RECONSTRUCCIÓN: UNION ALL de fragmentos produce la relación global
SELECT 'Reconstrucción clientes'          AS condicion,
       (SELECT COUNT(*) FROM clientes) AS filas_globales,
       COUNT(*)                        AS filas_reconstruidas,
       CASE
           WHEN (SELECT COUNT(*) FROM clientes) = COUNT(*)
           THEN 'OK' ELSE 'FALLA'
       END AS resultado
FROM (
    SELECT * FROM clientes WHERE region IN ('norte', 'este')
    UNION ALL
    SELECT * FROM clientes WHERE region IN ('sur', 'oeste')
) AS reconstruccion;
-- ----- VERIFICACIÓN CONDENSADA - PEDIDOS -----
SELECT 'Compleitud pedidos'          AS condicion,
       CASE
           WHEN (SELECT COUNT(*) FROM pedidos) =
                (SELECT COUNT(*) FROM pedidos WHERE region IN
 ('norte','este')) +
                (SELECT COUNT(*) FROM pedidos WHERE region
IN ('sur','oeste'))
           THEN 'OK' ELSE 'FALLA'
       END AS resultado
UNION ALL
SELECT 'Disjunción pedidos',
       CASE WHEN
           (SELECT COUNT(*) FROM pedidos
            WHERE region IN ('norte','este') AND region IN ('sur','oeste'))
= 0
       THEN 'OK' ELSE 'FALLA' END
UNION ALL
SELECT 'Reconstrucción pedidos',
       CASE WHEN (SELECT COUNT(*) FROM pedidos) =
                (SELECT COUNT(*) FROM pedidos WHERE region IN
 ('norte','este')) +
                (SELECT COUNT(*) FROM pedidos WHERE region
IN ('sur','oeste'))
       THEN 'OK' ELSE 'FALLA' END;
-- ----- DETALLE_PEDIDOS: fragmentación derivada -----
-- detalle_pedidos no tiene columna 'region' propia; se fragmenta
-- siguiendo al pedido al que pertenece (co-localización obligatoria
-- por la FK pedido_id → pedidos.id que impide joins remotos).
SELECT 'detalle_frag_A (pedidos norte+este)' AS fragmento,
       COUNT(*)                        AS filas

```

```

FROM detalle_pedidos dp
JOIN pedidos p ON p.id = dp.pedido_id
WHERE p.region IN ('norte', 'este');
SELECT 'detalle_frag_B (pedidos sur+oeste)' AS fragmento,
COUNT(*) AS filas
FROM detalle_pedidos dp
JOIN pedidos p ON p.id = dp.pedido_id
WHERE p.region IN ('sur', 'oeste');
-- Verificación de completitud de la fragmentación derivada
SELECT 'Completitud detalle_pedidos' AS
condicion,
(SELECT COUNT(*) FROM detalle_pedidos)
AS total_global,
(SELECT COUNT(*) FROM detalle_pedidos dp
JOIN pedidos p ON p.id = dp.pedido_id
WHERE p.region IN ('norte', 'este')) +
(SELECT COUNT(*) FROM detalle_pedidos dp
JOIN pedidos p ON p.id = dp.pedido_id
WHERE p.region IN ('sur', 'oeste')) AS
suma_fragmentos,
CASE
WHEN (SELECT COUNT(*) FROM detalle_pedidos) =
(SELECT COUNT(*) FROM detalle_pedidos dp
JOIN pedidos p ON p.id = dp.pedido_id
WHERE p.region IN ('norte', 'este')) +
(SELECT COUNT(*) FROM detalle_pedidos dp
JOIN pedidos p ON p.id = dp.pedido_id
WHERE p.region IN ('sur', 'oeste'))
THEN 'OK' ELSE 'FALLA'
END AS resultado;
EOF

```

D.4 Fragmentación vertical de **productos** — definición y verificación (VM)

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- =====
-- DEFINICIÓN DE FRAGMENTOS VERTICALES
-- =====
-- Tabla fuente: productos (10 columnas de datos + PK 'id')
--
-- Criterio de agrupación: patrón de uso esperado
-- COLUMNAS OPERACIONALES (consulta frecuente):
--     id*, sku, nombre, categoria, precio, stock, fecha_creacion
--     → aparecen en listados, búsquedas y cálculos de precios
--     → Fragmento V_basico → bdd-nodo04 (192.168.56.104)
--
-- COLUMNAS DE DETALLE (consulta poco frecuente, volumen alto):
--     id*, sku, descripcion, ficha_tecnica, imagen_url, peso_kg
--     → aparecen solo al consultar la ficha completa de un producto
--     → Columnas TEXT: promedio de 60-170 bytes/fila (análisis D.2)
--     → Fragmento V_detalle → bdd-nodo05 (192.168.56.105)
--
-- Nota: 'id' (PK) se duplica en ambos fragmentos para reconstrucción
--       por JOIN. 'sku' (clave alternativa) también se duplica por
--       ser referencia natural de negocio usada en ambos contextos;
--       esto es una decisión de diseño deliberada, no una violación
--       de la disjunción (que aplica a columnas NO CLAVE).
-- =====
-- Vista previa del fragmento básico (7 columnas)
SELECT id, sku, nombre, categoria, precio, stock, fecha_creacion
FROM   productos
ORDER BY id;
-- Vista previa del fragmento detalle (6 columnas)
SELECT id, sku,
       SUBSTRING(descripcion, 1, 45) AS descripcion_preview,
       SUBSTRING(ficha_tecnica, 1, 45) AS ficha_preview,
       imagen_url,
       peso_kg
FROM   productos
ORDER BY id;
-- ---- VERIFICACIÓN DE CORRECTITUD — VERTICAL ----
-- 1. COMPLETITUD: todos los productos deben estar en ambos fragmentos
SELECT 'Completitud V_basico' AS condicion,
       (SELECT COUNT(*) FROM productos) AS total_global,
       COUNT(*) AS
filas_en_fragmento,
       CASE WHEN COUNT(*) = (SELECT COUNT(*) FROM productos)
           THEN 'OK' ELSE 'FALLA' END AS resultado
```

```

FROM (SELECT id, sku, nombre, categoria, precio, stock, fecha_creacion
      FROM productos) AS frag_basico
UNION ALL
SELECT 'Completitud V_detalle',
      (SELECT COUNT(*) FROM productos),
      COUNT(*),
      CASE WHEN COUNT(*) = (SELECT COUNT(*) FROM productos)
            THEN 'OK' ELSE 'FALLA' END
FROM (SELECT id, sku, descripcion, ficha_tecnica, imagen_url, peso_kg
      FROM productos) AS frag_detalle;
-- 2. DISYUNCIÓN DE COLUMNAS NO CLAVE
--   V_basico  (no clave): nombre, categoria, precio,
--   stock, fecha_creacion
--   V_detalle (no clave): descripcion, ficha_tecnica,
--   imagen_url, peso_kg
--   Intersección = vacío → condición cumplida
SELECT 'Disyunción de columnas no clave' AS condicion,
      'nombre, categoria, precio, stock, fecha_creacion'
AS cols_V_basico,
      'descripcion, ficha_tecnica, imagen_url, peso_kg'
AS cols_V_detalle,
      'Sin superposición (OK)' AS resultado;
-- 3. RECONSTRUCCIÓN: JOIN por PK reconstituye la relación completa
SELECT 'Reconstrucción productos (JOIN en id)' AS condicion,
      (SELECT COUNT(*) FROM productos) AS filas_globales,
      COUNT(*) AS
filas_reconstruidas,
      CASE WHEN COUNT(*) = (SELECT COUNT(*) FROM productos)
            THEN 'OK' ELSE 'FALLA' END AS resultado
FROM (
      SELECT b.id, b.sku, b.nombre, b.categoria, b.precio, b.stock,
            d.descripcion, d.ficha_tecnica, d.imagen_url, d.peso_kg,
            b.fecha_creacion
      FROM (SELECT id, sku, nombre, categoria, precio,
stock, fecha_creacion
            FROM productos) b
      JOIN (SELECT id, sku, descripcion, ficha_tecnica, imagen_url, peso_kg
            FROM productos) d ON b.id = d.id
) AS reconstruccion;
-- Verificación visual: primer registro reconstruido vs. original
SELECT 'Original' AS origen, id, sku, nombre, precio,
      SUBSTRING(descripcion, 1, 35) AS desc_preview
FROM productos WHERE id = 1
UNION ALL
SELECT 'Reconstruido' AS origen, b.id, b.sku, b.nombre, b.precio,
      SUBSTRING(d.descripcion, 1, 35)
FROM (SELECT id, sku, nombre, precio FROM productos WHERE id = 1) b
JOIN (SELECT id, descripcion FROM productos WHERE id = 1) d ON

```

```
b.id = d.id;  
EOF
```

D.5 Simulación de consultas distribuidas en el entorno centralizado (VM)

*Estas consultas representan las operaciones que **bdd-nodo06 (coordinador)** ejecutará en las Fases 14–16 mediante el motor Spider. Ejecutarlas ahora — cuando todos los datos conviven en **bdd-nodo01** — permite verificar que los resultados esperados son correctos antes de dispersar los datos entre nodos.*

```

sudo mariadb lab_bdd << 'EOF'
-- =====
-- CONSULTA DISTRIBUIDA 1 – Pedidos con datos de cliente
-- Fragmentos: clientes_frag_A + pedidos_frag_A (nodo04)
--             clientes_frag_B + pedidos_frag_B (nodo05)
-- El coordinador envía la subconsulta a cada shard y hace
-- UNION ALL del resultado. No hay transferencia de datos entre
-- nodo04 y nodo05 porque los fragmentos de clientes y pedidos
-- están co-localizados por región.
-- =====
SELECT c.region,
       CONCAT(c.nombre, ' ', c.apellido) AS cliente,
       p.id                               AS pedido_id,
       p.estado,
       p.total
FROM   clientes c
JOIN   pedidos  p ON p.cliente_id = c.id
WHERE  c.region IN ('norte', 'este')      -- → subconsulta a nodo04
UNION ALL
SELECT c.region,
       CONCAT(c.nombre, ' ', c.apellido),
       p.id, p.estado, p.total
FROM   clientes c
JOIN   pedidos  p ON p.cliente_id = c.id
WHERE  c.region IN ('sur', 'oeste')       -- → subconsulta a nodo05
ORDER BY region, pedido_id;
-- =====
-- CONSULTA DISTRIBUIDA 2 – Líneas de pedido con nombre de producto
-- Fragmentos: pedidos_frag_* (nodo04/05) + V_basico (nodo04)
-- Solo se necesita el fragmento básico de productos (precio y nombre);
-- V_detalle (nodo05) no interviene → el coordinador evita un join
-- innecesario al nodo que contiene columnas TEXT.
-- =====
SELECT p.region,
       p.id                               AS pedido_id,
       pr.nombre                          AS producto,
       pr.categoria,
       dp.cantidad,
       dp.precio_unitario AS precio_vendido,
       dp.subtotal
FROM   detalle_pedidos dp
JOIN   pedidos          p  ON p.id = dp.pedido_id
JOIN   productos        pr ON pr.id = dp.producto_id
ORDER BY p.region, p.id;
-- =====
-- CONSULTA DISTRIBUIDA 3 – Ficha completa de un producto

```

```

-- Fragmentos: V_basico (nodo04) + V_detalle (nodo05)
-- El coordinador hace JOIN entre los dos fragmentos verticales
-- para reconstruir la fila completa. Solo se activa cuando el
-- usuario necesita ver la descripción y ficha técnica.
-- =====
SELECT b.id, b.sku, b.nombre, b.categoria,
       b.precio, b.stock,
       d.descripcion,
       d.ficha_tecnica,
       d.peso_kg
FROM   (SELECT id, sku, nombre, categoria, precio, stock
        FROM productos WHERE id = 1) b
JOIN   (SELECT id, descripcion, ficha_tecnica, peso_kg
        FROM productos WHERE id = 1) d ON b.id = d.id;
-- =====

-- CONSULTA DISTRIBUIDA 4 – Híbrida: ventas por región con
-- categorías de productos (cruza fragmentación horizontal de
-- clientes/pedidos y fragmentación vertical de productos)
-- =====
SELECT c.region,
       COUNT(DISTINCT c.id)           AS clientes_activos,
       COUNT(DISTINCT p.id)           AS pedidos,
       GROUP_CONCAT(DISTINCT pr.categoria
                    ORDER BY pr.categoria) AS categorias_compradas,
       ROUND(SUM(dp.subtotal), 2)      AS facturacion_total
FROM   clientes      c
JOIN   pedidos        p  ON  p.cliente_id = c.id
JOIN   detalle_pedidos dp ON dp.pedido_id = p.id
JOIN   productos      pr ON pr.id = dp.producto_id
GROUP BY c.region
ORDER BY facturacion_total DESC;
-- =====

-- RESUMEN: métricas que el diseño debe conservar íntegras
-- =====
SELECT 'clientes total'           AS metrica, COUNT(*) AS valor FROM clientes
UNION ALL
SELECT 'clientes frag_A',         COUNT(*)           FROM clientes
WHERE region IN ('norte','este') UNION ALL
SELECT 'clientes frag_B',         COUNT(*)           FROM clientes
WHERE region IN ('sur','oeste')  UNION ALL
SELECT 'pedidos total',           COUNT(*)           FROM pedidos
UNION ALL
SELECT 'pedidos frag_A',          COUNT(*)           FROM pedidos
WHERE region IN ('norte','este') UNION ALL
SELECT 'pedidos frag_B',          COUNT(*)           FROM pedidos
WHERE region IN ('sur','oeste')  UNION ALL
SELECT 'detalle total',           COUNT(*)           FROM
detalle_pedidos UNION ALL

```

```
SELECT 'productos total (V_basico)', COUNT(*) FROM productos
UNION ALL
SELECT 'productos total (V_detalle)', COUNT(*)
FROM productos;
EOF
```

D.6 Diagrama de topología de replicación (VM — referencia para el DDD)

 Bash

```
cat > /tmp/topologia_lab_bdd.txt << 'EOF'
=====
TOPOLOGÍA COMPLETA DEL LABORATORIO BDD — REFERENCIA FASE 9
=====
REPLICACIÓN FÍSICA — Fase 10


---


bdd-nodo01 [MAESTRO | 192.168.56.101]
  lab_bdd (esquema completo)
  Binary log activo, GTID habilitado
  Acepta lecturas Y escrituras
    |
    | binlog stream (row-based, GTID)
    ▼
bdd-nodo02 [ESCLAVO | 192.168.56.102]
  lab_bdd (réplica completa, solo lectura)
  read_only = ON
REPLICACIÓN LÓGICA / MULTI-MAESTRO — Fase 11


---


bdd-nodo03 [MULTI-MAESTRO | 192.168.56.103]
  lab_bdd (réplica completa)
  Acepta lecturas Y escrituras
  Replicación bidireccional con nodo01
FRAGMENTACIÓN HORIZONTAL — Fase 13


---


bdd-nodo04 [SHARD-A | 192.168.56.104]
  clientes      WHERE region IN ('norte','este') → 10 filas
  pedidos      WHERE region IN ('norte','este') → 10 filas
  detalle_pedidos co-localizado con pedidos_frag_A → ~18 filas
bdd-nodo05 [SHARD-B | 192.168.56.105]
  clientes      WHERE region IN ('sur','oeste') → 10 filas
  pedidos      WHERE region IN ('sur','oeste') → 10 filas
  detalle_pedidos co-localizado con pedidos_frag_B → ~18 filas
FRAGMENTACIÓN VERTICAL — Fase 14


---


bdd-nodo04: V_productos_basico
  Columnas: id*, sku, nombre, categoria, precio, stock, fecha_creacion
bdd-nodo05: V_productos_detalle
  Columnas: id*, sku, descripcion, ficha_tecnica, imagen_url, peso_kg
COORDINADOR DE CONSULTAS DISTRIBUIDAS — Fases 14-16


---


bdd-nodo06 [COORDINADOR | 192.168.56.106]
  Motor Spider habilitado
  Tablas tipo SPIDER apuntando a nodo04 y nodo05
  Recibe consultas globales del cliente, las descompone
```

```
y combina resultados (no almacena datos de lab_bdd)
CLIENTE EXTERNO (opcional) – Fases 16-17
```

```
bdd-cliente [192.168.56.107]
  Conecta solo a nodo06 (ignora la topología interna)
  Prueba de transparencia de distribución
=====
EOF
cat /tmp/topologia_lab_bdd.txt
```

D.7 Matriz de asignación completa (VM — visualización SQL)

 Bash

```
sudo mariadb << 'EOF'
-- Tabla virtual de referencia: matriz de asignación del laboratorio
SELECT fragmento, tabla_origen, tipo_frag,
       predicado_o_columnas, nodo_destino, ip_destino, fase
FROM (
  SELECT 'lab_bdd_completa (maestro)' AS fragmento,
         'todas' AS tabla_origen,
         'REPLICADA' AS tipo_frag,
         'esquema completo' AS predicado_o_columnas,
         'bdd-nodo01' AS nodo_destino,
         '192.168.56.101' AS ip_destino,
         '10' AS fase
  UNION ALL SELECT 'lab_bdd_completa (esclavo)', 'todas', 'REPLICADA',
                  'esquema completo', 'bdd-nodo02', '192.168.56.102', '10'
  UNION ALL SELECT 'lab_bdd_multimaestro', 'todas', 'REPLICADA',
                  'esquema completo', 'bdd-nodo03', '192.168.56.103', '11'
  UNION ALL SELECT 'clientes_frag_A', 'clientes', 'HORIZONTAL',
                  "region IN ('norte', 'este')", 'bdd-nodo04', '192.168.56.104', '13'
  UNION ALL SELECT 'clientes_frag_B', 'clientes', 'HORIZONTAL',
                  "region IN ('sur', 'oeste')", 'bdd-nodo05', '192.168.56.105', '13'
  UNION ALL SELECT 'pedidos_frag_A', 'pedidos', 'HORIZONTAL',
                  "region IN ('norte', 'este')", 'bdd-nodo04', '192.168.56.104', '13'
  UNION ALL SELECT 'pedidos_frag_B', 'pedidos', 'HORIZONTAL',
                  "region IN ('sur', 'oeste')", 'bdd-nodo05', '192.168.56.105', '13'
  UNION ALL SELECT 'detalle_frag_A', 'detalle_pedidos', 'HORIZONTAL
(dervivada)',
                  'JOIN pedidos frag_A', 'bdd-nodo04', '192.168.56.104', '13'
  UNION ALL SELECT 'detalle_frag_B', 'detalle_pedidos', 'HORIZONTAL
(dervivada)',
                  'JOIN pedidos frag_B', 'bdd-nodo05', '192.168.56.105', '13'
  UNION ALL SELECT 'V_productos_basico', 'productos', 'VERTICAL',
                  'id,sku,nombre,categoria,precio,stock,fecha_crea', 'bdd-
nodo04', '192.168.56.104', '14'
  UNION ALL SELECT 'V_productos_detalle', 'productos', 'VERTICAL',
                  'id,sku,descripcion,ficha_tecnica,imagen_url,peso', 'bdd-
nodo05', '192.168.56.105', '14'
) AS matriz
ORDER BY fase, tipo_frag, tabla_origen;
EOF
```

D.8 Crear el Documento de Diseño Distribuido en el host (host — PowerShell)

PowerShell

```
$ddd = @"
# Documento de Diseño Distribuido — Laboratorio BDD
Generado en la Fase 9. Referencia técnica obligatoria para las Fases 10-16.
Fecha de generación: $(Get-Date -Format 'yyyy-MM-dd HH:mm')
---
## 1. Esquema global
Base de datos : lab_bdd
Nodo de origen: bdd-nodo01 (192.168.56.101)
| Tabla                | Filas | Motor   | Atributo de distribución          |
|-----|-----|-----|-----|
| clientes              | 20    | InnoDB | region ENUM (fragmentación horiz.) |
| productos             | 10    | InnoDB | columnas (fragmentación vertical)  |
| pedidos               | 20    | InnoDB | region ENUM (fragmentación horiz.) |
| detalle_pedidos      | 36    | InnoDB | derivada de pedidos (co-localizada) |
---
## 2. Plan de replicación
### Fase 10 — Replicación física (maestro-esclavo)
| Nodo   | IP                | Rol      | Modo de acceso          |
|-----|-----|-----|-----|
| nodo01 | 192.168.56.101    | MAESTRO  | Lectura + escritura     |
| nodo02 | 192.168.56.102    | ESCLAVO  | Solo lectura (RO)       |
Método: binary log basado en GTID, replicación row-based.
### Fase 11 — Replicación lógica
| Nodo   | IP                | Rol      |
|-----|-----|-----|
| nodo03 | 192.168.56.103    | MULTI-MAESTRO |
---
## 3. Plan de fragmentación horizontal (Fase 13)
Atributo de fragmentación: region
| Fragmento            | Predicado                                | Nodo   | IP |
| Filas |
|-----|-----|-----|-----|
|-----|-----|-----|-----|
| clientes_frag_A      | region IN ('norte','este')              | nodo04 |   |
192.168.56.104 | 10 |
| clientes_frag_B      | region IN ('sur','oeste')                | nodo05 |   |
192.168.56.105 | 10 |
| pedidos_frag_A       | region IN ('norte','este')              | nodo04 |   |
192.168.56.104 | 10 |
| pedidos_frag_B       | region IN ('sur','oeste')                | nodo05 |   |
192.168.56.105 | 10 |
| detalle_frag_A       | JOIN pedidos frag_A                     | nodo04 |   |
192.168.56.104 | ~18 |

```

```

| detalle_frag_B      | JOIN pedidos frag_B      | nodo05 |
192.168.56.105 | ~18 |
Correctitud verificada: completitud OK, disjunción OK, reconstrucción OK.
---
## 4. Plan de fragmentación vertical (Fase 14)
Tabla fuente: productos
| Fragmento          | Columnas          |
Nodo | IP          |
|-----|-----|
| V_productos_basico | id*, sku, nombre, categoria, precio, stock, fecha |
nodo04 | 192.168.56.104 |
| V_productos_detalle | id*, sku, descripcion, ficha_tecnica, imagen, peso |
nodo05 | 192.168.56.105 |
Clave de reconstrucción: JOIN en columna 'id'.
Correctitud verificada: completitud OK, disjunción de columnas OK,
reconstrucción OK.
---
## 5. Coordinador de consultas distribuidas (Fases 14-16)
Nodo      : bdd-nodo06 (192.168.56.106)
Tecnología: Motor Spider de MariaDB
Rol       : recibir consultas globales, descomponerlas en subconsultas
            dirigidas a nodo04/nodo05 y combinar resultados.
---
## 6. Transparencia de distribución objetivo
| Tipo de transparencia | Mecanismo          | Implementada
en |
|-----|-----|-----|
| Replicación          | Maestro-esclavo transparente | Fases 10-
11 |
| Fragmentación        | Spider oculta la partición    | Fase
14+ |
| Ubicación            | Spider oculta el nodo físico  | Fase
14+ |
| Concurrencia         | InnoDB MVCC + pruebas de fallo | Fase
17+ |
---
## 7. Historial de snapshots del laboratorio
| Snapshot          | Estado en ese punto          |
|-----|-----|
| fase05-completa   | Ubuntu Server instalado, SSH activo |
| fase06-completa   | IP estática configurada, SSH verificado |
| fase07-completa   | MariaDB 10.11 instalado, utf8mb4 activo |
| fase08-completa   | lab_bdd creada y poblada, usuarios, slow log |
| fase09-completa   | Diseño distribuido definido y validado |
"@
$ruta = "C:\LabBDD\Documentacion\fase09-disenyo-distribuido.md"

```

```
$ddd | Out-File -FilePath $ruta -Encoding UTF8
Write-Host "DDD creado en: $ruta"
# Verificar
Get-Item $ruta | Select-Object Name, Length, LastWriteTime
```

D.9 Apagar la VM y tomar los snapshots de cierre de fase (host)

Desde la sesión SSH abierta en `bdd-nodo01`:

```
Bash

sudo poweroff
```

Confirmar desde el host que la VM se detuvo:

```
PowerShell

VBoxManage list runningvms
```

La salida debe estar vacía. Tomar los snapshots:

```
PowerShell

VBoxManage snapshot "bdd-nodo01" take "fase09-completa" `
  --description "Diseño distribuido validado: frag. horizontal
(clientes/pedidos/detalle), vertical (productos). DDD generado
en Documentacion/"
VBoxManage snapshot "bdd-nodo02" take "fase09-completa" `
  --description "Sin cambios respecto a fase08-completa. Snapshot de hito de
diseño para coherencia de numeración."
```

Nota: `bdd-nodo02` no recibió cambios en esta fase, pero se toma el snapshot para mantener la coherencia de numeración entre ambos nodos, lo que simplifica la identificación del estado de cada VM en cualquier punto del historial.

Confirmar los snapshots:

```
PowerShell

VBoxManage snapshot "bdd-nodo01" list
VBoxManage snapshot "bdd-nodo02" list
```

Cada VM debe mostrar cinco snapshots: `fase05-completa` hasta `fase09-completa`.

E. Verificación de funcionamiento

Esta fase se considera completa cuando se cumplen todos los puntos siguientes:

1. El análisis estadístico (D.2) se ejecutó sin errores y el estudiante puede interpretar los resultados: 5 clientes por región, distribución de tamaño de columnas TEXT de `productos` (descripcion y ficha_tecnica: $\approx 100-170$ bytes promedio; columnas operacionales: $\approx 10-15$ bytes).
2. Las **tres condiciones de correctitud de la fragmentación horizontal** de `clientes` (D.3) devuelven `OK`:
 - Completitud: suma de fragmentos = 20.
 - Disjunción: filas en intersección = 0.
 - Reconstrucción: UNION ALL produce 20 filas.
3. Las mismas tres condiciones se verificaron para `pedidos` (resultado `OK`) y la completitud para `detalle_pedidos` (resultado `OK`, con 36 filas en total).
4. Las **condiciones de correctitud de la fragmentación vertical** de `productos` (D.4) devuelven `OK`:
 - Completitud: los 10 productos aparecen en `v_basico` y en `v_detalle`.
 - Disjunción de columnas: confirmada (sin columnas no clave compartidas).
 - Reconstrucción: JOIN por `id` produce 10 filas con todos los atributos.
5. Las **cuatro consultas distribuidas simuladas** (D.5) devuelven resultados no vacíos y coherentes; en particular, la Consulta 4 devuelve exactamente 4 filas (una por región) con datos de facturación correctos.
6. El diagrama de topología (D.6) se generó en `/tmp/topologia_lab_bdd.txt` y el estudiante puede describir el rol de cada nodo sin leerlo.
7. La consulta de la matriz de asignación (D.7) muestra 11 filas, una por cada fragmento planificado (3 replicadas + 6 horizontales + 2 verticales).
8. El archivo `C:\LabBDD\Documentacion\fase09-disenyo-distribuido.md` existe en el host con un tamaño mayor a cero y contiene las siete secciones del diseño.
9. El estudiante puede explicar de memoria:
 - Por qué `detalle_pedidos` se co-localiza con `pedidos` (no se fragmenta independientemente).
 - Por qué la PK `id` debe estar en **ambos** fragmentos verticales.
 - Por qué `bdd-nodo06` es el coordinador y no `bdd-nodo01`.

- La diferencia entre la fragmentación híbrida “en una sola tabla” y la que emerge de combinar tablas con distintos tipos de fragmentación.

10. Los snapshots `fase09-completa` existen en `bdd-nodo01` y `bdd-nodo02`.

F. Problemas comunes y soluciones

Problema	Causa probable	Solución
La verificación de completitud devuelve <code>FALLA</code> para <code>clientes</code>	Los predicados IN no cubren todos los valores del dominio de <code>region</code>	Verificar con <code>SELECT DISTINCT region FROM clientes</code> que los cuatro valores reales son exactamente <code>norte, sur, este, oeste</code> ; asegurarse de que los dos predicados formen una partición completa del dominio
La verificación de disjunción devuelve un conteo > 0	Los predicados de los dos fragmentos comparten algún valor de <code>region</code>	Revisar que <code>{'norte','este'} ∩ {'sur','oeste'} = ∅</code> ; si se modificaron los predicados de ejemplo, confirmar que no hay valor compartido
La reconstrucción vertical cuenta menos filas que la tabla original	El JOIN entre fragmentos excluye filas (posible valor NULL en la PK)	Ejecutar <code>SELECT COUNT(*) FROM productos WHERE id IS NULL;</code> — en InnoDB la PK nunca es NULL; si el resultado es > 0, hay corrupción de datos que debe investigarse antes de continuar
La consulta distribuida simulada (D.5) devuelve 0 filas	La sesión no estaba en el contexto de <code>lab_bdd</code> al ejecutar el bloque	Asegurar que el comando inicia con <code>sudo mariadb lab_bdd</code> (base de datos especificada) o agregar <code>USE lab_bdd;</code> al inicio del bloque HERE-DOC
<code>Out-File</code> en PowerShell genera el archivo con codificación BOM o con saltos de línea incorrectos	Comportamiento de PowerShell 5.x con <code>-Encoding UTF8</code>	En PowerShell 7+ usar <code>-Encoding UTF8NoBOM</code> ; en PowerShell 5.x, el BOM no afecta la legibilidad del archivo pero puede causar problemas si se edita en ciertos programas
El estudiante no puede justificar la elección de <code>region</code> como atributo de fragmentación	No se analizaron los datos antes de tomar la decisión	Revisar el Bloque 2 del análisis estadístico (D.2): la distribución uniforme de 5 clientes por región confirma que <code>region</code> produce fragmentos balanceados; un atributo desbalanceado (ej. <code>ciudad</code>) produciría hotspots
Hay dudas sobre si duplicar <code>sku</code> en ambos fragmentos verticales viola la disjunción	Confusión entre columnas clave y no clave	La disjunción aplica solo a columnas no clave ; <code>sku</code> es una clave alternativa (UNIQUE) y puede — y conviene — aparecer en ambos fragmentos para que las aplicaciones puedan referenciar un producto sin

		necesitar un JOIN cuando solo saben el SKU. Documentarlo como decisión de diseño intencional
El snapshot de <code>bdd-nodo02</code> falla porque la VM está activa	<code>bdd-nodo02</code> quedó encendida desde una sesión de trabajo anterior	Ejecutar <code>VBoxManage list runningvms</code> ; si aparece, apagarla con <code>VBoxManage controlvm "bdd-nodo02" acpipowerbutton</code> antes de tomar el snapshot
La Consulta 4 (D.5) devuelve menos de 4 filas	Algún cliente de alguna región no tiene pedidos, o las regiones están mal escritas en <code>/etc/hosts</code> (no aplica aquí)	Verificar con <code>SELECT DISTINCT region FROM pedidos</code> que los cuatro valores existen; la Consulta 4 usa LEFT JOIN implícito vía JOIN, por lo que regiones sin pedidos no aparecerán — si se desea incluirlas, cambiar a <code>LEFT JOIN</code>

G. Checklist de validación

- ☐ Ejecuté el análisis estadístico completo (D.2) en `bdd-nodo01` y tomé nota de los resultados: distribución por región, tamaño de columnas TEXT, FKs que imponen co-localización.
- ☐ Identifiqué `region` como atributo de fragmentación horizontal de `clientes` y `pedidos`, con justificación cuantitativa (distribución uniforme, 5 filas por valor).
- ☐ Definí los predicados de los dos fragmentos: `frag_A = region IN ('norte','este')`, `frag_B = region IN ('sur','oeste')`.
- ☐ Las tres condiciones de correctitud de la fragmentación horizontal de `clientes` devuelven `OK`: completitud, disjunción y reconstrucción.
- ☐ Las mismas condiciones se verificaron para `pedidos` (OK) y la completitud para `detalle_pedidos` (OK, 36 filas).
- ☐ Definí los dos fragmentos verticales de `productos`: `v_basico` (columnas operacionales de alta frecuencia) y `v_detalle` (columnas TEXT de baja frecuencia).
- ☐ Las condiciones de correctitud de la fragmentación vertical devuelven `OK`: completitud en ambos fragmentos, disjunción de columnas no clave, reconstrucción por JOIN en `id`.
- ☐ Ejecuté las cuatro consultas distribuidas simuladas (D.5) y confirmé resultados correctos y no vacíos.
- ☐ Generé el diagrama de topología (D.6) y puedo describir el rol de cada nodo.
- ☐ Ejecuté la consulta de la matriz de asignación (D.7) y puedo explicar cada fila.
- ☐ El archivo `C:\LabBDD\Documentacion\fase09-disenyo-distribuido.md` existe y contiene las siete secciones del diseño completo.

- ☐ Apagué `bdd-nodo01` de forma ordenada y tomé el snapshot `fase09-completa` en `bdd-nodo01` y `bdd-nodo02`.
- ☐ Puedo explicar la diferencia entre fragmentación horizontal, vertical e híbrida usando ejemplos concretos del esquema `lab_bdd`.
- ☐ Puedo justificar por qué `detalle_pedidos` se co-localiza con `pedidos` en lugar de asignarse a un nodo independiente.
- ☐ Puedo explicar por qué la PK `id` debe duplicarse en los dos fragmentos verticales de `productos`.
- ☐ Puedo describir el rol de `bdd-nodo06` como coordinador y la tecnología que usará (motor Spider de MariaDB).

Preguntas teóricas para estudiantes

1. La fragmentación horizontal de `clientes` agrupa `norte+este` en un fragmento y `sur+oeste` en otro, en lugar de hacer un fragmento por cada valor de `region` (cuatro fragmentos). Explica las ventajas e inconvenientes de ambas estrategias. ¿Qué criterio debería dominar la decisión: el número de nodos disponibles, el volumen de datos por fragmento, la localidad de las consultas más frecuentes o la facilidad de mantenimiento futuro del esquema de distribución?
2. La tabla `detalle_pedidos` no tiene columna `region` propia, pero el diseño la co-localiza con los fragmentos de `pedidos`. Explica en detalle por qué esta co-localización es correcta desde el punto de vista del rendimiento de las consultas distribuidas. ¿Qué problema concreto y medible (en términos de tráfico de red y latencia) surgiría si `detalle_pedidos` se asignara a un único nodo centralizado, independientemente de la fragmentación de `pedidos`?
3. En la fragmentación vertical de `productos`, la columna `sku` aparece en **ambos** fragmentos (`v_basico` y `v_detalle`), aunque la condición de disjunción exige que las columnas no clave no se repitan. ¿Está esto mal? Justifica técnicamente por qué la duplicación de `sku` (siendo una clave alternativa, no la clave primaria) puede considerarse una decisión de diseño deliberada y correcta, y no una violación de la condición de disjunción.
4. El diseño establece que `bdd-nodo06` actuará como coordinador usando el motor Spider de MariaDB. Describe conceptualmente cómo procesaría la Consulta Distribuida 4 (de la sección D.5) en un sistema completamente distribuido: ¿cuántas subconsultas generaría el coordinador, a qué nodos se las enviaría, qué resultados intermedios recibiría y cómo los combinaría para producir la respuesta final?

5. El teorema CAP establece que ante una partición de red, el sistema debe elegir entre Consistencia y Disponibilidad. Dado el diseño completo del laboratorio (replicación en nodo01–03, fragmentación en nodo04–05, coordinador en nodo06), identifica al menos dos escenarios de fallo de red distintos y explica qué dilema CAP presentaría cada uno: ¿qué parte del sistema elegiría Consistencia y qué parte elegiría Disponibilidad?
-

Ejercicios prácticos

1. **Script de validación automática de correctitud.**

Escribir un único script `.sql` que verifique las tres condiciones (completitud, disjunción, reconstrucción) para las **cuatro** tablas incluidas en el plan de fragmentación, de forma automática. El script debe producir una tabla con columnas `condicion`, `tabla` y `resultado` (`OK` / `FALLA`). Ejecutarlo y documentar la salida completa. Modificar deliberadamente un predicado para introducir una violación de disjunción o completitud, observar cómo el script la detecta, y restaurar el predicado correcto.

2. **Propuesta de fragmentación alternativa de cuatro fragmentos.**

Proponer un diseño de fragmentación horizontal para `clientes` que use **cuatro fragmentos** (uno por valor de `region`), pensado para un escenario futuro con cuatro nodos de sharding (nodo04 a nodo07). Reescribir los predicados, identificar el nodo destino de cada uno, y ejecutar el equivalente del bloque D.3 para verificar la correctitud del diseño alternativo. Discutir las ventajas y desventajas frente al diseño de dos fragmentos actual.

3. **Análisis del grado de localidad de cada consulta simulada.**

Para cada una de las cuatro consultas distribuidas de D.5, indicar: (a) qué fragmentos necesita el coordinador, (b) si la consulta se puede resolver accediendo a un solo nodo o requiere múltiples nodos, (c) cuál de las cuatro consultas tiene el mayor grado de localidad (menor número de nodos involucrados) y cuál el menor, y (d) qué volumen estimado de datos intermedios (filas × bytes) debería transferirse entre nodos para resolver cada consulta. Apoyarse en las estadísticas del análisis D.2.

Reto adicional para alumnos avanzados

Ampliar el plan de fragmentación para incluir a `productos` también en la fragmentación **horizontal**, no solo en la vertical. Dado que `productos` no tiene una columna `region`, proponer la adición de un atributo de partición (por ejemplo, `tipo_producto ENUM('hardware','mobiliario','accesorios')` u otro atributo de negocio

que se pueda derivar del campo `categoria` existente), justificar la modificación del DDL, redefinir la estrategia híbrida que combine la fragmentación vertical ya diseñada con esta nueva fragmentación horizontal, y verificar las condiciones de correctitud del esquema híbrido resultante. Incluir también el análisis de integridad referencial: si `detalle_pedidos` tiene FK hacia `productos`, ¿cómo se gestionaría esa integridad referencial en un sistema verdaderamente distribuido donde las FK no pueden cruzar nodos? ¿Qué opciones ofrece MariaDB Spider para este escenario?

Criterios de evaluación para el profesor

Criterio	Peso	Indicador de logro
Análisis estadístico del esquema	10%	Se ejecutaron los tres bloques de D.2 y el estudiante interpreta correctamente la distribución de filas por región y el perfil de tamaño de las columnas TEXT; puede relacionar cada métrica con una decisión de diseño
Fragmentación horizontal: definición y correctitud	25%	Los predicados de <code>frag_A</code> y <code>frag_B</code> están bien definidos para las tres tablas involucradas; las tres condiciones de correctitud devuelven <code>OK</code> con la evidencia SQL; el estudiante puede explicar qué significaría un resultado <code>FALLA</code> en cada condición
Fragmentación vertical: definición y correctitud	20%	Los dos fragmentos de <code>productos</code> están bien definidos con justificación de afinidad de columnas; las condiciones de correctitud (incluida la reconstrucción por JOIN) devuelven <code>OK</code> ; el estudiante puede explicar el rol de la PK duplicada
Consultas distribuidas simuladas	20%	Las cuatro consultas de D.5 producen resultados correctos y no vacíos; el estudiante puede identificar qué fragmentos consultaría el coordinador para cada una y estimar el número de nodos involucrados
Documento de Diseño Distribuido	15%	El archivo <code>fase09-disenyo-distribuido.md</code> existe, es internamente coherente y cubre las siete secciones requeridas; podrá usarse como referencia sin ambigüedad en las Fases 10–16
Comprensión conceptual (preguntas teóricas)	10%	Las respuestas a las cinco preguntas usan vocabulario técnico correcto (predicado, minterm, completitud, disjunción, co-localización, coordinador, CAP) y hacen referencia explícita al esquema específico del laboratorio

Preparación para la siguiente fase

La Fase 10: Replicación Física (Maestro-Esclavo) requerirá:

- El esquema `lab_bdd` idéntico y consistente en `bdd-nodo01` y `bdd-nodo02` (Fase 8).
- Snapshot `fase09-completa` tomado en ambas VMs (esta fase).
- Acceso SSH funcional desde el host hacia ambos nodos (Fase 6).
- El Documento de Diseño Distribuido disponible en `C:\LabBDD\Documentacion\fase09-disenyo-distribuido.md` como referencia: la sección 2 define que `nodo01` es el maestro y `nodo02` el esclavo, con binary log basado en GTID y replicación row-based.
- Ningún nodo adicional todavía: `bdd-nodo03`, `bdd-nodo04`, `bdd-nodo05` y `bdd-nodo06` no existen aún como VMs. Se crearán en sus respectivas fases.

En la Fase 10 se modificará la configuración de MariaDB en `bdd-nodo01` (habilitando el binary log, asignando un `server-id` único y configurando GTID) y en `bdd-nodo02` (configurando el agente de replicación esclavo y el `server-id` propio), y se verificará que los cambios escritos en el maestro se propagan automáticamente al esclavo en tiempo real.

Fase 10 — Replicación Física (Maestro-Esclavo)

Continuación directa de la Fase 9. `bdd-nodo01` y `bdd-nodo02` tienen MariaDB 10.11 instalado con el esquema `lab_bdd` idéntico y poblado en ambos nodos, y el snapshot `fase09-completa` está tomado. Esta fase convierte `bdd-nodo01` en el **MAESTRO** de replicación y `bdd-nodo02` en su **ESCLAVO**: cualquier escritura en el maestro se propagará automáticamente al esclavo en tiempo real usando binary log en formato ROW con GTID de MariaDB. Al finalizar, el laboratorio tendrá su primera capacidad de alta disponibilidad operativa: los datos del maestro quedan preservados —y accesibles en modo lectura— en el esclavo de forma continua.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Explicar el papel del binary log en la replicación física y las diferencias entre los formatos STATEMENT, ROW y MIXED, justificando por qué se elige ROW.
2. Describir los dos hilos del esclavo (I/O thread y SQL thread) y la responsabilidad individual de cada uno en el proceso de replicación.
3. Distinguir la replicación basada en posición de archivo (file+pos) de la basada en GTID, y articular por qué GTID simplifica la administración del esclavo.
4. Configurar el binary log con GTID en el nodo maestro mediante un archivo de configuración dedicado, sin alterar `50-server.cnf` más allá del ajuste de `bind-address`.
5. Crear un usuario de replicación aplicando el principio de mínimo privilegio (`REPLICATION SLAVE ON *.*` restringido a la IP exacta del esclavo).
6. Establecer el enlace maestro-esclavo con `CHANGE MASTER TO ... MASTER_USE_GTID = slave_pos` y verificar que ambos hilos quedan en estado `Yes`.
7. Verificar experimentalmente la propagación en tiempo real de INSERT, UPDATE, DELETE y ALTER TABLE del maestro al esclavo.
8. Comprobar que `read_only = ON` bloquea escrituras directas de usuarios no privilegiados en el esclavo.
9. Interpretar los campos clave de `SHOW SLAVE STATUS` y usar `Seconds_Behind_Master` como indicador de lag de replicación.
10. Cerrar la fase con el snapshot `fase10-completa` en ambos nodos.

B. Conceptos teóricos necesarios

1. El binary log y su rol en la replicación.

El binary log (binlog) es un archivo de registro secuencial que MariaDB escribe en el maestro cada vez que se modifica el estado de los datos: DML (`INSERT` , `UPDATE` , `DELETE`) y DDL (`CREATE TABLE` , `ALTER TABLE` , `DROP DATABASE` , etc.). Las consultas de solo lectura (`SELECT`) no se registran. Los archivos se numeran progresivamente (`mariadb-bin.000001` , `.000002` , ...) y un archivo índice (`mariadb-bin.index`) los rastrea. El parámetro `expire_logs_days` controla cuántos días se conservan antes de purgarse.

El binlog tiene tres propósitos principales en un entorno de producción:

- **Replicación:** el esclavo lee sus eventos para reproducirlos localmente.
- **Recuperación punto-en-tiempo** (point-in-time recovery): restaurar una copia de seguridad y aplicar los eventos del binlog hasta un instante preciso.
- **Auditoría de cambios:** reconstrucción forense de qué cambió y cuándo.

2. Formatos del binary log.

Formato	Descripción	Ventajas	Inconvenientes
STATEMENT	Registra la sentencia SQL tal cual fue ejecutada.	Compacto, legible con <code>mysqlbinlog</code> .	Inseguro con funciones no deterministas (<code>NOW()</code> , <code>UUID()</code> , <code>RAND()</code>): el esclavo puede obtener valores diferentes al ejecutar la misma sentencia.
ROW	Registra los valores concretos (imagen antes y después) de cada fila afectada.	Completamente determinista; correcto para cualquier sentencia.	Mayor volumen en operaciones masivas (ej. <code>UPDATE ... WHERE 1</code> sin índice).
MIXED	Usa STATEMENT por defecto; cambia a ROW automáticamente ante sentencias no deterministas.	Balance entre los dos anteriores.	Menos predecible en tamaño del log; no recomendado para GTID.

Este laboratorio usa `ROW` exclusivamente, que es el estándar recomendado para producción, el único seguro con GTID y el requerido por las topologías multi-maestro de la Fase 11.

3. Los dos hilos del esclavo.

La replicación funciona con dos hilos independientes que corren permanentemente en el proceso MariaDB del esclavo:

- **Hilo I/O** (*IO thread*): actúa como cliente MariaDB. Se conecta al maestro usando el usuario de replicación, solicita los eventos del binlog a partir de la última posición GTID conocida, los descarga y los almacena en el **relay log** local del esclavo. Su estado se reporta en `SHOW SLAVE STATUS` como `Slave_IO_Running`.
- **Hilo SQL** (*SQL thread*): lee los eventos del relay log (ya locales, sin depender de la red) y los aplica al motor del esclavo, exactamente en el orden en que fueron generados en el maestro. Su estado se reporta como `Slave_SQL_Running`. La diferencia

entre el último evento descargado y el último aplicado determina el lag de replicación (`Seconds_Behind_Master`).

Esta arquitectura de dos hilos desacopla la descarga de la aplicación: si el hilo SQL falla por un error de datos, el hilo I/O puede seguir descargando eventos y el relay log acumula las transacciones pendientes para cuando se corrija el error.

4. GTID en MariaDB.

GTID (*Global Transaction Identifier*) es un identificador único que MariaDB asigna a cada transacción comprometida cuando el binary log está activo. En MariaDB (a diferencia de MySQL), el formato es:

Text

```
domain_id - server_id - sequence_number
```

Por ejemplo, `1-1-42` representa la transacción número 42 generada por el servidor con `server_id = 1` en el dominio de replicación `1`. Variables clave:

Variable	Nodo	Significado
<code>server_id</code>	Ambos	Identificador único del nodo (obligatorio; dos nodos con el mismo valor se rechazan).
<code>gtid_domain_id</code>	Ambos	Dominio de replicación; todos los nodos del laboratorio usan <code>1</code> .
<code>gtid_binlog_pos</code>	Maestro	Último GTID escrito en el binlog del maestro.
<code>gtid_slave_pos</code>	Esclavo	Último GTID aplicado por el esclavo (persistido en <code>mysql.gtid_slave_pos</code>).
<code>log_slave_updates</code>	Ambos	<code>ON</code> = el esclavo también escribe en su propio binlog los eventos recibidos del maestro; necesario para cadenas A→B→C y para failover.

Con `MASTER_USE_GTID = slave_pos`, el esclavo comunica al maestro su `gtid_slave_pos` y el maestro le envía exactamente los eventos que el esclavo todavía no ha aplicado, sin necesidad de conocer el nombre del archivo binlog ni la posición en bytes.

5. El relay log.

El relay log es el almacén local del esclavo para los eventos descargados del maestro. Usa el mismo formato binario que el binlog. Una vez que el hilo SQL aplica un evento, MariaDB puede purgarlo automáticamente (`relay_log_purge = ON`). El relay log también se divide en archivos numerados y tiene su propio archivo índice.

6. `read_only` en el esclavo.

El parámetro `read_only = ON` impide que usuarios sin el privilegio `SUPER` ejecuten DML o DDL directamente en el esclavo. Esto preserva la consistencia con el maestro: si un usuario externo modifica datos directamente en el esclavo, crea una divergencia que el binlog no conoce y que puede provocar errores en el hilo SQL cuando llegue la misma transacción del maestro (ej. `Duplicate entry` o `Row doesn't exist`). El hilo SQL del esclavo opera como `system user` interno, que está exento de `read_only` .

7. Asincronismo y su implicación en la consistencia.

La replicación clásica maestro-esclavo es **asíncrona**: el maestro confirma la transacción al cliente tan pronto como la escribe en su binlog local, sin esperar que ningún esclavo la haya descargado ni aplicado. Ventaja: menor latencia para el cliente. Riesgo: si el maestro cae justo después de confirmar y antes de que el esclavo descargue el evento, ese evento se pierde en el esclavo (*replication lag = data loss window*). La replicación semi-síncrona (plugin `rpl_semi_sync`) mitiga esto obligando al maestro a esperar el ACK de al menos un esclavo antes de confirmar; se introduce aquí como concepto pero su implementación corresponde a fases avanzadas.

C. Procedimiento paso a paso

Paso 1 — Iniciar ambas VMs y conectarse por SSH.

Paso 2 — Modificar `bind-address` en el maestro (nodo01).

El esclavo conecta al puerto 3306 del maestro a través de la red Host-Only. La configuración actual `bind-address = 127.0.0.1` impide conexiones desde otras IPs. Se cambia a `0.0.0.0` (todas las interfaces), lo cual es seguro en la red Host-Only aislada de este laboratorio.

Paso 3 — Crear el archivo de configuración de replicación del maestro.

Se crea `/etc/mysql/mariadb.conf.d/60-replication-master.cnf` con los parámetros de binary log, GTID y `server_id` . Al usar prefijo `60` , queda cargado después de `50-server.cnf` (cuya configuración base se conserva intacta) y antes de `99-lab-logs.cnf` .

Paso 4 — Reiniciar MariaDB en el maestro y verificar que el binlog quedó activo.

Paso 5 — Crear el usuario de replicación en el maestro.

Se crea `repl_user@'192.168.56.102'` con el privilegio mínimo `REPLICATION SLAVE`. Restringir el host a la IP exacta del esclavo es esencial: un usuario `repl_user@'%'` permitiría que cualquier máquina de la red se hiciera pasar por el esclavo.

Paso 6 — Registrar la posición GTID del maestro como referencia de partida.

Confirmar que `gtid_binlog_pos` inicial es vacío: ninguna transacción con GTID se ha aplicado desde que se habilitó el binlog. Este es el punto de sincronía: ambos nodos tienen los mismos datos históricos (cargados en la Fase 8) y el esclavo comenzará a recibir solo los cambios futuros del maestro.

Paso 7 — Crear el archivo de configuración de replicación del esclavo (nodo02).

Se crea `/etc/mysql/mariadb.conf.d/60-replication-slave.cnf` con `server_id = 2`, configuración del relay log y `read_only = ON`.

Paso 8 — Reiniciar MariaDB en el esclavo y verificar sus variables.

Paso 9 — Configurar el enlace de replicación en el esclavo con `CHANGE MASTER TO`.

Paso 10 — Iniciar los hilos de replicación con `START SLAVE`.

Paso 11 — Verificar el estado de replicación con `SHOW SLAVE STATUS`.

Confirmar que `Slave_IO_Running` y `Slave_SQL_Running` son `Yes` y `Seconds_Behind_Master` es `0`.

Paso 12 — Pruebas de propagación: INSERT, UPDATE, DELETE y ALTER TABLE en el maestro.

Ejecutar cada operación en nodo01 y confirmar en nodo02 que el cambio aparece en tiempo real.

Paso 13 — Prueba de `read_only`: intentar escribir directamente en el esclavo.

Confirmar que el esclavo rechaza escrituras de usuarios sin privilegio `SUPER`.

Paso 14 — Monitoreo de lag y posición GTID en ambos nodos.

Paso 15 — Apagar ambas VMs y tomar el snapshot `fase10-completa`.

D. Comandos completos

Los bloques **(host)** se ejecutan en PowerShell en Windows. Los bloques **(VM — nodoXX)** se ejecutan en una sesión SSH al nodo indicado. Los bloques SQL dentro de `sudo mariadb` se ejecutan directamente en el prompt del motor.

D.1 Iniciar ambas VMs y conectarse por SSH (host)

PowerShell

```
VBoxManage startvm "bdd-nodo01" --type headless
VBoxManage startvm "bdd-nodo02" --type headless
```

Esperar 30 segundos y abrir dos terminales SSH simultáneas:

PowerShell

```
# Terminal 1 - maestro
ssh bddadmin@192.168.56.101
# Terminal 2 - esclavo (segunda ventana de PowerShell)
ssh bddadmin@192.168.56.102
```

D.2 Modificar `bind-address` en el maestro (VM — nodo01)

El archivo `50-server.cnf` tiene `bind-address = 127.0.0.1`. Se cambia a `0.0.0.0` para que el esclavo pueda conectarse al puerto 3306 a través de la red Host-Only:

Bash

```
# Ver el valor actual antes de modificar
grep bind-address /etc/mysql/mariadb.conf.d/50-server.cnf
# Reemplazar la línea (el patrón cubre posibles espacios alrededor del =)
sudo sed -i \
    's/^bind-address[ ](:space:)*=.*\/bind-address          =
0.0.0.0/' \
    /etc/mysql/mariadb.conf.d/50-server.cnf
# Confirmar el cambio
grep bind-address /etc/mysql/mariadb.conf.d/50-server.cnf
```

Salida esperada después del cambio:

Text

```
bind-address          = 0.0.0.0
```

Si la línea `bind-address` no existe en `50-server.cnf` o está comentada, agregarla explícitamente con `echo 'bind-address = 0.0.0.0' | sudo tee -a /etc/mysql/mariadb.conf.d/50-server.cnf` y verificar con `grep bind-address /etc/mysql/*.cnf /etc/mysql/**/*.cnf` que no hay otro archivo que la defina.

D.3 Crear el archivo de configuración de replicación del maestro (VM — nodo01)

 Bash

```
sudo tee /etc/mysql/mariadb.conf.d/60-replication-master.cnf > /dev/null
<< 'EOF'

# =====
# Configuración de replicación — ROL: MAESTRO
# Nodo: bdd-nodo01 | IP: 192.168.56.101 | server_id: 1
# Fase 10 del Laboratorio BDD.
# Cargado después de 50-server.cnf (número de archivo mayor).
# =====

[mariadb]
# --- Identificación única del nodo (NUNCA debe repetirse en el
ecosistema) ---
server_id                = 1
# --- Binary log ---
# Prefijo de los archivos de binlog en /var/log/mysql/
log_bin                  = /var/log/mysql/mariadb-bin
# ROW: registra valores concretos de cada fila, no la sentencia SQL.
# Obligatorio para GTID y para replicación multi-maestro (Fase 11).
binlog_format            = ROW
# Retención automática de archivos binlog (en días)
expire_logs_days         = 7
# Tamaño máximo por archivo antes de rotar al siguiente
max_binlog_size          = 100M
# Incluir la sentencia SQL original en eventos ROW (facilita mysqlbinlog)
binlog_annotate_row_events = ON
# Garantiza que el binlog se escribe a disco en cada COMMIT
# (consistencia máxima; puede reducir rendimiento en disco lento)
sync_binlog              = 1
# --- GTID ---
# Dominio de replicación del laboratorio (todos los nodos usan 1)
gtid_domain_id           = 1
# El maestro también escribe en su binlog los eventos que recibe de otros
# (necesario para cadenas A→B→C y para failover controlado)
log_slave_updates        = ON
# --- Seguridad de red ---
# Evitar resolución DNS por hostname: más rápido y previene errores
# cuando el DNS no responde. Los GRANTS deben usar IPs, no nombres.
skip_name_resolve        = ON
EOF
```

Verificar que el archivo se creó correctamente:

 Bash

```
cat /etc/mysql/mariadb.conf.d/60-replication-master.cnf
```

D.4 Reiniciar MariaDB en el maestro y verificar el binary log activo (VM — nodo01)

 Bash

```
sudo systemctl restart mariadb  
sudo systemctl status mariadb --no-pager -l
```

La salida debe mostrar `active (running)` . Si el servicio no arranca, revisar el log:

 Bash

```
sudo journalctl -u mariadb --no-pager -n 40
```

Verificar que el binary log y las variables de replicación quedaron activos:

 Bash

```
sudo mariadb << 'EOF'  
-- ¿Está el binlog habilitado?  
SHOW VARIABLES LIKE 'log_bin';  
-- ¿Cuál es el formato?  
SHOW VARIABLES LIKE 'binlog_format';  
-- server_id, GTID y otras variables de replicación  
SHOW VARIABLES LIKE 'server_id';  
SHOW VARIABLES LIKE 'gtid_domain_id';  
SHOW VARIABLES LIKE 'log_slave_updates';  
SHOW VARIABLES LIKE 'sync_binlog';  
-- ¿El motor ahora escucha en 0.0.0.0?  
SHOW VARIABLES LIKE 'bind_address';  
-- Listar archivos binlog existentes (debe haber mariadb-bin.000001)  
SHOW BINARY LOGS;  
-- Estado completo del maestro: archivo actual, posición y GTID  
SHOW MASTER STATUS;  
EOF
```

Salida esperada de `SHOW MASTER STATUS` (los valores numéricos variarán):

Text

```
+-----+-----+-----+-----+-----+
| File           | Position | Binlog_Do_DB | Binlog_Ignore_DB |
Gtid_binlog_pos |
+-----+-----+-----+-----+-----+
| mariadb-bin.000001 |      351 |              |                  |
|                  |          |              |                  |
+-----+-----+-----+-----+-----+
```

El campo `Gtid_binlog_pos` estará vacío o mostrará `0-1-N` si el motor ya escribió algún evento interno. Anotar el valor exacto: es el punto de partida del esclavo.

D.5 Crear el usuario de replicación en el maestro (VM — nodo01)

Bash

```
sudo mariadb << 'EOF'

-- -----
-- Usuario de replicación
-- El host '192.168.56.102' restringe el acceso a la IP exacta
-- del esclavo (principio de mínimo privilegio).
-- Se usa mysql_native_password porque el protocolo de replicación
-- binaria no soporta auth_socket.
-- -----

CREATE USER IF NOT EXISTS 'repl_user'@'192.168.56.102'
  IDENTIFIED BY 'ReplUser_2025!';
GRANT REPLICATION SLAVE ON *.* TO 'repl_user'@'192.168.56.102';
FLUSH PRIVILEGES;
-- Verificar que el usuario fue creado con el host correcto
SELECT User, Host, plugin
FROM   mysql.user
WHERE  User = 'repl_user';
-- Verificar que tiene exactamente REPLICATION SLAVE y nada más
SHOW GRANTS FOR 'repl_user'@'192.168.56.102';
EOF
```

Salida esperada de `SHOW GRANTS` :

Text

```
GRANT REPLICATION SLAVE ON *.* TO 'repl_user'@'192.168.56.102' IDENTIFIED  
BY PASSWORD '...'
```

D.6 Registrar la posición GTID de partida (VM — nodo01)

Bash

```
sudo mariadb -e "SELECT @@global.gtid_binlog_pos AS gtid_inicio_maestro;"  
sudo mariadb -e "SHOW MASTER STATUS\G"
```

Punto de sincronía importante: en este momento `bdd-nodo01` tiene el binlog habilitado pero no ha recibido ninguna transacción de aplicación desde la habilitación. Ambos nodos tienen la misma base de datos `lab_bdd` con los datos de la Fase 8. El esclavo comenzará a replicar exactamente desde este punto en adelante.

Entre este paso y `START SLAVE` del Paso 10, no ejecutar ninguna operación DML ni DDL en `bdd-nodo01`. Si lo haces, esas transacciones quedarán en el binlog y el esclavo intentará aplicarlas sobre datos que ya tiene, generando un error `Duplicate entry` en el hilo SQL.

D.7 Crear el archivo de configuración de replicación del esclavo (VM — nodo02)

 Bash

```
sudo tee /etc/mysql/mariadb.conf.d/60-replication-slave.cnf > /dev/null
<< 'EOF'

# =====
# Configuración de replicación — ROL: ESCLAVO
# Nodo: bdd-nodo02 | IP: 192.168.56.102 | server_id: 2
# Fase 10 del Laboratorio BDD.
# =====

[mariadb]
# --- Identificación única del nodo ---
server_id                = 2
# --- Relay log: almacena eventos descargados del maestro ---
relay_log                 = /var/log/mysql/mariadb-relay-bin
relay_log_index           = /var/log/mysql/mariadb-relay-bin.index
# Purgar automáticamente los segmentos del relay log ya aplicados
relay_log_purge           = ON
# --- Binary log del esclavo ---
# Necesario para log_slave_updates y para que este nodo pueda
# convertirse en maestro de otro esclavo en el futuro (failover)
log_bin                   = /var/log/mysql/mariadb-bin
binlog_format              = ROW
expire_logs_days          = 7
max_binlog_size            = 100M
binlog_annotate_row_events = ON
# Escribir en el binlog propio los eventos recibidos del maestro
log_slave_updates          = ON
# --- GTID (debe coincidir con el maestro) ---
gtid_domain_id            = 1
# --- Protección de escrituras directas ---
# Rechaza DML/DDL de usuarios sin privilegio SUPER
# El hilo SQL de replicación (system user) está exento
read_only                  = ON
# --- Seguridad de red ---
skip_name_resolve          = ON
EOF
```

Verificar el archivo:

 Bash

```
cat /etc/mysql/mariadb.conf.d/60-replication-slave.cnf
```

D.8 Reiniciar MariaDB en el esclavo y verificar sus variables (VM — nodo02)

 Bash

```
sudo systemctl restart mariadb
sudo systemctl status mariadb --no-pager -l
```

Verificar las variables clave del esclavo:

 Bash

```
sudo mariadb << 'EOF'
SHOW VARIABLES LIKE 'server_id';
SHOW VARIABLES LIKE 'read_only';
SHOW VARIABLES LIKE 'relay_log';
SHOW VARIABLES LIKE 'log_bin';
SHOW VARIABLES LIKE 'gtid_domain_id';
SHOW VARIABLES LIKE 'log_slave_updates';
-- Confirmar que aún no hay enlace configurado (resultado vacío)
SHOW SLAVE STATUS\G
EOF
```

`SHOW SLAVE STATUS` debe devolver un resultado vacío porque aún no se ejecutó `CHANGE MASTER TO`.

D.9 Configurar el enlace de replicación en el esclavo (VM — nodo02)

 Bash

```
sudo mariadb << 'EOF'

-- -----
-- CHANGE MASTER TO: define de dónde y cómo replicar.
--
-- MASTER_USE_GTID = slave_pos:
--   El esclavo enviará su gtid_slave_pos al maestro.
--   Como está vacío (ningún GTID aplicado aún), el maestro enviará
--   desde el inicio del binary log – exactamente lo que queremos,
--   porque los datos actuales en ambos nodos provienen de la Fase 8
--   (antes de que el binlog existiera) y solo se replicarán cambios
--   futuros.
-- -----

CHANGE MASTER TO
  MASTER_HOST      = '192.168.56.101',
  MASTER_PORT      = 3306,
  MASTER_USER      = 'repl_user',
  MASTER_PASSWORD  = 'ReplUser_2025!',
  MASTER_USE_GTID  = slave_pos;
-- Verificar que la configuración se almacenó (sin iniciar todavía)
SHOW SLAVE STATUS\G
EOF
```

En la salida de `SHOW SLAVE STATUS`, antes de iniciar, confirmar:

- `Master_Host: 192.168.56.101`
- `Master_User: repl_user`
- `Master_Port: 3306`
- `Using_Gtid: Slave_Pos`
- `Slave_IO_Running: No` (todavía no iniciado)
- `Slave_SQL_Running: No` (todavía no iniciado)

D.10 Iniciar los hilos de replicación (VM — nodo02)

 Bash

```
sudo mariadb -e "START SLAVE;"
# Dar 5 segundos para que los hilos establezcan la conexión inicial
sleep 5
sudo mariadb -e "SHOW SLAVE STATUS\G"
```

Campos críticos a verificar:

 Text

Slave_IO_Running: Yes	← hilo I/O conectado al maestro	✓
Slave_SQL_Running: Yes	← hilo SQL aplicando eventos	✓
Seconds_Behind_Master: 0	← sin retraso	✓
Master_Host: 192.168.56.101		
Master_User: repl_user		
Using_Gtid: Slave_Pos		
Gtid_IO_Pos:	← GTID hasta el que se descargó	
Last_IO_Error:	← vacío = sin errores	✓
Last_SQL_Error:	← vacío = sin errores	✓

Si cualquiera de los dos hilos muestra **No**, revisar **Last_IO_Error** y **Last_SQL_Error** y consultar la sección F.

D.11 Pruebas de propagación en tiempo real (VM — nodo01 y nodo02)

Ejecutar cada bloque de nodo01 y verificar inmediatamente en nodo02.

Prueba 1 — INSERT

En **bdd-nodo01** (maestro):

 Bash

```
sudo mariadb lab_bdd << 'EOF'
INSERT INTO clientes (nombre, apellido, email, telefono, region, ciudad)
VALUES ('Test', 'Replicacion', 'test.repl@lab.test', '5550000001',
'norte', 'CDMX');
-- Confirmar que la fila existe en el maestro
SELECT id, nombre, apellido, region, ciudad
FROM   clientes
WHERE  email = 'test.repl@lab.test';
EOF
```

En `bdd-nodo02` (esclavo), verificar que la fila apareció sin haberla insertado ahí:

 Bash

```
sudo mariadb lab_bdd -e "
SELECT id, nombre, apellido, region, ciudad
FROM   clientes
WHERE  email = 'test.repl@lab.test';
"
```

Prueba 2 — UPDATE

En `bdd-nodo01`:

 Bash

```
sudo mariadb lab_bdd -e "
UPDATE clientes
SET     ciudad = 'Guadalajara', region = 'oeste'
WHERE  email = 'test.repl@lab.test';
"
```

En `bdd-nodo02`, verificar el cambio:

 Bash

```
sudo mariadb lab_bdd -e "  
    SELECT id, nombre, ciudad, region  
    FROM    clientes  
    WHERE   email = 'test.repl@lab.test';  
"
```

La ciudad debe ser `Guadalajara` y la región `oeste` .

Prueba 3 — DELETE

En `bdd-nodo01` :

 Bash

```
sudo mariadb lab_bdd -e "  
    DELETE FROM clientes WHERE email = 'test.repl@lab.test';  
"
```

En `bdd-nodo02` , verificar que la fila desapareció:

 Bash

```
sudo mariadb lab_bdd -e "  
    SELECT COUNT(*) AS debe_ser_cero  
    FROM    clientes  
    WHERE   email = 'test.repl@lab.test';  
"
```

Prueba 4 — DDL (ALTER TABLE)

En `bdd-nodo01` , agregar una columna de auditoría:

 Bash

```
sudo mariadb lab_bdd -e "  
  ALTER TABLE clientes  
    ADD COLUMN ultima_modificacion DATETIME  
      DEFAULT CURRENT_TIMESTAMP  
      ON UPDATE CURRENT_TIMESTAMP  
      AFTER fecha_alta;  
"
```

En `bdd-nodo02`, verificar que la columna se replicó:

 Bash

```
sudo mariadb lab_bdd -e "SHOW COLUMNS FROM clientes  
LIKE 'ultima_modificacion';"
```

Prueba 5 — Verificación de conteo global de datos

Ejecutar en **ambos nodos** y comparar los resultados; deben ser idénticos:

 Bash

```
sudo mariadb -e "  
SELECT 'clientes'          AS tabla, COUNT(*) AS filas FROM lab_bdd.clientes  
UNION ALL  
SELECT 'productos',        COUNT(*)          FROM  
lab_bdd.productos          UNION ALL  
SELECT 'pedidos',          COUNT(*)          FROM lab_bdd.pedidos  
UNION ALL  
SELECT 'detalle_pedidos',  COUNT(*)          FROM lab_bdd.detalle_pedidos;  
"
```

Salida esperada (idéntica en nodo01 y nodo02):

Text

```
+-----+-----+
| tabla          | filas |
+-----+-----+
| clientes       | 20    |
| productos      | 10    |
| pedidos        | 20    |
| detalle_pedidos | 36    |
+-----+-----+
```

D.12 Prueba de `read_only` en el esclavo (VM — nodo02)

Bash

```
# Intento con app_user (no tiene SUPER)
mariadb -u app_user -p'AppUser_2025!' lab_bdd \
-e "INSERT INTO clientes (nombre, apellido, email, region)
VALUES ('Intento', 'Directo', 'intento@lab.test', 'norte');" \
2>&1
# Intento con lab_admin (tampoco tiene SUPER)
mariadb -u lab_admin -p'LabAdmin_2025!' lab_bdd \
-e "INSERT INTO clientes (nombre, apellido, email, region)
VALUES ('Intento', 'Admin', 'intento.admin@lab.test', 'norte');" \
2>&1
```

Ambos intentos deben producir el error:

Text

```
ERROR 1290 (HY000): The MariaDB server is running with the --read-
only option
so it cannot execute this statement
```

D.13 Monitoreo de lag y posición GTID (VM — ambos nodos)

 Bash

```
# En nodo01: posición GTID actual del binlog del maestro
sudo mariadb -e "SELECT @@global.gtid_binlog_pos
AS posicion_gtid_maestro;"
sudo mariadb -e "SHOW MASTER STATUS\G"
# En nodo02: posición GTID aplicada por el esclavo y estado de hilos
sudo mariadb -e "SELECT @@global.gtid_slave_pos AS posicion_gtid_esclavo;"
sudo mariadb -e "SHOW SLAVE STATUS\G" | \
    grep -E
'Slave_IO_Running|Slave_SQL_Running|Seconds_Behind|Gtid|Last.*Error'
```

Para monitoreo continuo del lag (se actualiza cada 2 segundos; **Ctrl+C** para salir):

 Bash

```
watch -n 2 "sudo mariadb -e 'SHOW SLAVE STATUS\G' 2>/dev/null | \
    grep -E
'Slave_IO_Running|Slave_SQL_Running|Seconds_Behind_Master|Gtid_IO_Pos'"
```

El `gtid_slave_pos` del esclavo debe coincidir con el `gtid_binlog_pos` del maestro cuando `Seconds_Behind_Master` es `0`, indicando que no hay retraso.

D.14 Apagar ambas VMs y tomar el snapshot `fase10-completa` (host)

Desde las sesiones SSH de cada nodo:

 Bash

```
# En la terminal SSH de nodo01
sudo poweroff
# En la terminal SSH de nodo02
sudo poweroff
```

Confirmar desde el host que ambas VMs están detenidas:

 PowerShell

```
VBoxManage list runningvms
```

La salida debe estar vacía. Tomar los snapshots:

 PowerShell

```
VBoxManage snapshot "bdd-nodo01" take "fase10-completa" `
  --description "MAESTRO: binlog ROW + GTID habilitado. server_id=1. repl_user
creado para 192.168.56.102. bind-address=0.0.0.0. Pruebas
INSERT/UPDATE/DELETE/DDl verificadas."
VBoxManage snapshot "bdd-nodo02" take "fase10-completa" `
  --description "ESCLAVO: read_only=ON. server_id=2. CHANGE MASTER TO con
MASTER_USE_GTID=slave_pos hacia 192.168.56.101. Slave_IO=Yes, Slave_SQL=Yes.
Columna ultima_modificacion replicada vía DDL."
```

Confirmar los snapshots:

 PowerShell

```
VBoxManage snapshot "bdd-nodo01" list
VBoxManage snapshot "bdd-nodo02" list
```

Cada VM debe mostrar seis snapshots: `fase05-completa` hasta `fase10-completa`.

E. Verificación de funcionamiento

Esta fase se considera completa cuando se cumplen **todos** los puntos siguientes en el estado de los nodos encendidos:

1. `SHOW VARIABLES LIKE 'log_bin'` devuelve `ON` en `bdd-nodo01`.
2. `SHOW BINARY LOGS` muestra al menos `mariadb-bin.000001` en `bdd-nodo01`.
3. `SHOW VARIABLES LIKE 'binlog_format'` devuelve `ROW` en `bdd-nodo01`.
4. `SHOW VARIABLES LIKE 'server_id'` devuelve `1` en `bdd-nodo01` y `2` en `bdd-nodo02`.
5. `SHOW GRANTS FOR 'repl_user'@'192.168.56.102'` en `bdd-nodo01` muestra exactamente `GRANT REPLICATION SLAVE ON *.* TO 'repl_user'@'192.168.56.102'`.
6. `SHOW SLAVE STATUS\G` en `bdd-nodo02` muestra:
 - `Slave_IO_Running: Yes`

- `Slave_SQL_Running: Yes`
- `Seconds_Behind_Master: 0`
- `Last_IO_Error:` (vacío)
- `Last_SQL_Error:` (vacío)
- `Using_Gtid: Slave_Pos`

7. `SHOW VARIABLES LIKE 'read_only'` devuelve `ON` en `bdd-nodo02`.
 8. La columna `ultima_modificacion` (añadida en la Prueba 4 de DDL) existe en la tabla `clientes` de **ambos** nodos.
 9. Los conteos de las cuatro tablas de `lab_bdd` son idénticos en ambos nodos (20 / 10 / 20 / 36 registros).
 10. Los intentos de INSERT directo con `app_user` y `lab_admin` en `bdd-nodo02` fallan con `ERROR 1290: read-only`.
 11. El `gtid_slave_pos` del esclavo coincide con el `gtid_binlog_pos` del maestro.
 12. Los snapshots `fase10-completa` existen en `bdd-nodo01` y `bdd-nodo02`.
 13. El estudiante puede explicar sin ver el documento la diferencia entre el hilo I/O y el hilo SQL, y qué ocurre si solo uno de los dos falla.
-

F. Problemas comunes y soluciones

Problema	Causa probable	Solución
<code>Slave_IO_Running: No ,</code> <code>Last_IO_Error: "Can't connect to MySQL server on '192.168.56.101'"</code>	<code>bind-address</code> sigue siendo <code>127.0.0.1</code> en nodo01; el <code>sed</code> no se aplicó o el servicio no se reinició	En nodo01: <code>grep bind-address /etc/mysql/mariadb.conf.d/50-server.cnf</code> ; si aún muestra <code>127.0.0.1</code> , ejecutar el <code>sed</code> de nuevo y <code>sudo systemctl restart mariadb</code>
<code>Slave_IO_Running: No ,</code> <code>Last_IO_Error: "Access denied for user 'repl_user'@'192.168.56.102'"</code>	El usuario fue creado con host incorrecto, o la contraseña difiere	En nodo01: <code>SELECT User, Host FROM mysql.user WHERE User='repl_user';</code> ; si el host es incorrecto, <code>DROP USER 'repl_user'@'host_incorrecto';</code> y recrear con la IP exacta
<code>Slave_SQL_Running: No ,</code> <code>Last_SQL_Error: "Duplicate entry 'X' for key 'PRIMARY'"</code>	Se ejecutó DML en nodo01 después de habilitar el binlog y antes de que el esclavo iniciara; el esclavo intenta insertar datos que ya existen	Para omitir ese evento puntual (solo si no representa pérdida de datos crítica): <code>STOP SLAVE;</code> <code>SET GLOBAL SQL_SLAVE_SKIP_COUNTER = 1;</code> <code>START SLAVE;</code> Si el problema se repite, ver el reto avanzado para resincronización completa
<code>Slave_IO_Running: No ,</code> <code>Last_IO_Error: "error connecting ... errno: 111 Connection refused"</code>	Puerto 3306 bloqueado por firewall (ufw o iptables activo en nodo01)	En nodo01: <code>sudo ufw status</code> ; si está activo: <code>sudo ufw allow from 192.168.56.102 to any port 3306 proto tcp</code>
MariaDB no arranca en nodo01 después de crear <code>60-replication-master.cnf</code>	Error de sintaxis en el archivo de configuración	<code>sudo journalctl -u mariadb --no-pager -n 30</code> ; buscar línea <code>[ERROR]</code> o <code>unknown variable</code> ; corregir el parámetro indicado
<code>SHOW SLAVE STATUS</code> muestra <code>Using_Gtid: No</code> en lugar de <code>Slave_Pos</code>	El <code>CHANGE MASTER TO</code> se ejecutó sin <code>MASTER_USE_GTID = slave_pos</code>	<code>STOP SLAVE;</code> <code>CHANGE MASTER TO MASTER_USE_GTID = slave_pos;</code> <code>START SLAVE;</code>
El <code>sed</code> no encontró la línea <code>bind-address</code> en <code>50-server.cnf</code>	El parámetro está comentado o en otro archivo	<code>grep -r bind-address /etc/mysql/</code> para localizarlo; descomentarlo o agregar <code>bind-address = 0.0.0.0</code> explícitamente en <code>60-replication-master.cnf</code>

<code>Seconds_Behind_Master</code> crece indefinidamente sin llegar a 0	El hilo SQL no puede aplicar eventos tan rápido como el I/O los descarga	Verificar con <code>SHOW PROCESSLIST</code> en nodo02 qué está haciendo el hilo SQL (<code>System lock</code> , <code>Updating</code> , etc.); comprobar que el disco de nodo02 no está saturado con <code>iostat -x 1</code>
<code>SHOW GRANTS FOR 'repl_user'@'192.168.56.102'</code> también aparece en nodo02	Los eventos <code>CREATE USER</code> y <code>GRANT</code> del maestro se replicaron al esclavo, como es esperado	Comportamiento correcto. El usuario <code>repl_user</code> en nodo02 es inofensivo; no tiene privilegios dañinos y no puede ser usado para configurar una replicación entrante al esclavo
<code>read_only</code> no bloquea escrituras del usuario <code>root</code>	<code>root</code> tiene el privilegio <code>SUPER</code> , que está exento de <code>read_only</code> por diseño	Comportamiento correcto: <code>read_only</code> protege de escrituras accidentales de aplicaciones, no de administradores. Para bloquear también a <code>SUPER</code> , usar <code>SET GLOBAL super_read_only = ON</code> (con precaución, bloquea al hilo SQL si se activa antes de que esté conectado)
El <code>ALTER TABLE</code> de la Prueba 4 falla en nodo02 con "Column already exists"	La columna <code>ultima_modificacion</code> se añadió directamente en nodo02 antes de esta prueba	En nodo02: <code>STOP SLAVE; SET GLOBAL SQL_SLAVE_SKIP_COUNTER = 1; START SLAVE;</code> . En el futuro, nunca ejecutar DDL directamente en el esclavo
MariaDB en nodo02 arranca pero la replicación no se inicia automáticamente tras un reinicio	<code>START SLAVE</code> no persiste entre reinicios si hay un error de hilo al arrancar	Verificar <code>SHOW SLAVE STATUS\G</code> después del reinicio; ejecutar <code>START SLAVE</code> manualmente si los hilos están en <code>No</code> . Para arranque automático: <code>--skip-slave-start=0</code> en la configuración (por defecto los hilos arrancan automáticamente en MariaDB 10.11)

G. Checklist de validación

- ☐ `grep bind-address /etc/mysql/mariadb.conf.d/50-server.cnf` devuelve `0.0.0.0` en `bdd-nodo01`.

- ☐ El archivo `60-replication-master.cnf` existe en `bdd-nodo01` con `server_id = 1`, `log_bin`, `binlog_format = ROW` y `gtid_domain_id = 1`.
 - ☐ `SHOW VARIABLES LIKE 'log_bin'` devuelve `ON` en `bdd-nodo01`.
 - ☐ `SHOW BINARY LOGS` muestra al menos el archivo `mariadb-bin.000001` en `bdd-nodo01`.
 - ☐ `SHOW GRANTS FOR 'repl_user'@'192.168.56.102'` muestra `GRANT REPLICATION SLAVE ON *.*` en `bdd-nodo01` y el usuario no existe con host distinto de `192.168.56.102`.
 - ☐ El archivo `60-replication-slave.cnf` existe en `bdd-nodo02` con `server_id = 2`, `relay_log` y `read_only = ON`.
 - ☐ `SHOW VARIABLES LIKE 'server_id'` devuelve `1` en `nodo01` y `2` en `nodo02`.
 - ☐ `SHOW VARIABLES LIKE 'read_only'` devuelve `ON` en `bdd-nodo02`.
 - ☐ `SHOW SLAVE STATUS\G` en `bdd-nodo02` muestra `Slave_IO_Running: Yes`.
 - ☐ `SHOW SLAVE STATUS\G` en `bdd-nodo02` muestra `Slave_SQL_Running: Yes`.
 - ☐ `Seconds_Behind_Master` es `0` en `bdd-nodo02`.
 - ☐ `Last_IO_Error` y `Last_SQL_Error` están vacíos en `bdd-nodo02`.
 - ☐ `Using_Gtid: Slave_Pos` aparece en `SHOW SLAVE STATUS` de `bdd-nodo02`.
 - ☐ El INSERT de prueba (`test.repl@lab.test`) apareció en `bdd-nodo02` sin ejecutarlo ahí directamente.
 - ☐ El UPDATE (ciudad → `Guadalajara`) se reflejó en `bdd-nodo02`.
 - ☐ El DELETE eliminó la fila en `bdd-nodo02` (conteo = 0).
 - ☐ La columna `ultima_modificacion` existe en `clientes` de **ambos** nodos (propagada por el ALTER TABLE en el maestro).
 - ☐ Los conteos de las cuatro tablas de `lab_bdd` son idénticos en `nodo01` y `nodo02`.
 - ☐ El intento de INSERT con `app_user` en `bdd-nodo02` falla con error `read-only`.
 - ☐ El intento de INSERT con `lab_admin` en `bdd-nodo02` falla con error `read-only`.
 - ☐ Los snapshots `fase10-completa` existen en `bdd-nodo01` y `bdd-nodo02` y cada VM muestra seis snapshots con `VBoxManage snapshot list`.
 - ☐ Puedo explicar sin ver el documento la diferencia entre el hilo I/O y el hilo SQL del esclavo, y qué ocurre si solo uno de los dos falla.
 - ☐ Puedo explicar qué es `Seconds_Behind_Master` y qué implica un valor distinto de cero.
 - ☐ Puedo explicar por qué `read_only` no afecta al hilo SQL de replicación pero sí a `app_user`.
-

Preguntas teóricas para estudiantes

1. El parámetro `log_slave_updates = ON` se habilitó tanto en el maestro como en el esclavo, aunque en esta fase el esclavo no tiene ningún esclavo propio. Explica en qué escenario futuro —dentro de las fases restantes del laboratorio o en una arquitectura real— sería indispensable que el esclavo también tenga su propio binary log activo. ¿Qué operación de administración (failover, clonación, auditoría) se volvería imposible o muy difícil sin él?
2. La replicación de MariaDB en este laboratorio es **asíncrona**. Describe un escenario concreto de fallo —no abstracto, sino usando los nodos de este laboratorio— en el que el asincronismo resulta en pérdida de datos: ¿qué operación debe haberse ejecutado en `bdd-nodo01`, cuándo debe fallar el nodo, y qué dato concreto de `lab_bdd` se perdería en `bdd-nodo02`? ¿Qué mecanismo de MariaDB (sin hardware redundante extra) mitigaría parcialmente este riesgo?
3. Compara la replicación basada en **posición de archivo** (`MASTER_LOG_FILE` + `MASTER_LOG_POS`) con la basada en **GTID** (`MASTER_USE_GTID = slave_pos`). Si el maestro rota el binary log (crea `mariadb-bin.000002`) exactamente cuando el esclavo está leyendo el último evento de `mariadb-bin.000001`, ¿cómo maneja cada modalidad esa transición? ¿Qué ventaja operacional concreta ofrece el GTID si en el futuro hay que re apuntar el esclavo a un maestro de sustitución?
4. En el esclavo, `read_only = ON` impide escrituras de usuarios normales, pero el hilo SQL de replicación puede seguir aplicando cambios. Explica bajo qué usuario interno de MariaDB opera el hilo SQL y qué privilegios tiene que lo eximen de `read_only`. ¿Qué ocurriría con la replicación si se activara `super_read_only = ON` en el esclavo *antes* de que el hilo SQL se haya iniciado? ¿Y si se activa *después* de que el hilo SQL ya está corriendo?
5. El formato ROW registra las filas afectadas (imágenes antes y después del cambio) en lugar de la sentencia SQL original. Analiza las implicaciones para la sentencia `UPDATE pedidos SET total = total * 1.10 WHERE region = 'norte'` si hay 10 pedidos en la región norte: ¿cuántos eventos ROW generaría esta sentencia en el binlog? ¿Cómo se compararía el tamaño del evento en formato ROW versus STATEMENT? ¿En qué circunstancia podría el formato STATEMENT producir un resultado distinto en el esclavo al aplicar esta misma sentencia?

Ejercicios prácticos

1. **Observación del binary log con `mysqlbinlog`.**
En `bdd-nodo01`, ejecutar un INSERT, un UPDATE y un DELETE sobre `lab_bdd`. Luego usar `mysqlbinlog` para leer el archivo de binary log y localizar los eventos:

 Bash

```
sudo mysqlbinlog --base64-output=DECODE-ROWS --verbose \  
/var/log/mysql/mariadb-bin.000001 | \  
grep -A 20 "### INSERT\|### UPDATE\|### DELETE"
```

Responder: ¿se ve la sentencia SQL original o los valores de columnas? ¿Puedes identificar el `server_id`, el timestamp y el GTID de cada transacción? ¿Cómo aparece el ALTER TABLE de la Prueba 4 en el log (como evento ROW o de otro tipo)? Documentar los hallazgos con la salida del comando.

2. Simulación de fallo del hilo SQL y recuperación.

En `bdd-nodo02`, detener únicamente el hilo SQL manteniendo el I/O activo:

 Bash

```
sudo mariadb -e "STOP SLAVE SQL_THREAD;"
```

En `bdd-nodo01`, ejecutar 5 INSERT en `lab_bdd.clientes`. Verificar que `Seconds_Behind_Master` crece en `bdd-nodo02` (el I/O sigue descargando pero el SQL no aplica). Reiniciar el hilo SQL y observar cómo el lag cae a 0:

 Bash

```
sudo mariadb -e "START SLAVE SQL_THREAD;"  
watch -n 1 "sudo mariadb -e 'SHOW SLAVE STATUS\G' |  
grep Seconds_Behind_Master"
```

Documentar los valores de `Seconds_Behind_Master` a lo largo del ejercicio y explicar el comportamiento observado. Al finalizar, eliminar las 5 filas de prueba con `DELETE FROM lab_bdd.clientes WHERE id > 20`.

3. Script de verificación de consistencia maestro-esclavo.

Escribir un script Bash en `bdd-nodo01` que: (a) inserte 30 filas de prueba en `lab_bdd.clientes` con valores de `region` aleatorios distribuidos entre los cuatro valores del ENUM; (b) espere 3 segundos; © compare automáticamente el conteo de filas en nodo01 (`mysql --host=127.0.0.1`) y en nodo02 (`mysql --host=192.168.56.102 -u repl_user -p'ReplUser_2025!'`) — para esta prueba, otorgar temporalmente el privilegio `SELECT` a `repl_user` en `lab_bdd.clientes`; (d) imprima `CONSISTENTE: maestro=X, esclavo=X` si los conteos coinciden o `DIVERGENCIA: maestro=X, esclavo=Y` si difieren. Ejecutar el script y documentar el resultado. Al finalizar, eliminar las filas de prueba y revocar el privilegio temporal de `repl_user`.

Reto adicional para alumnos avanzados

Implementar el procedimiento de **resincronización completa del esclavo desde cero**, aplicable cuando el esclavo se desincronizó gravemente (por ejemplo, alguien ejecutó `DROP TABLE pedidos` directamente en `bdd-nodo02` o modificó filas manualmente). El procedimiento debe poder ejecutarse sin detener el maestro ni interrumpir a los clientes que escriben en `bdd-nodo01`. Pasos a documentar con comandos exactos:

1. Detener los hilos de replicación en el esclavo (`STOP SLAVE`).
2. Generar un volcado consistente del maestro con `mysqldump` , incluyendo la posición GTID del punto en que se tomó el volcado (investigar las opciones específicas de `mysqldump` de MariaDB 10.11 que registran la información GTID en el encabezado del archivo; comparar `--master-data=2` , `--dump-slave` y la variable de sesión `@@global.gtid_binlog_pos` anotada manualmente antes del volcado).
3. Restaurar el volcado en el esclavo.
4. Reconfigurar `CHANGE MASTER TO` con la posición GTID exacta registrada en el paso 2.
5. Iniciar los hilos de replicación y verificar con `SHOW SLAVE STATUS\G` que ambos hilos muestran `Yes` , el lag es `0` y `Last_SQL_Error` está vacío.

Incluir en la entrega: los comandos exactos con su salida, el razonamiento técnico de cada decisión (en particular, por qué se usa `--single-transaction` y qué garantía ofrece), y la diferencia entre resincronizar desde un volcado `mysqldump` versus desde un snapshot de VirtualBox del nodo01 en `fase08-completa` .

Criterios de evaluación para el profesor

Criterio	Peso	Indicador de logro
Configuración del maestro	20%	<code>SHOW VARIABLES</code> confirma <code>log_bin=ON</code> , <code>binlog_format=ROW</code> , <code>server_id=1</code> , <code>gtid_domain_id=1</code> ; <code>60-replication-master.cnf</code> existe con el contenido correcto; <code>repl_user</code> tiene exactamente <code>REPLICATION SLAVE ON *.*</code> para la IP correcta del esclavo
Configuración del esclavo	20%	<code>SHOW VARIABLES</code> confirma <code>server_id=2</code> , <code>read_only=ON</code> ; <code>60-replication-slave.cnf</code> existe; <code>SHOW SLAVE STATUS</code> muestra <code>Slave_IO_Running: Yes</code> , <code>Slave_SQL_Running: Yes</code> , <code>Using_Gtid: Slave_Pos</code>
Pruebas de propagación	25%	El estudiante ejecuta y documenta las cinco pruebas (INSERT, UPDATE, DELETE, DDL, conteo global) y muestra evidencia de que cada cambio en el maestro se reflejó en el esclavo; puede presentar los resultados de ambos nodos simultáneamente
Verificación de <code>read_only</code>	10%	Se demuestra con evidencia del mensaje de error que usuarios sin SUPER no pueden escribir en el esclavo; el estudiante explica por qué el hilo SQL es la excepción y el usuario <code>root</code> también lo es
Comprensión conceptual (preguntas teóricas)	20%	Las respuestas demuestran comprensión de los hilos de replicación, GTID vs. file+pos, asincronismo y <code>read_only</code> usando terminología técnica precisa con ejemplos del propio laboratorio
Snapshots y documentación	5%	Los snapshots <code>fase10-completa</code> existen en ambos nodos con descripciones informativas; el estudiante puede listar los seis snapshots de cada VM con <code>VBoxManage snapshot list</code>

Preparación para la siguiente fase

La Fase 11: Replicación Lógica (Multi-Maestro con `bdd-nodo03`) requerirá:

- La replicación física entre `bdd-nodo01` y `bdd-nodo02` funcionando correctamente (esta fase), con snapshots `fase10-completa` tomados en ambos nodos.
- Comprensión sólida de GTID y binary log: la Fase 11 reutilizará exactamente los mismos conceptos pero en una topología bidireccional donde nodo03 actúa como maestro simultáneo de nodo01.
- **Creación de `bdd-nodo03`** : será la primera VM nueva del laboratorio desde la Fase 5. Se creará como un **clon enlazado** (*linked clone*) de `bdd-nodo01` tomado del snapshot `fase08-completa` —que tiene `lab_bdd` instalado pero sin configuración de replicación— y se le asignará la IP `192.168.56.103` y el hostname `bdd-nodo03` . El procedimiento de clonación y configuración inicial del nuevo nodo se hará íntegramente al inicio de la Fase 11.
- El Documento de Diseño Distribuido (`fase09-disenyo-distribuido.md` , sección 2) como referencia: define que nodo03 tendrá el esquema `lab_bdd` completo y replicación bidireccional con nodo01.

No es necesario instalar, configurar ni crear nada adicional ahora.

Fase 11 — Replicación Lógica Multi-Maestro con MariaDB Galera Cluster

Continuación directa de la Fase 10. `bdd-nodo01` opera como maestro de replicación física y `bdd-nodo02` como esclavo con `read_only = ON`; el snapshot `fase10-completa` fue tomado en ambos nodos. Esta fase introduce la **replicación lógica multi-maestro** mediante **MariaDB Galera Cluster**: los tres nodos se convierten en pares iguales que aceptan lecturas y escrituras simultáneamente con garantías de consistencia síncrona. Se crea `bdd-nodo03`, se instala la librería Galera en los tres nodos, se migra la topología de la Fase 10 y se verifican el comportamiento multi-maestro, la detección automática de conflictos y la recuperación ante la caída de un nodo. Esta fase no introduce fragmentación de datos (eso corresponde a la Fase 13): los tres nodos mantienen una copia completa e idéntica de `lab_bdd`.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Distinguir con precisión la replicación **asíncrona** (Fase 10) de la replicación **síncrona por certificación** de Galera, explicando las implicaciones de cada modelo para consistencia y disponibilidad según el teorema CAP.
2. Explicar qué es un **write-set**, en qué consiste la **certificación** de transacciones en Galera y por qué este mecanismo puede detectar conflictos que la replicación asíncrona no puede.
3. Crear y configurar `bdd-nodo03` a partir de un clon de un nodo existente, ajustando los parámetros que identifican de forma única a cada miembro del clúster.
4. Instalar la librería `galera-4` y definir los parámetros `wsrep` esenciales en los tres nodos del laboratorio.
5. Inicializar un clúster Galera mediante la operación de **bootstrap** y unir los nodos restantes usando SST (*State Snapshot Transfer*).
6. Ejecutar escrituras en **cualquier nodo del clúster** y verificar que se propagan sincrónicamente a los demás nodos, demostrando el comportamiento multi-maestro genuino.
7. Forzar un **escenario de conflicto** (dos transacciones que modifican la misma fila en distintos nodos simultáneamente) y observar cómo Galera resuelve el conflicto con el **rollback automático** de la transacción perdedora.
8. Simular la **caída y recuperación de un nodo**, interpretar los indicadores `wsrep_cluster_size` y `wsrep_cluster_status`, y verificar que el nodo recuperado obtiene los datos que le faltaban sin intervención manual.

9. Cerrar la fase con el snapshot `fase11-completa` en los **tres nodos** en el estado de mayor preparación alcanzado hasta ahora.

B. Conceptos teóricos necesarios

1. Replicación asíncrona vs. síncrona: el compromiso CAP en la práctica.

En la Fase 10 se implementó replicación **asíncrona**: el maestro confirma (`COMMIT`) la transacción al cliente *antes* de que el esclavo la haya aplicado. El esclavo puede quedar rezagado. Si el maestro cae en ese instante, el esclavo puede no tener los últimos cambios — esto es pérdida de datos aceptada a cambio de mayor disponibilidad. Esta es la postura **AP** del teorema CAP: disponibilidad sobre consistencia ante partición.

Galera implementa replicación **síncrona por certificación**: antes de que el `COMMIT` se confirme al cliente, el nodo que originó la transacción envía su *write-set* a todos los demás nodos y espera que todos lo certifiquen. Solo cuando todos confirman que no hay conflicto se confirma la transacción. Esta es la postura **CP**: todos los nodos siempre tienen los mismos datos, pero un nodo que pierde contacto con el clúster deja de aceptar escrituras hasta recuperar la comunicación.

2. El protocolo wsrep y los write-sets.

`wsrep` (*Write-Set REplication*) es una interfaz estándar que MariaDB expone y que la librería Galera implementa. El flujo de una transacción con wsrep es:

Text

```
Cliente → BEGIN en nodo01
Cliente → INSERT / UPDATE / DELETE (se ejecuta localmente en memoria)
Cliente → COMMIT
      ↓
nodo01 serializa los cambios como write-set:
      { primary_keys_afectadas | valores_nuevos | número_de_secuencia_global
      (seqno) }
      ↓
nodo01 difunde el write-set a nodo02 y nodo03 (broadcast)
      ↓
CADA NODO certifica: ¿algún write-set anterior en vuelo toca las
mismas PKs?
      ↓ NO → certificación exitosa → COMMIT aplicado en todos los nodos
      ↓ SÍ → conflicto → el write-set con seqno menor gana;
                el otro recibe ROLLBACK automático
```

La ventaja clave es que la certificación no requiere comunicación durante la ejecución de la transacción — solo en el momento del `COMMIT`. Esto minimiza la penalización de latencia a una sola ronda de red por transacción.

3. SST e IST: cómo un nodo nuevo o recuperado obtiene datos.

Cuando un nodo se une al clúster (o vuelve tras estar offline), necesita sincronizarse:

- **SST (State Snapshot Transfer):** copia completa del estado actual del nodo donante al nodo solicitante (*joiner*). El método `rsync` copia los archivos de datos directamente y es el más simple; `mariabackup` es no bloqueante y se recomienda en producción. Se usa cuando el nodo joiner está completamente desactualizado o es nuevo.
- **IST (Incremental State Transfer):** solo transfiere los write-sets que el joiner no tiene, usando el *gcache* (caché en disco de write-sets recientes). Se usa cuando el nodo estuvo offline poco tiempo y los write-sets faltantes aún están en el gcache. Es mucho más rápido que SST.

El clúster elige automáticamente SST o IST según el estado del nodo que se une.

4. Componente Primario y quórum.

Galera usa el concepto de **Componente Primario (PC)**: la partición del clúster que tiene quórum ($n/2 + 1$ nodos). Solo la PC puede procesar transacciones. Si la red se divide en dos particiones, la que tenga mayoría de nodos continúa como PC; la minoría entra en estado `non-Primary` y rechaza todas las escrituras.

Para un clúster de 3 nodos:

- Quórum = 2 nodos.
- Si cae 1 nodo → los 2 restantes mantienen el quórum y siguen operando.
- Si caen 2 nodos → el nodo restante queda solo, pierde quórum y entra en `non-Primary` (no acepta escrituras). Esto es intencional: se prefiere detener al nodo antes que aceptar datos que podrían no ser consistentes con el resto del clúster cuando este se recupere.

5. El problema del split-brain.

Si una partición de red divide el clúster en dos mitades exactamente iguales, ambas podrían creer que son la PC, aceptar escrituras incompatibles y producir divergencia de datos — el *split-brain*. Un clúster de número **impar** de nodos (3, 5, 7...) nunca produce empate, razón fundamental por la que Galera se opera con 3 nodos mínimo. Un clúster de 2 nodos puede complementarse con un árbitro ligero (`g arb d`) que ocupa el tercer “voto” de quórum sin almacenar datos.

6. `auto_increment` en multi-maestro: prevención de colisiones de clave primaria.

Si dos nodos generan simultáneamente el valor `AUTO_INCREMENT` siguiente, podrían producir el mismo número. Galera resuelve esto con `wsrep_auto_increment_control = ON` (activado por defecto), que ajusta automáticamente dos variables en todos los nodos:

- `auto_increment_increment = N` (donde N = número de nodos del clúster): cada nodo avanza de N en N.
- `auto_increment_offset = [1, 2, 3, ...]` según el número de secuencia del nodo.

Con 3 nodos: nodo01 genera `1, 4, 7, ...`; nodo02 genera `2, 5, 8, ...`; nodo03 genera `3, 6, 9, ...`. Nunca habrá colisiones de PK aunque dos nodos inserten al mismo tiempo.

7. Galera y el binary log: coexistencia con replicación asíncrona downstream.

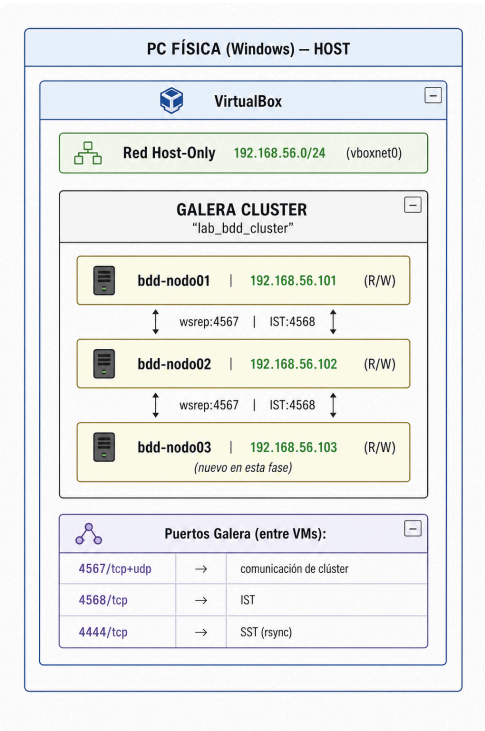
Un clúster Galera puede mantener el binary log activo en uno o todos sus nodos. Esto permite añadir **esclavos asíncronos tradicionales** apuntando a cualquier miembro del clúster — arquitectura conocida como *Galera + async slave* — que se usa para offloading de respaldos o replicación geográfica. En este laboratorio el binary log de nodo01 (activo desde la Fase 10) se conserva con este propósito didáctico.

8. Contraste entre replicación “física” (Fase 10) y “lógica” (Fase 11).

Aspecto	Fase 10 — Física (binlog row-based)	Fase 11 — Lógica (Galera wsrep)
Unidad de replicación	Evento de fila binario (imagen antes/después)	Write-set (PKs afectadas + valores + seqno)
Sincronía	Asíncrona	Síncrona por certificación
Topología	Maestro → esclavo (unidireccional)	Todos los nodos son pares iguales
Escrituras	Solo en el maestro	En cualquier nodo
Detección de conflictos	No (divergencia silenciosa posible)	Automática; rollback determinista
Postura CAP	AP (disponibilidad ante partición)	CP (consistencia ante partición)
Implementación en MariaDB	<code>log_bin</code> , <code>CHANGE MASTER TO</code> , <code>START SLAVE</code>	<code>wsrep_on = ON</code> , <code>galera_new_cluster</code>

C. Arquitectura utilizada

La arquitectura final de esta fase integra tres nodos Galera en la misma red Host-Only:



Nodo	Hostname	IP	Rol en Galera	¿Acepta escrituras?
1	bdd-nodo01	192.168.56.101	Nodo Galera 1 (bootstrapper inicial)	Sí
2	bdd-nodo02	192.168.56.102	Nodo Galera 2 (joiner)	Sí
3	bdd-nodo03	192.168.56.103	Nodo Galera 3 (joiner, nuevo)	Sí

El término “bootstrapper” indica únicamente qué nodo inicializa el clúster en el primer arranque. En operación normal los tres nodos son **pares completamente iguales**: no existe un maestro, no existe un esclavo.

D. Procedimiento paso a paso

Paso 1 — Crear `bdd-nodo03` como clon enlazado de `bdd-nodo01`.

Clonar la VM `bdd-nodo01` desde el snapshot `fase09-completa` (estado previo a la configuración de replicación física), que proporciona una base limpia con Ubuntu Server, MariaDB 10.11 y el esquema `lab_bdd` sin configuración de maestro ni esclavo.

Paso 2 — Personalizar `bdd-nodo03`: `hostname`, `IP` y `server_id`.

Cambiar el `hostname` a `bdd-nodo03`, la IP estática a `192.168.56.103` y el `server_id` de MariaDB a `3`. Actualizar el archivo `/etc/hosts` en los tres nodos para incluir los nombres de los tres miembros.

Paso 3 — Instalar la librería Galera en los tres nodos.

Instalar el paquete `galera-4` que provee la librería `libgalera_smm.so` requerida por MariaDB para el protocolo `wsrep`.

Paso 4 — Limpiar la configuración de esclavo de la Fase 10 en `bdd-nodo02`.

Detener el agente esclavo, ejecutar `RESET SLAVE ALL` y eliminar `read_only = ON` del archivo de configuración. `bdd-nodo02` pasa de esclavo a par Galera.

Paso 5 — Crear el archivo de configuración Galera en cada nodo.

Crear `/etc/mysql/mariadb.conf.d/61-galera.cnf` en los tres nodos con los parámetros `wsrep` necesarios, diferenciando únicamente `wsrep_node_address` y `wsrep_node_name` en cada uno.

Paso 6 — Inicializar el clúster (bootstrap desde `bdd-nodo01`).

Detener MariaDB en `nodo01` y arrancar el clúster con `galera_new_cluster`. En este momento existe un clúster de un solo nodo en estado Primary.

Paso 7 — Unir `bdd-nodo02` al clúster.

Arrancar MariaDB en `nodo02` con `systemctl start mariadb`. Galera detecta el clúster existente, solicita SST y se sincroniza automáticamente.

Paso 8 — Unir `bdd-nodo03` al clúster.

Igual que el paso anterior: arrancar MariaDB en `nodo03` y verificar la sincronización.

Paso 9 — Verificar el estado completo del clúster.

Consultar las variables `wsrep_*` en los tres nodos para confirmar: `wsrep_cluster_size = 3`, `wsrep_cluster_status = Primary`, `wsrep_local_state_comment = Synced` en todos.

Paso 10 — Demostrar escrituras multi-maestro.

Insertar registros en `nodo01` y verificar su presencia inmediata en `nodo02` y `nodo03`. Insertar en `nodo03` y verificar en `nodo01` y `nodo02`.

Paso 11 — Demostrar la detección de conflictos.

Abrir dos sesiones simultáneas, una en nodo01 y otra en nodo03, que modifiquen la misma fila. Observar el rollback automático de la transacción perdedora y verificar que todos los nodos ven el mismo resultado final (determinismo Galera).

Paso 12 — Simular caída y recuperación automática de un nodo.

Detener bruscamente nodo02, insertar datos con el clúster en estado de 2 nodos, reiniciar nodo02 y verificar que recuperó los datos faltantes por IST sin intervención manual.

Paso 13 — Apagar los tres nodos ordenadamente y tomar snapshots.

E. Comandos completos

Los comandos marcados **(host)** se ejecutan en PowerShell en Windows. Los marcados **(VM — nodoXX)** se ejecutan en SSH dentro del nodo indicado. Los bloques SQL dentro de `sudo mariadb` se pueden pegar directamente en el prompt del motor.

E.1 Crear `bdd-nodo03` como clon de `bdd-nodo01` (host)

PowerShell

```
# Verificar que todas las VMs relevantes están apagadas antes de clonar
VBoxManage list runningvms
# Clonar bdd-nodo01 desde el snapshot fase09-completa
# --options link crea un clon enlazado (usa el disco base del snapshot
de origen;
# solo ocupa espacio para los cambios diferenciales). Omitir --options
link para
# un clon completo e independiente (tarda más y ocupa más disco).
VBoxManage clonevm "bdd-nodo01" `
  --snapshot "fase09-completa" `
  --options link `
  --name "bdd-nodo03" `
  --basefolder "C:\LabBDD\VMs" `
  --register
# Confirmar que bdd-nodo03 aparece en la lista de VMs
VBoxManage list vms | Select-String "bdd-nodo03"
# Verificar que el adaptador de red del clon es Host-Only
VBoxManage showvminfo "bdd-nodo03" | Select-String "NIC 1"
# Si el adaptador no está configurado correctamente:
VBoxManage modifyvm "bdd-nodo03" `
  --nic1 hostonly `
  --hostonlyadapter1 "VirtualBox Host-Only Ethernet Adapter"
```

¿Por qué `fase09-completa` y no `fase10-completa` ? El snapshot `fase09-completa` contiene el esquema `lab_bdd` correctamente poblado (Fase 8) sin la configuración de maestro de la Fase 10, lo que da a `nodo03` una base completamente neutra para configurar Galera desde cero.

E.2 Iniciar `bdd-nodo03` y cambiar hostname e IP (host + VM — nodo03)

Arrancar `nodo03` en modo headless:

PowerShell

```
VBoxManage startvm "bdd-nodo03" --type headless
```

El clon hereda la IP `192.168.56.101` de `nodo01`, lo que provoca conflicto de IP. Acceder por la **consola de VirtualBox** (no por SSH todavía): en VirtualBox Manager, seleccionar `bdd-nodo03` → botón **Mostrar** (Show). Iniciar sesión con las credenciales de `bddadmin`.

Dentro de `bdd-nodo03` (consola VirtualBox):

 Bash

```
# ---- Cambiar hostname ----
sudo hostnamectl set-hostname bdd-nodo03
# ---- Identificar nombre de la interfaz de red ----
ip link show
# Normalmente: enp0s3 o enp0s8 (heredado del clon)
# ---- Cambiar la IP estática en Netplan ----
# El nombre del archivo puede variar; listar para encontrarlo:
ls /etc/netplan/
sudo nano /etc/netplan/00-installer-config.yaml
```

Contenido del archivo (ajustar el nombre de interfaz si difiere):

 YAML

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: false
      addresses:
        - 192.168.56.103/24
      routes:
        - to: default
          via: 192.168.56.1
      nameservers:
        addresses:
          - 8.8.8.8
          - 8.8.4.4
```

Aplicar la configuración de red:

 Bash

```
sudo netplan apply
# Verificar la nueva IP
ip addr show
hostname
```

La interfaz debe mostrar `192.168.56.103` y el hostname debe ser `bdd-nodo03`. Desde este momento SSH funciona desde el host:

 PowerShell

```
# Verificar desde el host
ssh bddadmin@192.168.56.103
```

E.3 Configurar `/etc/hosts` y `server_id` en `bdd-nodo03` (VM — nodo03)

 Bash

```
# Actualizar /etc/hosts para que el nodo conozca a todos los miembros
del clúster
sudo tee /etc/hosts > /dev/null << 'EOF'
127.0.0.1      localhost
127.0.1.1      bdd-nodo03
# Laboratorio BDD — clúster Galera
192.168.56.101 bdd-nodo01
192.168.56.102 bdd-nodo02
192.168.56.103 bdd-nodo03
EOF
# Verificar conectividad de red hacia los otros nodos
ping -c 3 192.168.56.101
ping -c 3 192.168.56.102
```

Cambiar el `server_id` de MariaDB a `3` (debe ser único en el clúster). El archivo de configuración de replicación se creó en la Fase 10; si no existe, crearlo:

 Bash

```
# Verificar si existe el archivo de configuración de la Fase 10
ls /etc/mysql/mariadb.conf.d/60-replication.cnf
# Editar (o crear) el archivo para establecer server_id = 3
sudo nano /etc/mysql/mariadb.conf.d/60-replication.cnf
```

El archivo debe contener como mínimo:

Text

```
[mariadb]
server_id      = 3
log_bin        = /var/log/mysql/mariadb-bin
binlog_format  = ROW
```

Guardar, reiniciar MariaDB y confirmar el `server_id`:

Bash

```
sudo systemctl restart mariadb
sudo mariadb -e "SELECT @@server_id AS server_id;"
```

La salida debe mostrar `3`.

E.4 Actualizar `/etc/hosts` en `bdd-nodo01` y `bdd-nodo02` (VM — nodo01 y nodo02)

Ejecutar en **cada** uno de los dos nodos existentes:

Bash

```
# Ajustar la segunda línea (127.0.1.1) según el hostname del nodo actual
sudo tee /etc/hosts > /dev/null << 'EOF'
127.0.0.1      localhost
127.0.1.1      bdd-nodo01
# Laboratorio BDD — clúster Galera
192.168.56.101 bdd-nodo01
192.168.56.102 bdd-nodo02
192.168.56.103 bdd-nodo03
EOF
```

En `bdd-nodo02`, cambiar la segunda línea a `127.0.1.1 bdd-nodo02`.

Verificar resolución de nombres:

 Bash

```
ping -c 2 bdd-nodo03
```

E.5 Instalar la librería Galera en los tres nodos (VM — nodo01, nodo02, nodo03)

Ejecutar en **cada uno de los tres nodos** (en tres terminales SSH simultáneas):

 Bash

```
# Actualizar lista de paquetes
sudo apt update
# Instalar la librería Galera 4
# En sistemas con MariaDB del repositorio oficial, galera-4 puede
# ya estar instalado como dependencia de mariadb-server.
sudo apt install -y galera-4
# Verificar que la librería existe en la ruta estándar
ls -lh /usr/lib/galera/libgalera_smm.so
```

Si el archivo no está en `/usr/lib/galera/`, ubicarlo con:

 Bash

```
find /usr -name "libgalera_smm.so" 2>/dev/null
```

Registrar la ruta exacta — se usará en el parámetro `wsrep_provider` del siguiente paso.

E.6 Limpiar la configuración de esclavo de la Fase 10 en `bdd-nodo02` (VM — nodo02)

 Bash

```
sudo mariadb << 'EOF'
-- Ver el estado actual del esclavo (documentar antes de eliminar)
SHOW SLAVE STATUS\G
-- Detener el agente de replicación esclavo
STOP SLAVE;
-- Eliminar TODA la configuración de esclavo (master.info, relay-log.info)
RESET SLAVE ALL;
-- Confirmar que ya no hay configuración activa
SHOW SLAVE STATUS\G
EOF
```

La última instrucción `SHOW SLAVE STATUS\G` debe devolver un conjunto vacío.

Eliminar también el parámetro `read_only` del archivo de configuración:

 Bash

```
sudo nano /etc/mysql/mariadb.conf.d/60-replication.cnf
```

Asegurarse de que el archivo contenga únicamente:

 Text

```
[mariadb]
server_id      = 2
log_bin        = /var/log/mysql/mariadb-bin
binlog_format  = ROW
# (sin read_only = ON)
```

| No reiniciar MariaDB todavía; se hará al aplicar la configuración Galera en E.9.

E.7 Crear el archivo de configuración Galera en bdd-nodo01 (VM — nodo01)

 Bash

```
sudo tee /etc/mysql/mariadb.conf.d/61-galera.cnf > /dev/null << 'EOF'
# =====
# Configuración Galera Cluster — bdd-nodo01
# Creado en la Fase 11.
# Cargado después de 60-replication.cnf (que ya
# establece binlog_format=ROW).
# =====
[mariadb]
# ---- Activar el protocolo wsrep ----
wsrep_on = ON
wsrep_provider = /usr/lib/galera/libgalera_smm.so
wsrep_provider_options = "pc.recovery=YES; gcache.size=256M"
# ---- Identificación del clúster ----
wsrep_cluster_name = lab_bdd_cluster
wsrep_cluster_address =
gcomm://192.168.56.101,192.168.56.102,192.168.56.103
# ---- Identificación de este nodo ----
wsrep_node_address = 192.168.56.101
wsrep_node_name = bdd-nodo01
# ---- Método de sincronización inicial (SST) ----
# rsync: simple, adecuado para este laboratorio.
# Alternativa de producción: wsrep_sst_method = mariabackup
wsrep_sst_method = rsync
# ---- Hilos para aplicar write-sets recibidos ----
wsrep_slave_threads = 4
# ---- Requerimiento de InnoDB para Galera ----
# innodb_autoinc_lock_mode=2: necesario para auto_increment en
multi-maestro
innodb_autoinc_lock_mode = 2
# ---- Control automático de auto_increment para evitar colisiones de PK -
---
wsrep_auto_increment_control = ON
# Nota: binlog_format = ROW ya está definido en 60-replication.cnf.
# Galera requiere ROW; no duplicar aquí para evitar advertencias
de conflicto.
EOF
# Verificar que no hay errores de sintaxis
cat /etc/mysql/mariadb.conf.d/61-galera.cnf
```

E.8 Crear el archivo de configuración Galera en `bdd-nodo02` (VM — nodo02)

 Bash

```
sudo tee /etc/mysql/mariadb.conf.d/61-galera.cnf > /dev/null << 'EOF'
# =====
# Configuración Galera Cluster — bdd-nodo02
# Creado en la Fase 11.
# =====
[mariadb]
wsrep_on                = ON
wsrep_provider           = /usr/lib/galera/libgalera_smm.so
wsrep_provider_options   = "pc.recovery=YES; gcache.size=256M"
wsrep_cluster_name       = lab_bdd_cluster
wsrep_cluster_address    =
gcomm://192.168.56.101,192.168.56.102,192.168.56.103
# ---- Solo estas dos líneas difieren de nodo01 ----
wsrep_node_address       = 192.168.56.102
wsrep_node_name          = bdd-nodo02
wsrep_sst_method         = rsync
wsrep_slave_threads      = 4
innodb_autoinc_lock_mode = 2
wsrep_auto_increment_control = ON
EOF
```

E.9 Crear el archivo de configuración Galera en `bdd-nodo03` (VM — nodo03)

 Bash

```
sudo tee /etc/mysql/mariadb.conf.d/61-galera.cnf > /dev/null << 'EOF'
# =====
# Configuración Galera Cluster — bdd-nodo03
# Creado en la Fase 11.
# =====
[mariadb]
wsrep_on                = ON
wsrep_provider           = /usr/lib/galera/libgalera_smm.so
wsrep_provider_options   = "pc.recovery=YES; gcache.size=256M"
wsrep_cluster_name       = lab_bdd_cluster
wsrep_cluster_address    =
gcomm://192.168.56.101,192.168.56.102,192.168.56.103
# ---- Solo estas dos líneas difieren de nodo01 ----
wsrep_node_address       = 192.168.56.103
wsrep_node_name          = bdd-nodo03
wsrep_sst_method         = rsync
wsrep_slave_threads      = 4
innodb_autoinc_lock_mode = 2
wsrep_auto_increment_control = ON
EOF
```

E.10 Verificar y abrir puertos Galera si el firewall está activo (VM — los tres nodos)

 Bash

```
# Comprobar si ufw está activo (Ubuntu 24.04 lo instala pero inactivo
por defecto)
sudo ufw status
# Si el resultado es "Status: inactive" no se necesita hacer nada.
# Si está activo, abrir los puertos Galera en CADA nodo:
sudo ufw allow 3306/tcp    comment 'MariaDB cliente'
sudo ufw allow 4567/tcp    comment 'Galera cluster wsrep'
sudo ufw allow 4567/udp    comment 'Galera cluster wsrep UDP'
sudo ufw allow 4568/tcp    comment 'Galera IST'
sudo ufw allow 4444/tcp    comment 'Galera SST rsync'
sudo ufw reload
```

E.11 Inicializar el clúster — Bootstrap desde `bdd-nodo01` (VM — nodo01)

 Bash

```
# Detener el servicio MariaDB estándar
sudo systemctl stop mariadb
# Inicializar el clúster Galera.
# galera_new_cluster arranca MariaDB con wsrep_cluster_address='gcomm://'
# (cadena vacía), señal para Galera de que este nodo es el primero
del clúster
# y debe inicializarlo con un nuevo UUID de clúster.
sudo galera_new_cluster
# Verificar que el servicio arrancó correctamente
sudo systemctl status mariadb --no-pager
```

Verificar el estado del clúster en nodo01 (en este punto solo hay 1 nodo):

 Bash

```
sudo mariadb -e "
SHOW STATUS WHERE Variable_name IN (
    'wsrep_cluster_size',
    'wsrep_cluster_status',
    'wsrep_connected',
    'wsrep_ready',
    'wsrep_local_state_comment',
    'wsrep_node_name'
);
"
```

Salida esperada:

Text

Variable_name	Value
wsrep_cluster_size	1
wsrep_cluster_status	Primary
wsrep_connected	ON
wsrep_ready	ON
wsrep_local_state_comment	Synced
wsrep_node_name	bdd-nodo01

Alternativa:

SQL

```
SELECT 'wsrep_node_name' AS Variable_name,
       @@wsrep_node_name AS Value
UNION ALL
SELECT VARIABLE_NAME, VARIABLE_VALUE
FROM information_schema.GLOBAL_STATUS
WHERE VARIABLE_NAME IN (
    'WSREP_CLUSTER_SIZE',
    'WSREP_CLUSTER_STATUS',
    'WSREP_CONNECTED',
    'WSREP_READY',
    'WSREP_LOCAL_STATE_COMMENT'
);
```

Salida esperada:

Text

Variable_name	Value
wsrep_node_name	bdd-nodo01
WSREP_LOCAL_STATE_COMMENT	Synced
WSREP_CLUSTER_SIZE	1
WSREP_CLUSTER_STATUS	Primary
WSREP_CONNECTED	ON
WSREP_READY	ON

`wsrep_cluster_size = 1` es correcto en este punto: solo nodo01 está en el clúster. Un valor de `wsrep_cluster_status = Primary` con un solo nodo es normal y esperado durante el bootstrap.

E.12 Unir `bdd-nodo02` al clúster (VM — nodo02)

Bash

```
# Arrancar MariaDB normalmente (NO usar galera_new_cluster -
# ese comando crea un clúster nuevo; aquí queremos UNIRSE al existente).
sudo systemctl start mariadb
# Observar el log en tiempo real durante el SST (puede tardar 30-120 s)
sudo journalctl -u mariadb -f --no-pager | grep -
E "WSREP|SST|Synced|state"
# Ctrl+C para salir cuando aparezca "WSREP: Synchronized with group"
```

Verificar que nodo02 se unió correctamente:

 Bash

```
sudo mariadb -e "  
SHOW STATUS WHERE Variable_name IN (  
    'wsrep_cluster_size',  
    'wsrep_cluster_status',  
    'wsrep_local_state_comment',  
    'wsrep_node_name'  
);  
"
```

Salida esperada en nodo02:

 Text

```
+-----+-----+  
| Variable_name          | Value          |  
+-----+-----+  
| wsrep_cluster_size     | 2              |  
| wsrep_cluster_status   | Primary        |  
| wsrep_local_state_comment | Synced         |  
| wsrep_node_name        | bdd-nodo02     |  
+-----+-----+
```

Confirmar desde nodo01 que el clúster creció:

 Bash

```
# En bdd-nodo01  
sudo mariadb -e "SHOW STATUS LIKE 'wsrep_cluster_size';"
```

E.13 Unir `bdd-nodo03` al clúster (VM — nodo03)

 Bash

```
sudo systemctl start mariadb  
# Monitorear el progreso del SST  
sudo journalctl -u mariadb -f --no-pager | grep -E "WSREP|SST|IST|Synced"  
# Ctrl+C al aparecer "Synchronized with group"
```

Verificar el estado final:

 Bash

```
sudo mariadb -e "  
SHOW STATUS WHERE Variable_name IN (  
    'wsrep_cluster_size',  
    'wsrep_cluster_status',  
    'wsrep_local_state_comment',  
    'wsrep_node_name'  
);  
"
```

Salida esperada en nodo03:

 Text

```
+-----+-----+  
| Variable_name          | Value          |  
+-----+-----+  
| wsrep_cluster_size     | 3              |  
| wsrep_cluster_status   | Primary        |  
| wsrep_local_state_comment | Synced         |  
| wsrep_node_name         | bdd-nodo03     |  
+-----+-----+
```

E.14 Verificación completa del estado del clúster en los tres nodos (VM)

Ejecutar en **cada uno de los tres nodos**:

 Bash

```
sudo mariadb << 'EOF'
-- Estado global del clúster
SELECT VARIABLE_NAME, VARIABLE_VALUE AS valor
FROM information_schema.GLOBAL_STATUS
WHERE VARIABLE_NAME IN (
    'WSREP_CLUSTER_SIZE',
    'WSREP_CLUSTER_STATUS',
    'WSREP_CONNECTED',
    'WSREP_READY',
    'WSREP_LOCAL_STATE_COMMENT',
    'WSREP_NODE_NAME',
    'WSREP_FLOW_CONTROL_PAUSED',
    'WSREP_CERT_DEPS_DISTANCE'
)
ORDER BY VARIABLE_NAME;
-- Miembros actuales del clúster (tabla disponible solo con Galera activo)
SELECT * FROM information_schema.WSREP_MEMBERSHIP
ORDER BY INDEX;
-- Verificar que auto_increment fue ajustado automáticamente por Galera
SHOW VARIABLES LIKE 'auto_increment_increment';
SHOW VARIABLES LIKE 'auto_increment_offset';
-- Confirmar que lab_bdd tiene los datos correctos
SELECT 'clientes' AS tabla, COUNT(*) AS registros FROM
lab_bdd.clientes UNION ALL
SELECT 'productos', COUNT(*) FROM
lab_bdd.productos UNION ALL
SELECT 'pedidos', COUNT(*) FROM
lab_bdd.pedidos UNION ALL
SELECT 'detalle_pedidos', COUNT(*) FROM
lab_bdd.detalle_pedidos;
EOF
```

`information_schema.WSREP_MEMBERSHIP` debe mostrar exactamente 3 filas con los nombres `bdd-nodo01`, `bdd-nodo02`, `bdd-nodo03`, sus IPs y estado `Synced`. Esta tabla solo existe cuando Galera está activo.

E.15 Demostrar escrituras multi-maestro (VM — nodo01 y nodo03)

Prueba A: escritura en nodo01, lectura inmediata en nodo02 y nodo03.

En `bdd-nodo01`:

 Bash

```
sudo mariadb lab_bdd << 'EOF'
INSERT INTO clientes (nombre, apellido, email, telefono, region, ciudad)
VALUES ('Prueba', 'GaleraA', 'prueba.galera.a@lab.test',
        '5500000021', 'norte', 'Ciudad de México');
SELECT id, nombre, apellido, email FROM clientes
WHERE email = 'prueba.galera.a@lab.test';
EOF
```

Verificar inmediatamente en nodo02 y nodo03 (la replicación es síncrona — no hay lag):

 Bash

```
# Ejecutar en bdd-nodo02 Y en bdd-nodo03
sudo mariadb lab_bdd -e "
SELECT id, nombre, apellido, email, ciudad
FROM clientes WHERE email = 'prueba.galera.a@lab.test';
"
```

El registro debe aparecer en los tres nodos con el **mismo** `id`.

Prueba B: escritura directamente en nodo03.

 Bash

```
# En bdd-nodo03
sudo mariadb lab_bdd << 'EOF'
INSERT INTO clientes (nombre, apellido, email, telefono, region, ciudad)
VALUES ('Prueba', 'GaleraC', 'prueba.galera.c@lab.test',
        '5500000022', 'sur', 'Guadalajara');
SELECT id, nombre, apellido, email FROM clientes
WHERE email = 'prueba.galera.c@lab.test';
EOF
```

Verificar en nodo01 y nodo02:

 Bash

```
# En bdd-nodo01 Y en bdd-nodo02
sudo mariadb lab_bdd -e "
SELECT id, nombre, apellido, email
FROM clientes WHERE email LIKE 'prueba.galera.%'
ORDER BY id;
"
```

Ambos registros (`.a` y `.c`) deben aparecer en los tres nodos. Verificar también el incremento automático de PK por nodo:

 Bash

```
# En cualquier nodo
sudo mariadb -e "
SHOW VARIABLES LIKE 'auto_increment_increment';
SHOW VARIABLES LIKE 'auto_increment_offset';
"
```

Con un clúster de 3 nodos y `wsrep_auto_increment_control = ON`, la salida típica muestra `auto_increment_increment = 3`. El offset varía: `1` en nodo01, `2` en nodo02, `3` en nodo03.

E.16 Demostrar detección de conflictos (VM — nodo01 y nodo03 simultáneamente)

Este experimento requiere **dos terminales SSH abiertas al mismo tiempo**: una a nodo01 y otra a nodo03.

Preparar la fila de prueba:

 Bash

```
# En cualquier nodo — identificar la fila que se usará
sudo mariadb lab_bdd -e "SELECT id, nombre, ciudad FROM clientes WHERE id
= 1;"
```

Terminal A — sesión en nodo01 (iniciar primero):

 Bash

```
sudo mariadb lab_bdd
```

Dentro del prompt de MariaDB en nodo01:

 SQL

```
-- La transacción modifica la fila id=1 localmente y luego espera
10 segundos
-- antes de intentar COMMIT; ese tiempo es la ventana para que
nodo03 cometa.
START TRANSACTION;
UPDATE clientes SET ciudad = 'Ciudad_desde_nodo01' WHERE id = 1;
SELECT SLEEP(10);
COMMIT;
```

Terminal B — sesión en nodo03 (ejecutar MIENTRAS el SLEEP corre en Terminal A):

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- Nodo03 modifica la misma fila y confirma ANTES de que nodo01 salga
del SLEEP
START TRANSACTION;
UPDATE clientes SET ciudad = 'Ciudad_desde_nodo03' WHERE id = 1;
COMMIT;
-- Esta transacción debe TENER ÉXITO (se certifica primero)
SELECT 'Transaccion nodo03: EXITOSA' AS resultado;
EOF
```

Resultado esperado: la sesión en nodo01 recibirá un error similar a:

 Text

```
ERROR 1213 (40001): Deadlock found when trying to get lock;
try restarting transaction
```

Verificar que todos los nodos tienen el **mismo valor final** (determinismo de Galera):

 Bash

```
# En los TRES nodos - deben devolver EXACTAMENTE el mismo resultado
sudo mariadb lab_bdd -e "SELECT id, nombre, ciudad FROM clientes WHERE id
= 1;"
```

Solo uno de los dos valores (`Ciudad_desde_nodo01` o `Ciudad_desde_nodo03`) estará presente, y será el **mismo en los tres nodos**. Este es el determinismo de Galera: el conflicto se resuelve de forma consistente globalmente.

Restaurar el valor original:

 Bash

```
sudo mariadb lab_bdd -e "UPDATE clientes SET ciudad = 'Monterrey' WHERE id
= 1;"
```

E.17 Simular caída y recuperación automática de un nodo (host + VM)

Paso 1: Verificar estado inicial (3 nodos).

 Bash

```
# En cualquier nodo
sudo mariadb -e "SHOW STATUS LIKE 'wsrep_cluster_size';"
# → debe devolver 3
```

Paso 2: Simular caída abrupta de `bdd-nodo02` .

 PowerShell

```
# Desde el host Windows (simula fallo de hardware, sin apagado limpio)
VBoxManage controlvm "bdd-nodo02" poweroff
```

Paso 3: Verificar que el clúster sigue operando con 2 nodos.

 Bash

```
# En bdd-nodo01 o bdd-nodo03
sudo mariadb -e "SHOW STATUS LIKE 'wsrep_cluster_size';"      # → 2
sudo mariadb -e "SHOW STATUS LIKE 'wsrep_cluster_status';"    # → Primary
```

Paso 4: Insertar datos mientras nodo02 está caído.

 Bash

```
# En bdd-nodo01
sudo mariadb lab_bdd -e "
INSERT INTO clientes (nombre, apellido, email, telefono, region, ciudad)
VALUES ('Prueba', 'Recuperacion', 'prueba.recovery@lab.test',
        '5500000023', 'este', 'Veracruz');
SELECT 'Registro insertado con nodo02 caido' AS mensaje,
       LAST_INSERT_ID() AS id_asignado;
"
```

Paso 5: Reiniciar `bdd-nodo02` y observar la rejoinura automática.

 PowerShell

```
# Desde el host
VBoxManage startvm "bdd-nodo02" --type headless
```

Esperar 30–60 segundos. Monitorear los logs de nodo02:

 Bash

```
# En bdd-nodo02 (cuando SSH esté disponible)
sudo journalctl -u mariadb -f --no-pager | grep -E
"WSREP|IST|SST|Synced|joining"
# Buscar: "WSREP: Synchronized with group, ready for connections"
```

Paso 6: Verificar que nodo02 recuperó el registro insertado durante la caída.

 Bash

```
# En bdd-nodo02
sudo mariadb lab_bdd -e "
SELECT id, nombre, apellido, email, ciudad
FROM clientes WHERE email = 'prueba.recovery@lab.test';
"

sudo mariadb -e "SHOW STATUS LIKE 'wsrep_cluster_size';"
# → debe volver a 3
```

E.18 Apagar los tres nodos ordenadamente y tomar snapshots (host + VM)

El orden de apagado de un clúster Galera importa: apagar los joiners primero y el bootstrapper (o el nodo con mayor `seqno`) al final. El último en apagarse limpiamente tendrá

`safe_to_bootstrap: 1` en su archivo `grastate.dat`.

 Bash

```
# En bdd-nodo03 (apagar primero)
sudo systemctl stop mariadb && sudo poweroff
# En bdd-nodo02 (apagar segundo)
sudo systemctl stop mariadb && sudo poweroff
# En bdd-nodo01 (apagar ÚLTIMO)
sudo systemctl stop mariadb && sudo poweroff
```

Confirmar desde el host:

 PowerShell

```
VBoxManage list runningvms
# La salida debe estar vacía
```

Tomar los snapshots de cierre de fase:

PowerShell

```
VBoxManage snapshot "bdd-nodo01" take "fase11-completa" `
  --description "Galera Cluster activo (nodo01 bootstrapper). lab_bdd con 22
clientes. wsrep_cluster_size=3, status=Primary. Binary log conservado para
async downstream."
VBoxManage snapshot "bdd-nodo02" take "fase11-completa" `
  --description "Galera Cluster miembro. Configuración esclavo Fase10
eliminada. Sincronizado via SST. lab_bdd verificada."
VBoxManage snapshot "bdd-nodo03" take "fase11-completa" `
  --description "Tercer miembro Galera. Creado como clon de nodo01@fase09-
completa. IP .103, server_id=3. lab_bdd sincronizada por Galera."
```

Confirmar los snapshots:

PowerShell

```
VBoxManage snapshot "bdd-nodo01" list
VBoxManage snapshot "bdd-nodo02" list
VBoxManage snapshot "bdd-nodo03" list
```

`bdd-nodo01` y `bdd-nodo02` deben mostrar los snapshots `fase05-completa` a `fase11-completa`. `bdd-nodo03` debe mostrar únicamente `fase11-completa` (su primer snapshot).

F. Verificación de funcionamiento

Esta fase se considera completa cuando se cumplen **todos** los puntos siguientes con el clúster activo:

1. Los tres nodos responden a `ssh bddadmin@192.168.56.10X` desde el host Windows sin errores.
2. `SHOW STATUS LIKE 'wsrep_cluster_size';` devuelve **3** en los **tres nodos**.
3. `SHOW STATUS LIKE 'wsrep_cluster_status';` devuelve **Primary** en los **tres nodos**.
4. `SHOW STATUS WHERE Variable_name = 'wsrep_local_state_comment';` devuelve **Synced** en los **tres nodos**.
5. `SELECT * FROM information_schema.WSREP_MEMBERSHIP;` muestra exactamente 3 filas con los nombres `bdd-nodo01`, `bdd-nodo02`, `bdd-nodo03` y sus IPs correctas.
6. El conteo de registros en `lab_bdd` es **idéntico** en los tres nodos: ≥22 clientes (20 originales + los insertados en las pruebas), 10 productos, 20 pedidos, 36 detalles.
7. Un `INSERT` ejecutado en nodo01 es **inmediatamente visible** en nodo02 y nodo03 sin ningún comando adicional.

8. Un `INSERT` ejecutado en nodo03 es **inmediatamente visible** en nodo01 y nodo02.
 9. El experimento de conflicto (E.16) produjo exactamente **un error** en una de las dos sesiones y el valor de la columna `ciudad` del cliente `id = 1` es **el mismo** en los tres nodos (resultado determinista).
 10. `SHOW VARIABLES LIKE 'auto_increment_increment';` devuelve **3** en los tres nodos.
 11. El experimento de caída de nodo02 (E.17) mostró: `wsrep_cluster_size = 2` durante la caída; `wsrep_cluster_status = Primary` con 2 nodos; el registro `prueba.recovery@lab.test` aparece en nodo02 tras su reinicio; `wsrep_cluster_size = 3` tras la recuperación.
 12. `SHOW VARIABLES LIKE 'wsrep_on';` devuelve **ON** en los tres nodos.
 13. Los snapshots `fase11-completa` existen en los **tres nodos**.
 14. El estudiante puede explicar de memoria la diferencia entre SST e IST y cuándo Galera elige cada mecanismo.
 15. El estudiante puede explicar qué es el Componente Primario y qué ocurre con un nodo que pierde quórum.
-

G. Problemas comunes y soluciones

Problema	Causa probable	Solución
<pre>galera_new_cluster</pre> falla con <pre>ERROR: MySQL is running, please stop it first</pre>	MariaDB sigue activo	Ejecutar <code>sudo systemctl stop mariadb</code> y luego <code>sudo galera_new_cluster</code>
El nodo joiner permanece en <code>wsrep_local_state_comment = Joining</code> sin avanzar	Falla de conectividad en el puerto 4567 entre los nodos	Verificar <code>ping</code> entre VMs; si <code>ufw</code> está activo, ejecutar E.10; probar con <code>nc -zv 192.168.56.101 4567</code> desde el joiner
El SST tarda más de 5 minutos	Con <code>rsync</code> , el SST es una copia completa; con mucha actividad en el donador puede ser lento	Es normal; esperar. Para datos crecientes, cambiar a <code>wsrep_sst_method = mariabackup</code> (requiere instalar <code>mariadb-backup</code>)
<pre>galera_new_cluster</pre> muestra <pre>WSREP: It may not be safe to bootstrap...</pre>	Galera detecta datos de un clúster anterior y teme un split-brain	Verificar que los otros nodos están completamente detenidos; editar <code>/var/lib/mysql/grastate.dat</code> en nodo01 y cambiar <code>safe_to_bootstrap: 0</code> a <code>safe_to_bootstrap: 1</code> , luego reintentar

MariaDB no arranca después de agregar <code>61-galera.cnf</code>	Error de sintaxis en el archivo o conflicto de parámetros entre archivos	Revisar el error con <code>sudo journalctl -u mariadb -n 30 --no-pager</code> ; buscar <code>unknown variable</code> o <code>conflicts</code> with para identificar el parámetro problemático
<code>wsrep_cluster_size</code> sigue en 1 después de unir el segundo nodo	Los nodos no se alcanzan en el puerto wsrep 4567 aunque SSH (22) sí funcione	Verificar que <code>wsrep_node_address</code> en <code>61-galera.cnf</code> coincide exactamente con la IP de la interfaz Host-Only; revisar ufw
El experimento de conflicto no genera error: ambas transacciones confirman	El SLEEP fue insuficiente; nodo01 confirmó antes de que nodo03 empezara	Aumentar el SLEEP a 15 segundos; asegurarse de que la sesión en nodo03 inició el UPDATE ANTES de que nodo01 llamara al COMMIT
<code>wsrep_cluster_status = non-Primary</code> en un nodo tras reinicio	El nodo perdió contacto con la mayoría del clúster	Si los otros dos nodos están <code>Primary</code>, el nodo aislado se reconectará automáticamente e al recuperar la red
Tras una caída total (todos los nodos apagados sin orden), ningún nodo arranca el clúster	Galera no sabe cuál nodo tiene el estado más reciente	Revisar <code>/var/lib/mysql/grastate.dat</code> en cada nodo; el que tenga <code>seqno</code> más alto Y

		<pre>safe_to_bootstrap: 1 debe ejecutar galera_new_cluster ; los demás luego con systemctl start mariadb</pre>
El clon de nodo03 tiene el mismo <code>/etc/machine-id</code> que nodo01	VirtualBox regenera el UUID de VM pero no el ID del sistema operativo	En nodo03: <pre>sudo rm /etc/machine-id && sudo systemd-machine-id-setup y reiniciar</pre>
<code>auto_increment_increment</code> sigue en 1 después de activar Galera	<code>wsrep_auto_increment_control = ON</code> no está en <code>61-galera.cnf</code> o el archivo no se cargó	Verificar con <pre>SHOW VARIABLES LIKE 'wsrep_auto_increment_control';</pre> si devuelve <code>OFF</code> , agregar el parámetro al archivo y reiniciar MariaDB

H. Checklist de validación

- ☐ `bdd-nodo03` existe en VirtualBox con IP `192.168.56.103`, hostname `bdd-nodo03` y acceso SSH desde el host.
- ☐ `bdd-nodo03` tiene `server_id = 3` confirmado con `SELECT @@server_id;`.
- ☐ El paquete `galera-4` está instalado y `/usr/lib/galera/libgalera_smm.so` existe en los **tres nodos**.
- ☐ `/etc/mysql/mariadb.conf.d/61-galera.cnf` existe en los **tres nodos** con `wsrep_node_address` y `wsrep_node_name` únicos para cada nodo.
- ☐ La configuración de esclavo de la Fase 10 fue eliminada de `bdd-nodo02`: `RESET SLAVE ALL` ejecutado y `read_only` eliminado del archivo de configuración.
- ☐ El bootstrap se completó en `bdd-nodo01` sin errores y el servicio `mariadb` está activo.

- ☐ `bdd-nodo02` y `bdd-nodo03` se unieron al clúster; sus logs muestran `Synchronized with group`.
- ☐ `SHOW STATUS LIKE 'wsrep_cluster_size';` devuelve **3** en los **tres nodos**.
- ☐ `SHOW STATUS LIKE 'wsrep_cluster_status';` devuelve **Primary** en los **tres nodos**.
- ☐ `wsrep_local_state_comment = Synced` en los **tres nodos**.
- ☐ `information_schema.WSREP_MEMBERSHIP` muestra 3 filas con nodos, IPs y nombres correctos.
- ☐ El conteo de registros en `lab_bdd` es idéntico en los **tres nodos**.
- ☐ Se demostró inserción en nodo01 visible inmediatamente en nodo02 y nodo03.
- ☐ Se demostró inserción en nodo03 visible inmediatamente en nodo01 y nodo02.
- ☐ El experimento de conflicto generó exactamente un error en una de las dos sesiones.
- ☐ El valor del cliente `id = 1` es idéntico en los tres nodos tras el experimento de conflicto.
- ☐ El experimento de caída de nodo02 mostró `wsrep_cluster_size = 2` durante la caída y **3** tras la recuperación.
- ☐ El registro `prueba.recovery@lab.test` aparece en nodo02 tras su reinicio, sin intervención manual.
- ☐ `SHOW VARIABLES LIKE 'auto_increment_increment';` devuelve **3** en los tres nodos.
- ☐ Los snapshots `fase11-completa` existen en los **tres nodos**.
- ☐ Puedo explicar la diferencia entre SST e IST y cuándo Galera elige cada uno.
- ☐ Puedo explicar qué es el Componente Primario y qué ocurre cuando un nodo pierde quórum.

Preguntas teóricas para estudiantes

1. La replicación de Galera se denomina “síncrona por certificación” pero **no** usa commit de dos fases clásico (2PC) ni locks distribuidos durante la ejecución de la transacción. Explica en detalle: ¿qué información contiene exactamente un *write-set*?, ¿qué condición verifica cada nodo durante la certificación?, y ¿por qué este mecanismo puede garantizar consistencia global sin necesitar que los nodos se comuniquen durante la ejecución de la transacción (solo al momento del `COMMIT`)?
2. Un clúster de 3 nodos tolera la caída de 1 nodo simultáneo. Si el sistema de producción necesitara tolerar 2 nodos caídos simultáneamente, ¿cuántos nodos totales se requerirían y por qué? Generaliza la fórmula para una tolerancia a `f` fallos y explica por qué Galera (y cualquier sistema basado en quórum) opera correctamente con número impar de nodos pero tiene problemas estructurales con número par.

3. En el experimento de conflicto (sección E.16), la transacción perdedora recibió un error de tipo *deadlock*. Desde el punto de vista del código de una aplicación que usa este clúster, ¿qué estrategia de manejo de errores es necesaria para que las escrituras sean robustas ante conflictos de certificación? ¿Es suficiente capturar el error y reintentar sin más? ¿Qué cuidados especiales hay que tener en el reintento para no violar la lógica de negocio?
 4. `wsrep_auto_increment_control = ON` hace que Galera ajuste `auto_increment_increment` y `auto_increment_offset` según el número de nodos. ¿Qué ocurre internamente cuando un cuarto nodo se une al clúster en caliente (sin reiniciar los otros nodos)? ¿cómo cambian estos valores en todos los nodos?, ¿qué sucede con las filas ya insertadas que tienen IDs generados con el patrón de 3 nodos?, ¿produce esto algún hueco en la secuencia y es eso un problema?
 5. Dado el diseño completo del laboratorio (Galera en nodo01–03, fragmentación en nodo04–05, coordinador Spider en nodo06), identifica **dos escenarios de partición de red distintos** y explica para cada uno qué dilema CAP presenta: ¿qué parte del sistema elegirá Consistencia (dejando de aceptar escrituras) y qué parte elegirá Disponibilidad (aceptando escrituras con riesgo de inconsistencia)?
-

Ejercicios prácticos

1. Comparación de latencia: asíncrona (Fase 10) vs. síncrona (Galera).

Usando los snapshots `fase10-completa` de nodo01 y nodo02 (desde una sesión VirtualBox paralela, sin afectar el clúster activo), diseñar un experimento que inserte 100 filas en el maestro de la Fase 10 y mida el tiempo promedio entre el `COMMIT` en el maestro y la disponibilidad del dato en el esclavo (via `SELECT COUNT(*)` iterativo). Realizar la misma medición en el clúster Galera activo. Comparar los resultados y explicar qué significa la diferencia para una aplicación que necesita leer inmediatamente después de escribir.

2. Script de monitoreo de salud del clúster Galera.

Escribir un script Bash (`/opt/lab_bdd_backups/galera_health.sh`) que se conecte a MariaDB y genere un reporte con: tamaño del clúster, estado de cada nodo (`wsrep_local_state_comment`), bytes replicados acumulados, transacciones en cola (`wsrep_local_send_queue`) y un semáforo basado en las condiciones: **verde** = 3 nodos Primary Synced; **amarillo** = 2 nodos Primary; **rojo** = menos de 2 nodos o estado non-Primary. El script debe poder ejecutarse desde cualquier nodo e indicar desde cuál nodo reporta.

3. Análisis del archivo `grastate.dat` antes y después de un apagado ordenado.

Antes de apagar el clúster (con `sudo systemctl stop mariadb`), registrar el contenido de `/var/lib/mysql/grastate.dat` en los tres nodos. Luego apagar en el orden correcto (nodo03, nodo02, nodo01) y volver a leer el archivo en cada nodo. Comparar los valores

de `uuid`, `seqno` y `safe_to_bootstrap` antes y después del apagado. Documentar qué nodo tiene `safe_to_bootstrap: 1` y explicar el procedimiento correcto para recuperar el clúster tras una caída total inesperada (todos los nodos apagados de golpe).

Reto adicional para alumnos avanzados

Configurar un **esclavo asíncrono tradicional** apuntando a `bdd-nodo01` (miembro del clúster Galera) desde un cuarto nodo hipotético creado como clon limpio. Las escrituras al clúster se harían con las garantías síncronas de Galera, y la réplica asíncrona serviría para offloading de respaldos. El reto requiere: (a) investigar y documentar el problema de los **GTIDs de Galera** (`wsrep_gtid_mode`) vs los GTIDs de MariaDB y cómo coexisten; (b) configurar nodo01 para emitir binary log con una configuración compatible; © verificar que una escritura en cualquier nodo del clúster llega eventualmente al esclavo async; y (d) elaborar un diagrama de flujo completo de una transacción desde el cliente hasta el esclavo async, indicando en qué puntos la propagación es síncrona y en cuáles es asíncrona, y qué datos podría perder el esclavo async en caso de fallo de nodo01 que no perdería ningún miembro Galera.

Criterios de evaluación para el profesor

Criterio	Peso	Indicador de logro
Creación y configuración de <code>bdd-nodo03</code>	10%	El nodo existe con IP <code>.103</code> , hostname correcto, <code>server_id = 3</code> y <code>61-galera.cnf</code> con parámetros únicos; clon tomado de <code>fase09-completa</code>
Configuración Galera en los tres nodos	20%	<code>61-galera.cnf</code> existe en los tres nodos sin errores de sintaxis; MariaDB arranca con Galera habilitado; limpieza de esclavo de Fase 10 realizada en nodo02
Estado del clúster verificado	25%	<code>wsrep_cluster_size = 3</code> , <code>wsrep_cluster_status = Primary</code> y <code>wsrep_local_state_comment = Synced</code> en los tres nodos simultáneamente; <code>WSREP_MEMBERSHIP</code> <code>IP</code> muestra los tres nodos
Demostración multi-maestro y conflictos	20%	Se demuestra escritura exitosa desde nodo01 visible en

		nodo03, y desde nodo03 visible en nodo01; el experimento de conflicto produce exactamente un rollback con resultado idéntico en los tres nodos
Caída y recuperación de nodo	15%	<code>wsrep_cluster_size = 2</code> durante la caída, <code>= 3</code> tras la recuperación; el registro insertado durante la caída aparece en nodo02 sin intervención manual
Comprensión conceptual (preguntas teóricas)	10%	Las respuestas distinguen correctamente la certificación Galera del 2PC clásico, calculan nodos necesarios para tolerancia a fallos y discuten la estrategia de reintento con vocabulario técnico apropiado

Preparación para la siguiente fase

La **Fase 12: Particionamiento** (técnica de organización interna de datos dentro de un nodo, puente conceptual hacia la fragmentación horizontal de la Fase 13) requerirá:

- El clúster Galera de tres nodos en estado `fase11-completa` (esta fase).

- El esquema `lab_bdd` con datos de prueba intactos y consistentes en los tres nodos.
- Acceso SSH y DBeaver funcionando hacia los tres nodos (nodo03 se agrega ahora como cuarta conexión con túnel SSH, siguiendo el mismo procedimiento de la Fase 8).
- Comprensión clara de la diferencia entre **particionamiento** (dividir filas dentro de un mismo nodo usando la directiva `PARTITION BY` de MariaDB, sin mover datos entre servidores) y **fragmentación** (distribuir datos entre nodos distintos, que se implementa en la Fase 13). Esa distinción es el hilo conductor de la Fase 12.

Cualquier operación DDL (`CREATE TABLE` , `ALTER TABLE ... PARTITION BY`) que se ejecute en un nodo del clúster Galera se propagará automáticamente a los otros dos, lo que permite verificar que el DDL de particionamiento se replica correctamente — una verificación adicional de que el clúster sigue funcionando bien después de esta fase.
